

July 27th, 2026

Hospital Fixed-Asset Tracking System System Design Document

Imperial Healthtech

Document Final Version

Supervisors: Choirunnisa Hapsari
Kemal Falatehan

Author: Farros Ramzy

Abstract

This document presents the software and system design of the Hospital Fixed-Asset Tracking System. The system uses UHF RFID tags, an ESP32-based Registration Desk, and Checkpoint Nodes. The system uses MQTT communication, a FastAPI backend, PostgreSQL data storage, and browser-based applications.

The document does not replace the System Requirements and Use Case Specification. Instead, it explains how the requirements and use cases are realized through architecture, subsystem responsibilities, class structure, data model, sequence behavior, state-machine behavior, communication interfaces, security design, deployment design, and testing design.

The design explains how the major system components are structured, how they communicate, how data is stored, how key workflows behave over time, and how the product is deployed and verified.

Version Table

Version	Date of Creation / Change	Changes	Reviewers	Approval Status
0.0	30/03/2026	Document Initiation: adding the abstract and the introduction of the design.		
0.0.1	31/03/2026	Adding the system overview and High-Level architecture.		
0.5.0	01/04/2026	Adding subsystem designs and initiating the Data design, which is the class diagram.		

Table of Contents

Abstract	I
Table of Contents.....	i
List of Figures	vi
List of Tables.....	ix
1. Introduction	1
1.1. Project Background	1
1.2. Problem Statement	1
1.3. Purpose of the System.....	1
1.4. Project Scope.....	2
1.4.1. In Scope.....	2
1.4.2. Out of Scope	2
1.5. Related Documents	2
2. System Overview	2
2.1. Product Overview	3
2.2. Main System Components.....	3
2.3. User Roles and Access Summary	3
2.4. Technology Stack	4
2.5. Design Assumptions and Constraints	4
3. System Architecture	5
3.1. High-Level Architecture	5
3.2. Architecture Style.....	6
3.3. System Context.....	6
3.4. Component Responsibilities	6
3.5. Communication Overview	7
3.6. Security Architecture Overview.....	7
4. Subsystem Design	7
4.1. Embedded Firmware Design	8
4.1.1. Registration Desk Node	8
4.1.2. Checkpoint Node	8
4.1.3. Wi-Fi Configuration.....	9
4.1.4. MQTT Communication.....	9
4.1.5. RFID Reading.....	9
4.1.6. LED and Board Control	9
4.2. Backend System Design	9
4.2.1. API Layer	10
4.2.2. Authentication and Authorization.....	10

4.2.3.	Node Management Services	11
4.2.4.	Asset Registration Services	11
4.2.5.	Asset Movement Services	11
4.2.6.	Alert and Activity Services	11
4.2.7.	MQTT Processing	11
4.2.8.	WebSocket Broadcasting	12
4.3.	Registration App Design	12
4.3.1.	Login and Session Handling	12
4.3.2.	Registration Dashboard	13
4.3.3.	Node Registration Page	13
4.3.4.	Asset Registration Page	13
4.3.5.	Movement Request Approval	13
4.3.6.	User Management	13
4.3.7.	Account Security	13
4.4.	Monitor App Design	14
4.4.1.	Login and Session Handling	14
4.4.2.	Monitor Dashboard	14
4.4.3.	Asset Monitor	14
4.4.4.	Node Monitor	14
4.4.5.	Alerts Page	14
4.4.6.	Activity Page	15
4.4.7.	Movement Request Modal	15
5.	Data Design	16
5.1.	Data Model Overview	16
5.2.	Entity-Relationship Diagrams	16
5.3.	Main Data Entities	20
5.4.	Database Relationships	20
5.5.	Data Validation Rules	21
5.6.	Status and Enum Values	21
5.7.	Time and Date Handling	22
5.8.	Data Integrity Rules	22
6.	Software Class and Component Design	23
6.1.	Firmware Class and Interface Design	23
6.1.1.	Core Firmware Classes and Main Relations	23
6.1.2.	Firmware Configuration and Data Structures	30
6.1.3.	Firmware Interface Diagrams	31
6.1.4.	Firmware Responsibility Summary	36
6.2.	Backend Class and Service Structure	36

6.2.1.	Backend Domain Models and Layer Overview	36
6.2.2.	Authentication, User, and Node Services	38
6.2.3.	Asset Registration, Movement, and Detection Services.....	40
6.2.4.	MQTT Processing and API Schemas	43
6.3.	Registration App Component Structure.....	46
6.3.1.	Registration App Architecture and Authentication	46
6.3.2.	Registration App Feature Components	49
6.3.3.	Registration App API and Realtime Structure	53
6.4.	Monitor App Component Structure	53
6.4.1.	Monitor App Architecture and Authentication	53
6.4.2.	Monitor App Feature Components	55
6.4.3.	Monitor Data and Realtime Infrastructure.....	57
7.	Behavioral Design.....	60
7.1.	Sequence Diagrams	60
7.1.1.	User Login	60
7.1.2.	Load Registration Dashboard	61
7.1.3.	Assign Node	62
7.1.4.	Identify Node Blink	63
7.1.5.	Scan Tag at Registration Desk	64
7.1.6.	Register Asset	65
7.1.7.	Request Asset Movement	66
7.1.8.	Approve Movement Request	67
7.1.9.	RFID Detection Updates Asset Location	68
7.1.10.	Complete Approved Movement.....	69
7.1.11.	Node Heartbeat.....	70
7.1.12.	Offline Node Detection	71
7.2.	Firmware State Machine Diagrams.....	71
7.2.1.	Firmware State Machines	72
7.2.2.	System and Application State Machines.....	83
8.	Data Flow and Communication Design.....	88
8.1.	Overall Communication Flow	88
8.2.	Web Application and API Communication.....	88
8.3.	Sensor Node and MQTT Communication	88
8.4.	Backend Data and Live Update Communication.....	88
8.5.	Communication Protocol Design	88
8.6.	MQTT Topic Design	89
8.7.	WebSocket Event Design.....	89
8.8.	API Request and Response Design.....	90

9.	Interface Design	90
9.1.	Hardware Interfaces.....	90
9.2.	Firmware Interfaces	91
9.3.	Backend API Interfaces.....	91
9.4.	MQTT Interfaces	91
9.5.	WebSocket Interfaces	91
9.6.	Frontend User Interface Design	91
9.7.	External Service Interfaces	91
10.	Error Handling and Recovery Design	91
10.1.	Frontend Error Handling	91
10.2.	Backend Error Handling.....	92
10.3.	MQTT Error Handling	92
10.4.	WebSocket Reconnection	92
10.5.	Firmware Recovery.....	92
10.6.	Database Error Handling	93
10.7.	User Feedback and Error Messages	93
11.	Security Design	93
11.1.	Authentication Design	93
11.2.	Role-Based Authorization	93
11.3.	Token and Session Handling	93
11.4.	Password Security.....	93
11.5.	API Protection	94
11.6.	Sensitive Configuration Handling.....	94
11.7.	Admin Safety Rules	94
12.	Deployment Design.....	94
12.1.	Deployment Overview	94
12.2.	Local Development Deployment.....	95
12.3.	Production Deployment	95
12.4.	Frontend Deployment.....	95
12.5.	Backend Deployment	96
12.6.	Database Deployment.....	96
12.7.	MQTT Broker Deployment	96
12.8.	Environment Variables and Configuration	96
12.9.	Firmware Deployment	97
13.	Testing and Verification	98
13.1.	Firmware Testing	98
13.2.	Backend Testing	98
13.3.	Frontend Testing.....	98

13.4.	Integration Testing	98
13.5.	End-to-End Testing	98
13.6.	Acceptance Testing	98
14.	Design Decisions and Trade-Offs	99
14.1.	Checkpoint-Based Tracking Decision	100
14.2.	MQTT Communication Decision	100
14.3.	WebSocket Live Update Decision	100
14.4.	Role-Based Access Decision	100
14.5.	Cloud Hosting Decisions	100
14.6.	Known Limitations.....	100
15.	Appendices	101
15.1.	Diagram List.....	102
15.2.	Status Value Reference	107
15.3.	MQTT Topic Reference	109
15.4.	WebSocket Event Reference.....	110
15.5.	API Endpoint Summary.....	111

List of Figures

Figure 3. 1. 1 System's high-level architecture diagram.....	5
Figure 3. 3. 1 System's context diagram.	6
Figure 3. 5. 1 System's communication overview diagram.	7
Figure 4. 1. 1. 1 System's registration desk firmware diagram.....	8
Figure 4. 1. 2. 1 System's checkpoint node firmware diagram.	9
Figure 4. 2. 1 System's backend and communication layer diagram.	10
Figure 4. 2. 7. 1 System's MQTT processing diagram.	12
Figure 4. 4. 1 Monitor app communication.	14
Figure 4. 4. 7. 1 Movement request modal communication.....	15
Figure 5. 2. 1 Identity and configuration logical ERD.	16
Figure 5. 2. 2 Hospital, room, and node logical ERD.....	17
Figure 5. 2. 3 Asset location and detection logical ERD.....	18
Figure 5. 2. 4 Movement request logical ERD.....	19
Figure 5. 2. 5 Asset and alert logical ERD.....	19
Figure 5. 2. 6 Activity event logical ERD.	20
Figure 5. 8. 1 Detection event and alert relationship.	22
Figure 6. 1. 1. 1 Node manager class.	23
Figure 6. 1. 1. 2 Node status LED class.	24
Figure 6. 1. 1. 3 Node board controller class.	24
Figure 6. 1. 1. 4 Node clock class.....	25
Figure 6. 1. 1. 5 Node MQTT class dependencies.....	25
Figure 6. 1. 1. 6 Node network configuration class dependencies.	26
Figure 6. 1. 1. 7 Node Wi-Fi class.	26
Figure 6. 1. 1. 8 Node RFID and config state relation with hardware serial dependency.	27
Figure 6. 1. 1. 9 Main module firmware coordination.	28
Figure 6. 1. 1. 10 Firmware module dependencies.....	29
Figure 6. 1. 2. 1 Bootstrap configuration class.	30
Figure 6. 1. 2. 2 Remote configuration class.	30
Figure 6. 1. 2. 3 Firmware roles and state enumerations.	30

Figure 6. 1. 3. 1 MQTT interface class diagram.....	31
Figure 6. 1. 3. 2 Wi-Fi interface class diagram.....	32
Figure 6. 1. 3. 3 Network configuration interface class diagram.....	32
Figure 6. 1. 3. 4 RFID interface class diagram.	33
Figure 6. 1. 3. 5 Status LED interface class diagram.....	33
Figure 6. 1. 3. 6 Node clock interface class diagram.	34
Figure 6. 1. 3. 7 Node board control interface class diagram.	34
Figure 6. 1. 3. 8 System firmware interface implementation.....	35
Figure 6. 2. 1. 1 Backend layer overview.	37
Figure 6. 2. 1. 2 Backend domain models.....	38
Figure 6. 2. 2. 1 System authentication and user services.	39
Figure 6. 2. 2. 2 System management services.	39
Figure 6. 2. 3. 1 Asset registration services.....	40
Figure 6. 2. 3. 2 Asset movement services.....	41
Figure 6. 2. 3. 3 Asset detection and flow resolution	42
Figure 6. 2. 4. 1 Backend MQTT processing.	43
Figure 6. 2. 4. 2 Authentication and user API schemas.....	44
Figure 6. 2. 4. 3 Node API schemas.	44
Figure 6. 2. 4. 4 Asset API schemas.....	45
Figure 6. 2. 4. 5 Movement request API schemas.....	45
Figure 6. 2. 4. 6 Configuration API schemas.	46
Figure 6. 3. 1. 1 Registration app architecture.....	47
Figure 6. 3. 1. 2 Registration app authentication and routing.....	48
Figure 6. 3. 2. 1 Registration dashboard components.	49
Figure 6. 3. 2. 2 Node registration components.	50
Figure 6. 3. 2. 3 Asset registration and movement components.	51
Figure 6. 3. 2. 4 User management components.	52
Figure 6. 3. 2. 5 Account security and settings components.....	52
Figure 6. 3. 3. 1 Registration app API and real-time flow.....	53
Figure 6. 4. 1. 1 Monitor app architecture.	54
Figure 6. 4. 1. 2 Monitor app authentication and routing.	54
Figure 6. 4. 2. 1 Monitor dashboard components.....	55
Figure 6. 4. 2. 2 Asset monitor components.	55
Figure 6. 4. 2. 3 Movement request modal components.....	56
Figure 6. 4. 2. 4 Node monitor components.	56
Figure 6. 4. 2. 5 Alert page components.	57
Figure 6. 4. 2. 6 Activity page components.	57

Figure 6. 4. 3. 1 Monitor data orchestration.	58
Figure 6. 4. 3. 2 Monitor API and WebSocket infrastructure.	59
Figure 7. 1. 1 User login sequence diagram.	60
Figure 7. 1. 2 Registration dashboard loading sequence diagram.	61
Figure 7. 1. 3 Assign node sequence diagram.	62
Figure 7. 1. 4 Identify node blink sequence diagram.	63
Figure 7. 1. 5 Registration desk scan sequence diagram.	64
Figure 7. 1. 6 Register asset sequence diagram.	65
Figure 7. 1. 7 Request asset movement sequence diagram.	66
Figure 7. 1. 8 Approved movement request sequence diagram.	67
Figure 7. 1. 9 RFID detection location update sequence diagram.	68
Figure 7. 1. 10 Approved movement completion sequence diagram.	69
Figure 7. 1. 11 Node heartbeat sequence diagram.	70
Figure 7. 1. 12 Offline node detection sequence diagram.	71
Figure 7. 2. 1. 1 Main firmware lifecycle state machine.	73
Figure 7. 2. 1. 2 Captive portal state machine.	74
Figure 7. 2. 1. 3 Provisioning state machine.	75
Figure 7. 2. 1. 4 MQTT connection state machine.	76
Figure 7. 2. 1. 5 MQTT blink command state machine.	77
Figure 7. 2. 1. 6 RFID scan state machine.	78
Figure 7. 2. 1. 7 Heartbeat publish state machine.	79
Figure 7. 2. 1. 8 Remote configuration refresh state machine.	80
Figure 7. 2. 1. 9 Reset button state machine.	81
Figure 7. 2. 1. 10 LED state machine.	82
Figure 7. 2. 2. 1 Node lifecycle state machine.	83
Figure 7. 2. 2. 2 Asset lifecycle state machine.	84
Figure 7. 2. 2. 3 Asset flow state machine.	84
Figure 7. 2. 2. 4 Movement request state machine.	85
Figure 7. 2. 2. 5 Authentication and session state machine.	86
Figure 7. 2. 2. 6 WebSocket connection state machine.	87
Figure 7. 2. 2. 7 Alert lifecycle state machine.	87
Figure 9. 1. 1 Hardware wiring overview.	90
Figure 12. 1. 1 Deployment overview.	94
Figure 12. 4. 1 Frontend deployment diagram.	95
Figure 12. 5. 1 Backend deployment diagram.	96
Figure 12. 9. 1 Firmware deployment diagram.	97

List of Tables

Table 1. 4. 1. 1 List of project system components in scope.	2
Table 2. 3. 1 List of user roles, access, and responsibilities.	3
Table 5. 6. 1 List of system's status and enum values.	21
Table 6. 1. 4. 1 List of the system's firmware responsibilities.	36
Table 8. 5. 1 List of system communication protocols and their purposes.	89
Table 8. 6. 1 List of system MQTT topic designs.	89
Table 8. 7. 1 List of system WebSocket designs.	89
Table 8. 8. 1 List of system API requests and responses.	90
Table 10. 1 List of the system's frontend error handling.	91
Table 10. 2 List of the system's backend error handling.	92
Table 10. 3 List of the system's MQTT error handling.	92
Table 10. 4 List of the system's WebSocket error handling.	92
Table 10. 5 List of the system's firmware recovery.	92
Table 10. 6 List of the system's database error handling.	93
Table 10. 7 List of the system's user feedback and error messages.	93
Table 14. 1 List of design trade-offs.	99
Table 15. 1 List of design diagrams.	102
Table 15. 2 List of system state status values.	107
Table 15. 3 System's MQTT communication reference list.	109
Table 15. 4 List of WebSocket event references.	110
Table 15. 5 List of API Endpoints.	111

1. Introduction

This section introduces the design document, the problem context, the system's purpose, and the design boundaries. It explains why the product is needed and how this design document relates to the requirements, use cases, and test documents.

1.1. Project Background

Hospitals manage many movable hospital fixed assets, such as infusion pumps, wheelchairs, portable monitors, and other medical equipment. These assets are frequently moved between rooms or departments, making it difficult for staff to know where the equipment is currently located. Traditional tracking methods rely on manual logging or staff memory, which can lead to:

1. Inefficient asset utilization.
2. Time wasted searching for equipment.
3. Increased risk of loss or misplacement.

To improve asset visibility and operational efficiency, an automated tracking system is required. Radio Frequency Identification (RFID) technology provides a practical solution by enabling contactless identification and detection of assets at specific locations.

This project proposes an RFID-based asset-tracking system that uses a checkpoint detection method, in which assets are detected at fixed locations (checkpoints) within the hospital environment.

1.2. Problem Statement

Hospitals face challenges in tracking the locations of movable assets due to the lack of a reliable automated tracking system.

Key problems include:

- Inability to determine the current or last known location of assets.
- Time-consuming manual searching by hospital staff.
- Lack of centralized asset tracking records.
- Inefficient asset utilization and possible duplication of equipment.
- Difficulty in monitoring asset movement across different departments.

Without a structured tracking mechanism, hospital operations can be disrupted, contributing to operational delays, inefficient equipment use, and increased staff workload.

1.3. Purpose of the System

The fixed-asset tracking system is designed to provide real-time visibility of movable medical equipment within hospital indoor environments. The system aims to reduce equipment search time, improve asset utilization, and support more efficient hospital operations through automated identification and monitoring.

The system should allow users to register and deregister RFID-tagged assets at a registration desk, automatically detect tagged assets when they pass through checkpoint scanners, store scan events along with checkpoint and timestamp information, determine and display the last known location of each asset based on the most recent scan, provide a monitoring interface (web or application) for users to locate assets easily, support scalability, allowing additional checkpoint scanners to be added without redesigning the system.

1.4. Project Scope

This project has in-scope and out-of-scope items that determine what should and should not be included in the product.

1.4.1. In Scope

The system includes the following components and functionalities, shown in the table below.

Table 1. 4. 1. 1 List of project system components in scope.

No.	System Component	Functionalities
1.	Registration Desk Node	- Reads UHF RFID tags during asset registration or deregistration and publishes registration-scan events through MQTT.
2.	Checkpoint Node	- Detects RFID-tagged assets at an assigned hospital location and publishes detections and heartbeat messages through MQTT.
3.	Registration App	- Supports first-admin setup, login, node management, asset registration and deregistration, movement decisions, user management, account security, and settings according to role permissions.
4.	Monitor App	- Displays assets, nodes, alerts, and activity records and allows authorized users to create or cancel movement requests.
5.	FastAPI Backend	- Provides authentication, authorization, business rules, REST APIs, MQTT processing, WebSocket broadcasting, alerts, activity logging, and movement processing.
6.	PostgreSQL Database	- Stores users, nodes, assets, detections, movement requests, alerts, activity records, and configuration data.
7.	MQTT Broker	- Relays sensor-node events, heartbeat messages, backend commands, and command acknowledgments between firmware and the backend MQTT client.

1.4.2. Out of Scope

The approaches below are not included in this project:

1. Real-time indoor positioning (continuous tracking between checkpoints).
2. Patient or staff tracking.
3. Integration with full hospital information systems (HIS/ERP).
4. Predictive analytics or asset usage optimization.
5. Advanced localization technologies.

1.5. Related Documents

This design document should be used together with the System Requirements Specification, Requirements and Use Case Specification, and the Software Test Report documents. The requirements document defines what the system shall provide; the system test report records test cases, execution results, evidence, defects, and requirement-verification outcomes; and this design document explains how the system is structured and implemented.

2. System Overview

Based on the project plan and the operational design system overview in the proposal document, the system product will consist of four main components. These components are the Registration Desk, Checkpoint Scanner System, Backend Server, and Item Monitor Application.

This section summarizes the product at a system level before the design is divided into technical subsystems. It also explains the main components, users, technologies, and assumptions that shape the design.

2.1. Product Overview

The Hospital Fixed-Asset Tracking System consists of physical sensor nodes, RFID-tagged assets, a backend platform, a database, and two browser-based frontend applications. The Registration App supports asset registration, node management, user management, and movement approval workflows. The Monitor App supports asset monitoring, node monitoring, alerts, activity logs, and movement-request creation.

Sensor nodes detect RFID tags and send messages to the backend through an MQTT broker. The backend validates incoming events, updates the database, and broadcasts live updates to connected frontend clients through WebSocket. This creates a near real-time operational view of asset movement and node availability.

2.2. Main System Components

The main components are the Registration App, Monitor App, FastAPI backend, PostgreSQL database, MQTT broker, Registration Desk Node, and Checkpoint Node. Each component has a specific responsibility and communicates with other components using defined interfaces.

The design separates user-facing responsibilities from backend processing and physical sensing responsibilities. This separation allows the web applications, backend services, and firmware to be developed, tested, deployed, and maintained independently.

2.3. User Roles and Access Summary

The system supports multiple user roles, including Initial System Administrator, Admin, Test User, Technician, Registration Staff, and Monitor Staff. Role-based access control determines which pages and actions are available to each user.

The Initial System Administrator only creates the first admin account when no admin exists. Admin users manage the complete system. Technicians mainly handle nodes. Registration Staff handles asset registration and movement decisions. Monitor Staff observes assets and requests movement. Test User access is used for broader testing without the normal admin account management.

Table 2. 3. 1 List of user roles, access, and responsibilities.

No.	Role or setup actor	Primary application access	Principal responsibilities	Important restriction
1.	Initial System Administrator	First-admin setup page only	Creates the first Admin account when no administrator has ever been created.	Unavailable after the first administrator exists.
2.	Admin	Registration App and Monitor App	Manages users, nodes, assets, movement workflows, monitoring, alerts, settings, and account security.	Cannot leave the system without an active administrator.
3.	Test User	Controlled test or demonstration access	Supports broader product verification without normal administrator account-management responsibility.	Not intended as a production administration substitute.
4.	Technician	Registration App node-management functions	Discovers, assigns, edits, identifies, enables, disables, unassigns, and maintains eligible nodes.	Does not receive unrestricted user-management or movement-decision authority.
5.	Registration Staff	Registration App	Registers and deregisters assets and approves or rejects pending movement requests.	Does not manage users or technical node configuration unless separately authorized.
6.	Monitor Staff	Monitor App	Monitors assets and nodes, reviews alerts and activity, and creates or cancels eligible movement requests.	Does not approve movement requests or manage users.

2.4. Technology Stack

The embedded firmware is designed for ESP32 using PlatformIO and C++. The backend is designed with Python, FastAPI, SQLAlchemy, PostgreSQL, MQTT client logic, and WebSocket broadcasting. The frontend applications are built with React and Vite.

MQTT is used for asynchronous communication between sensor nodes and the backend. HTTPS JSON APIs are used for frontend-to-backend requests. WebSocket is used for live backend-to-frontend updates.

2.5. Design Assumptions and Constraints

The design assumes that hospital Wi-Fi or local network connectivity is available in the areas where sensor nodes and client devices are used. It also assumes that every tracked asset has a compatible RFID tag and that checkpoint nodes are installed in meaningful hospital locations.

Important constraints include ESP32 hardware limitations, RFID reading behavior, MQTT broker availability, browser compatibility, cloud hosting limitations, secure configuration management, and the need to keep the source code modular and maintainable.

3. System Architecture

This section describes the product's architectural structure and the responsibilities of its major components. It focuses on how the product is organized rather than on implementation code.

3.1. High-Level Architecture

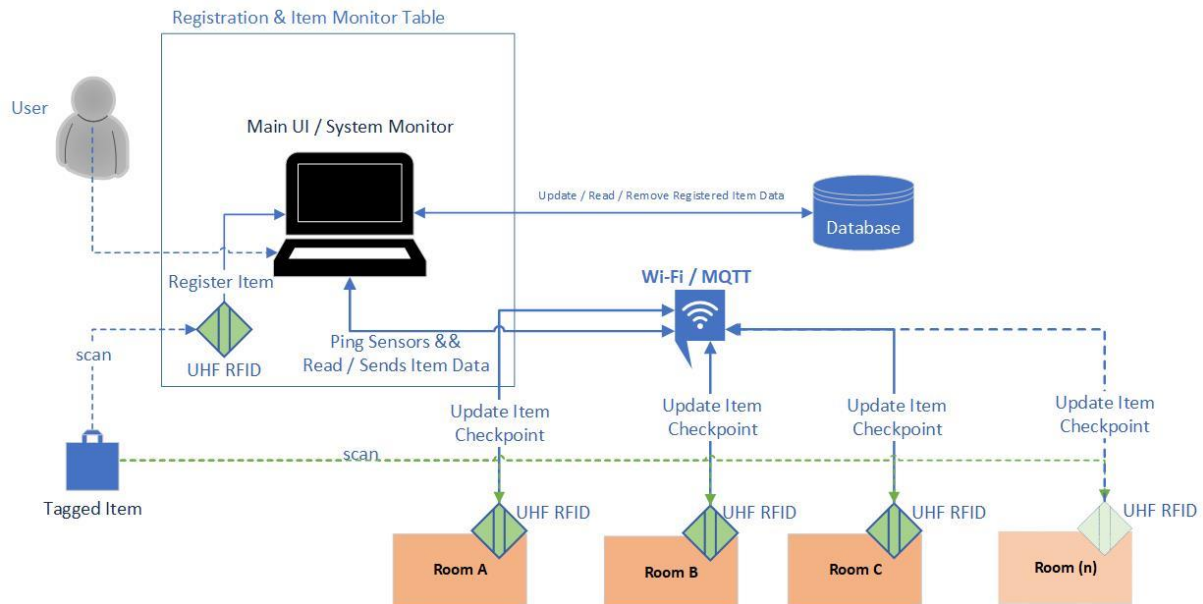


Figure 3. 1. 1 System's high-level architecture diagram.

The picture above is a simple architecture that explains how the system must work by design.

The Registration Table includes a UHF RFID sensor for registering and deregistering fixed hospital assets with embedded RFID tags. In addition, the same table contains the main UI and the system monitor, which displays all registered items in the system.

This monitoring system connects to the database via HTTPS to read and save all registered items and remove them upon deregistration. The API the system uses to connect between the app and the database is FastAPI.

Using a Wi-Fi network and the MQTT protocol through the backend, the system monitor can see all UHF RFID scanner nodes installed at each hospital room entrance, whether they are online or not. This feature ensures product safety by informing users when a sensor is damaged, offline, or otherwise unusable, even while the entire system remains online.

These nodes are designed to be extensible, which means the base code of each node must be duplicable.

Using the same protocol, the nodes update the tagged asset's location whenever it is detected within their respective reading ranges, treating each detection as a checkpoint.

The checkpoint ping is forwarded to the main monitor, which automatically updates the tagged item's checkpoint time and location in the database.

3.2. Architecture Style

The design combines a web application architecture, event-driven communication, and a service-oriented backend structure. User actions are handled synchronously through API requests, while RFID detections and heartbeat messages are handled asynchronously through MQTT.

This combination is suitable because hospital users need predictable web forms and dashboards, while sensor nodes need lightweight message publishing that does not require direct browser communication.

3.3. System Context

The system exists between hospital users, RFID-tagged assets, sensor nodes, cloud-hosted services, and network infrastructure. Human users interact with the Registration App and Monitor App, while physical assets interact indirectly through RFID tags detected by sensor nodes.

The context view should show the product boundary, external actors, the hospital network, the MQTT broker, and the major data exchanged among them. The diagram for this section should remain high-level and should not include internal source code modules.

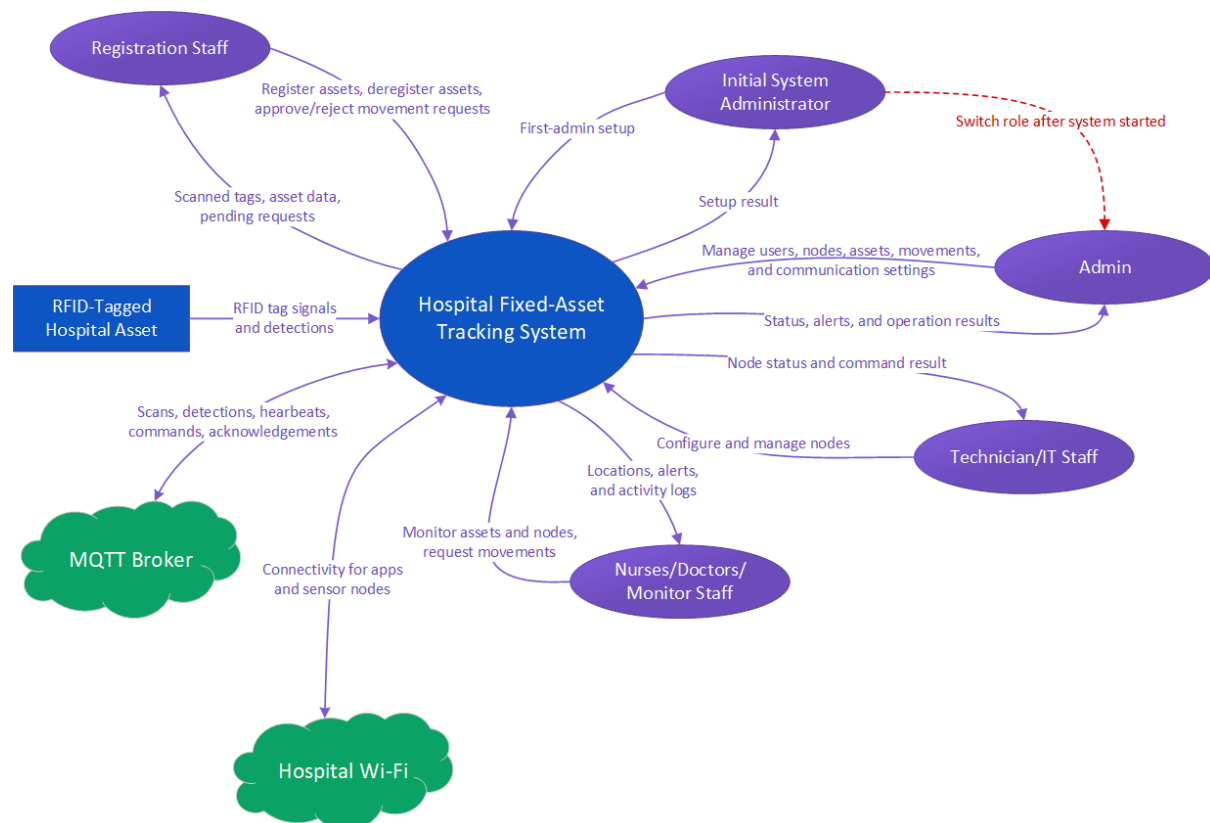


Figure 3. 3. 1 System's context diagram.

3.4. Component Responsibilities

The Registration App is responsible for registration-side workflows, including node management, asset registration, movement approval, user management, and password management. The Monitor App is responsible for operational visibility, including asset status, node status, alerts, activity logs, and movement requests.

The backend is responsible for authentication, authorization, business rules, data persistence, MQTT processing, and WebSocket broadcasting. The firmware is responsible for Wi-Fi setup, RFID reading, MQTT publishing, MQTT command handling, heartbeat reporting, and LED control.

3.5. Communication Overview

The communication design uses three main protocols. HTTPS is used for normal frontend-to-backend API communication. MQTT is used for sensor-node events and backend-to-node commands. WebSocket is used for live updates from the backend to connected browser clients.

The frontend applications do not communicate directly with the MQTT broker. All MQTT data is processed by the backend, keeping validation, authorization, persistence, and event broadcasting centralized.

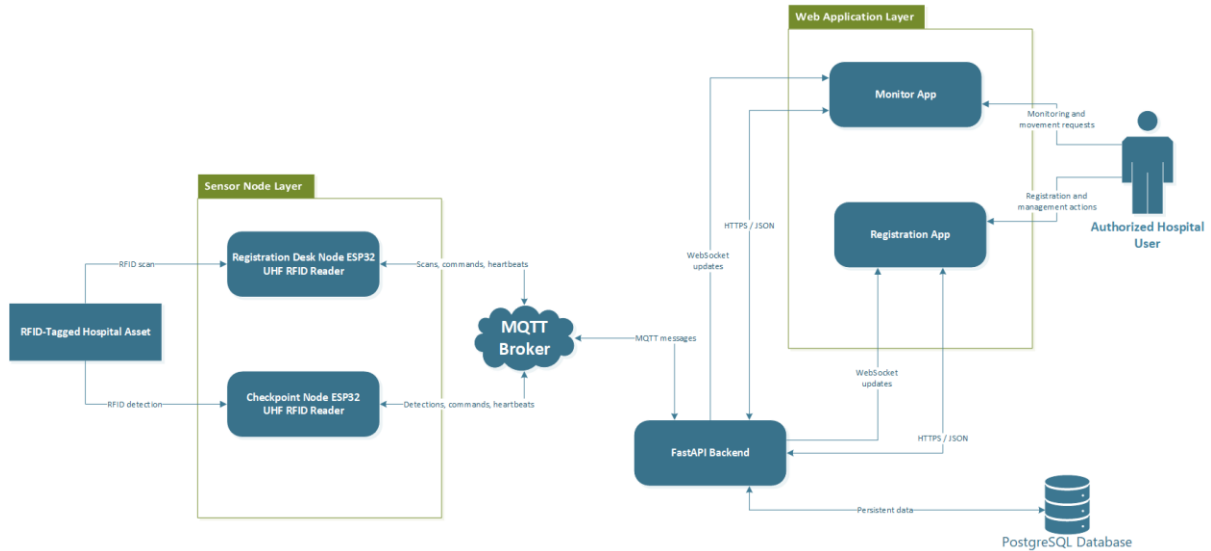


Figure 3. 5. 1 System's communication overview diagram.

3.6. Security Architecture Overview

Security is centered around authenticated API access, role-based authorization, password hashing, session handling, protected backend routes, and safe frontend navigation. The backend is the primary security boundary because users and browser clients cannot be trusted to enforce permissions on their own.

Sensitive deployment values, database credentials, JWT secrets, and broker credentials should be managed through environment variables or protected configuration. Production deployments should use HTTPS and secure WebSocket communication.

4. Subsystem Design

The Hospital Asset Tracker subsystems include the registration desk, scanner nodes, backend system, and monitor app. The registration desk is implemented to register and deregister assets with tags in or out of the system database. The scanner nodes detect registered assets and update their checkpoints whenever the asset passes through the node. The backend system manages the product's logic and the communication lines between other parts of the system. The monitor app is responsible for displaying the data for all registered items, along with their latest checkpoints, to the user.

All the details about these subsystems are provided in the subsections below.

4.1. Embedded Firmware Design

The embedded firmware runs on ESP32-based sensor nodes. It manages Wi-Fi configuration, MQTT connection, RFID reading, heartbeat publishing, command acknowledgment, and status LED behavior. The firmware is designed to support different node roles using shared modules wherever possible.

The firmware design should remain modular, separating network configuration, MQTT communication, RFID reading, time handling, and board control into maintainable classes.

4.1.1. Registration Desk Node

The Registration Desk Node is used near the registration workflow to scan RFID tags when assets are registered or deregistered. Its main responsibility is to read the tag ID and publish a registration-scan event to the backend through MQTT.

The node does not decide whether the tag may be registered. It only reports the scan. The backend and Registration App are responsible for enforcing registration rules, performing duplicate checks, and handling user approval.

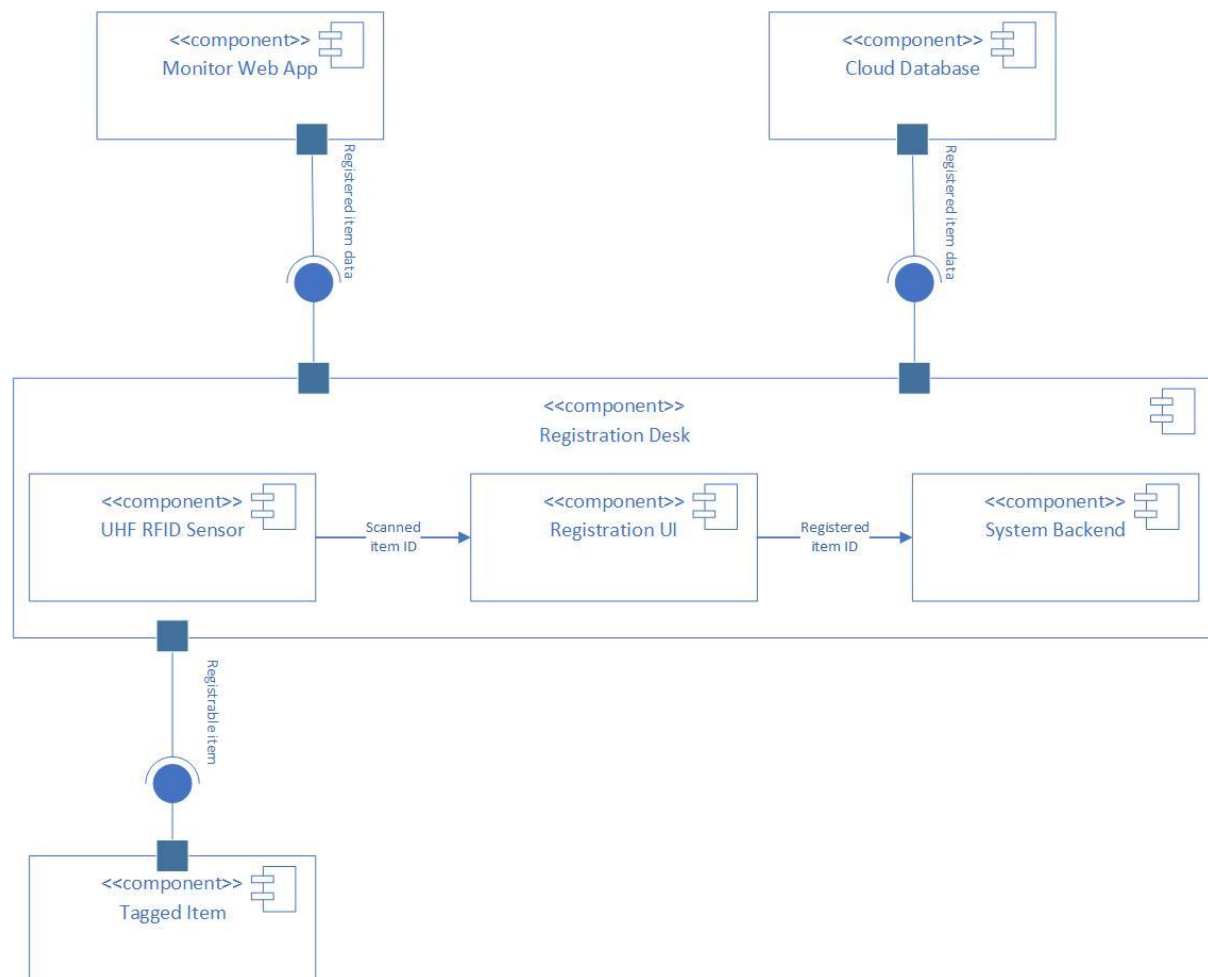


Figure 4. 1. 1 System's registration desk firmware diagram.

4.1.2. Checkpoint Node

The Checkpoint Node is installed at a designated hospital location and detects RFID-tagged assets as they enter or pass through its reading area. It publishes RFID detection messages and heartbeat messages through MQTT.

The backend uses the node identity and the assigned room to determine whether the detected asset is in place, in transit, at an approved destination, or in an unauthorized location.

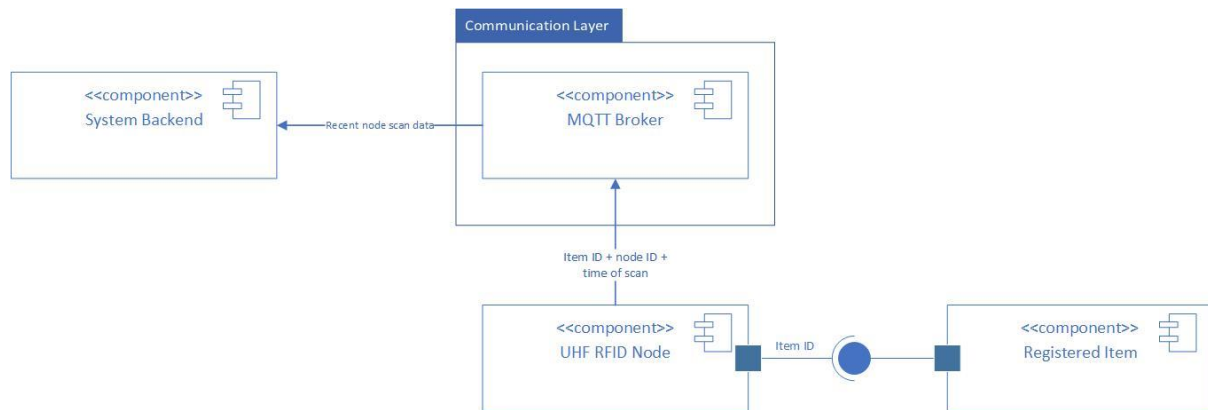


Figure 4.1.2. 1 System's checkpoint node firmware diagram.

4.1.3. Wi-Fi Configuration

The firmware provides a configuration mechanism for Wi-Fi credentials and node setup. When a valid configuration is unavailable, the node should enter a setup mode so that a technician can configure network and node information.

After configuration, the node attempts to connect to Wi-Fi and recover from a temporary disconnection. The design should avoid requiring a manual reset for ordinary network interruptions.

4.1.4. MQTT Communication

MQTT communication is responsible for publishing scans, detections, heartbeat messages, and command acknowledgments. The node also subscribes to command topics so the backend can send supported commands, such as blink identification.

The firmware should resubscribe after reconnection and build payloads consistently so that the backend can validate the topic structure, node identity, event type, and payload fields.

4.1.5. RFID Reading

The RFID module reads tag identifiers from the UHF RFID reader and forwards valid scan results to the MQTT module. The design should prevent unnecessary repeated publishing when the same tag is detected too frequently within a short interval.

The RFID module should not contain backend business rules. It should only identify tags and report them in a consistent message format.

4.1.6. LED and Board Control

LED and board-control logic support node status indication and physical node identification. For example, a blink command can help a technician identify which physical node corresponds to a record in the web application.

This logic should be separated from the MQTT and RFID code so that hardware behavior can be tested and changed without affecting communication logic.

4.2. Backend System Design

The backend system is the product's core processing layer. It exposes REST APIs to the frontend applications, subscribes to MQTT events from sensor nodes, updates the database, processes

movement workflows, generates alerts and activity records, and broadcasts live WebSocket events.

The backend should be organized into route layers, service layers, model layers, schema layers, and communication services. This separation improves testability and keeps business logic out of route handlers.

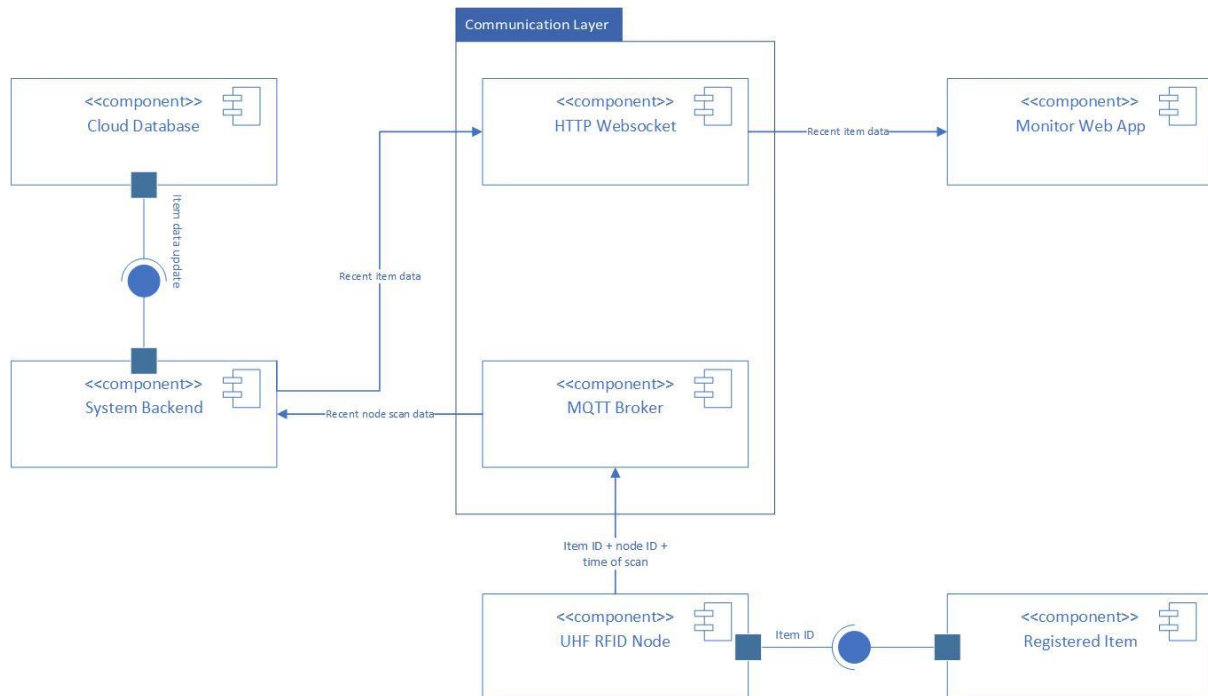


Figure 4.2. 1 System's backend and communication layer diagram.

4.2.1. API Layer

The API layer receives HTTP requests from the Registration App and Monitor App. It is responsible for parsing input, applying authentication and role dependencies, returning consistent responses, and delegating business work to services.

API routes should remain thin. Complex operations such as asset registration, movement approval, node assignment, and alert generation should be handled inside backend services.

4.2.2. Authentication and Authorization

Authentication verifies user credentials and creates a token-based session for the frontend applications. Authorization checks the user role before allowing protected actions. Role checks must be enforced on the backend even when the frontend hides unavailable pages or buttons.

The design also includes first-admin setup, password hashing, disabled-account rejection, log-out behavior, and account safety rules that prevent unsafe admin changes.

4.2.3. Node Management Services

Node management services handle discovered nodes, assignment, editing, unassignment, deletion rules, heartbeat updates, offline detection, disabled nodes, and blink commands. These services maintain the relationship between physical nodes and their configured hospital location.

The backend should treat node identity carefully. Matching by stable node ID is more reliable than matching by alias because aliases may change during assignment or maintenance.

4.2.4. Asset Registration Services

Asset registration services connect an RFID tag to an asset record and assign the asset to an expected room. The service validates duplicate tags, active status, registration-node availability, assigned room data, and deregistration rules.

After a successful registration or deregistration, the service updates persistent data and sends live updates to keep the Monitor App and Registration App synchronized.

4.2.5. Asset Movement Services

Asset movement services manage the workflows for requests, cancellations, approvals, rejections, and completions. An authorized monitoring-side user creates a movement request, and an authorized registration-side user decides it.

The backend must prevent duplicate pending requests for the same asset and complete approved movement only when the asset is detected at the approved destination node.

4.2.6. Alert and Activity Services

Alert services create user-visible warnings for important conditions such as unknown RFID tags, offline nodes, and unauthorized movement. Activity services record important system events so users can review system history.

These services support operational awareness and debugging. They should avoid recording sensitive data such as plain-text passwords or secret configuration values.

4.2.7. MQTT Processing

The MQTT processing layer subscribes to configured topics, receives node messages, validates payloads, and passes valid events to backend services. Invalid messages should be rejected safely without corrupting the database state.

The MQTT client also publishes supported commands to nodes, such as blink commands, and can receive acknowledgments from firmware to confirm command processing.

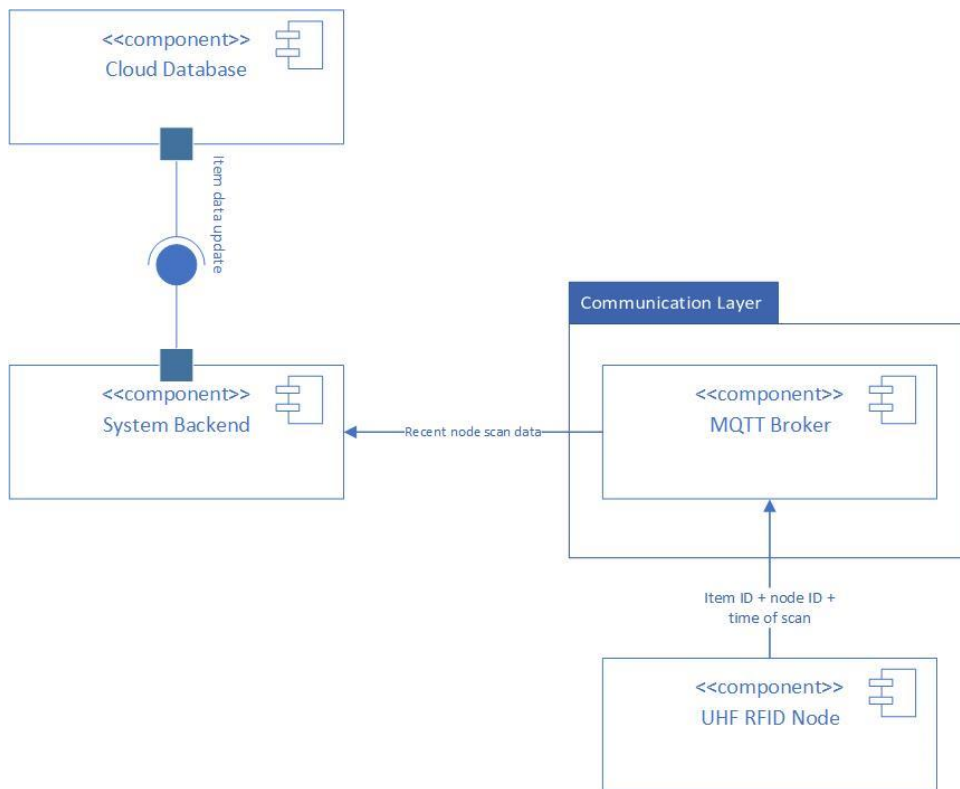


Figure 4. 2. 7. 1 System's MQTT processing diagram.

4.2.8. WebSocket Broadcasting

The WebSocket broadcasting layer sends live events to connected frontend clients after important data changes. Events may include asset updates, node updates, alerts, movement request changes, activity records, and connection status changes.

The design should allow frontend clients to reconnect after a temporary disconnection and safely ignore unsupported event types.

4.3. Registration App Design

The Registration App provides registration-side workflows through a browser interface. It communicates with the backend via HTTPS APIs and receives live updates via WebSocket when needed.

The app should be organized into pages, reusable components, hooks, services, utilities, and styles. Page components should focus on layout, while hooks and service files handle state management and API communication.

4.3.1. Login and Session Handling

The Registration App login page sends credentials to the backend and stores the returned session information when authentication succeeds. The app uses the user role to decide which pages and navigation items are visible.

When a token is missing, expired, or rejected by the backend, the app should clear the session and redirect the user to the login page.

4.3.2. Registration Dashboard

The Registration Dashboard acts as the main entry point for registration-side users. It displays quick-access cards and summary information according to the current user role.

The dashboard should not expose actions unavailable to the user. For example, Monitor Staff may only see limited registration-side information, while Admin and Test User roles have broader access.

4.3.3. Node Registration Page

The Node Registration page allows authorized users to view discovered and assigned nodes, assign nodes to roles and locations, edit assignments, identify nodes using blink, unassign nodes, and delete eligible records.

The UI should clearly distinguish node role, status, hospital, room, last ping time, and available actions. Offline or disabled nodes should not offer actions that require active communication with other nodes.

4.3.4. Asset Registration Page

The Asset Registration page supports RFID-based asset registration and deregistration, viewing the asset table, search, sorting, pagination, movement request approval, and movement rejection.

The page uses scanned RFID tag data from the Registration Desk Node and combines it with user-entered asset information. The backend remains responsible for validating the final registration request.

4.3.5. Movement Request Approval

Authorized registration-side users handle approval of movement requests. The page displays pending requests and provides actions to approve or reject them.

When a request is approved, the asset enters an in-transit state until a checkpoint node detects it at the approved destination. When the request is rejected, the request is closed, and the asset should remain accessible in the UI.

4.3.6. User Management

The User Management page allows Admin users to create accounts, update roles, deactivate or activate accounts, and manage account safety policies. It should not be accessible to non-admin roles.

The frontend may hide restricted actions, but the backend must enforce all admin-only rules and prevent unsafe changes, such as disabling the last active admin.

4.3.7. Account Security

The Account Security page allows authenticated users to change their own passwords. The design should validate required fields, confirmation matching, and backend rejection cases.

The page should provide clear feedback without exposing sensitive security information. Password visibility toggles may be implemented for usability, but should not store or log plain-text passwords.

4.4. Monitor App Design

The Monitor App is focused on operational visibility. It allows authorized users to view asset status, node status, alerts, activity records, and movement workflows.

The app communicates with the backend via HTTPS APIs for initial data loading and user actions, then uses WebSocket updates to keep visible information up to date.

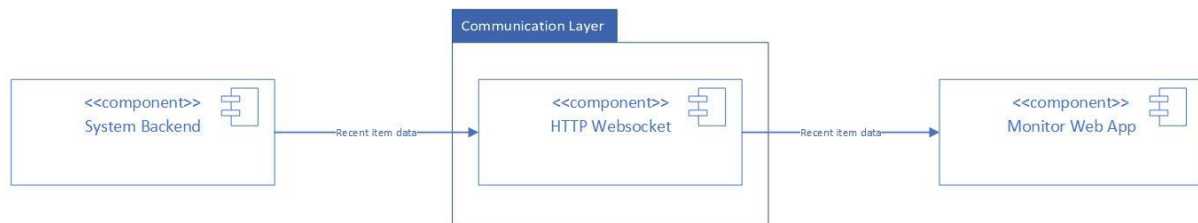


Figure 4. 4. 1 Monitor app communication.

4.4.1. Login and Session Handling

The Monitor App uses the same authentication model as the Registration App, but allows only roles that are permitted to access monitoring functions. The session is stored locally and cleared when logout or access rejection occurs.

The app should avoid opening WebSocket connections when no valid token exists. This prevents unnecessary connection attempts and confusing error messages.

4.4.2. Monitor Dashboard

The Monitor Dashboard summarizes key operational information, including asset counts, node availability, movement states, alerts, and recent activity. It gives monitoring users a quick overview of system health.

Dashboard counts should follow the same data rules as the detailed pages, so the overview does not conflict with the asset, node, or alert lists.

4.4.3. Asset Monitor

The Asset Monitor displays registered assets, assigned rooms, current rooms, latest detection times, and flow status. It also provides a movement request action where the user role allows it.

The page should clearly show whether an asset is in place, on the move, pending approval, approved for movement, rejected, or in an unauthorized location.

4.4.4. Node Monitor

The Node Monitor displays sensor-node status, role, location, and last heartbeat information. It helps users identify nodes that are offline, disabled, unknown, assigned, or online.

This page supports operational awareness, as unreliable nodes can affect tracking accuracy. Clear status display reduces confusion when assets are not being updated.

4.4.5. Alerts Page

The Alerts page lists important warnings such as unknown RFID tags, offline nodes, and unauthorized asset movement. It should help users identify which asset or node needs attention.

Alerts should be searchable, sortable, and readable on different screen sizes. The design should prioritize severity, time, and clear user-facing messages.

4.4.6. Activity Page

The Activity page displays important system events, including registration, deregistration, movement decisions, unauthorized movement, node status changes, and other operational records.

The activity feed should avoid excessive noise. For example, repeated heartbeat events may be hidden from the user-facing activity list while still being available in technical logs.

4.4.7. Movement Request Modal

The Movement Request Modal allows monitoring users to request that an asset be moved to a destination room. It should prevent duplicate submission and provide a clear cancel or close behavior.

The modal should validate the destination selection and display backend errors when the requested movement is not allowed.

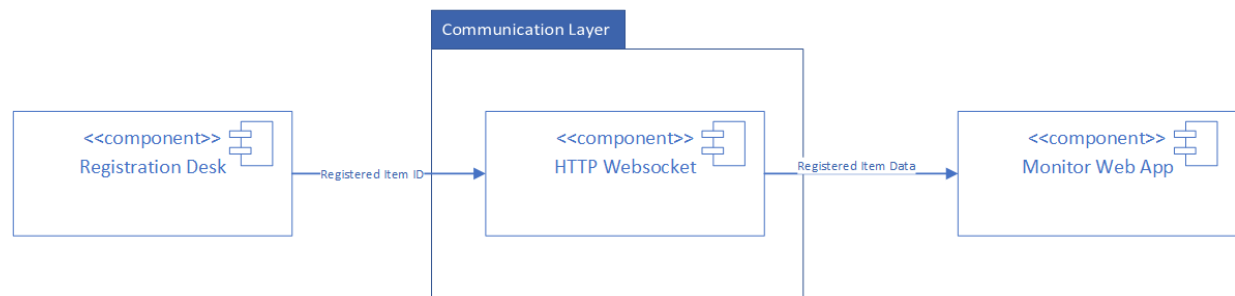


Figure 4. 4. 7. 1 Movement request modal communication.

5. Data Design

This section describes how the system data is organized, validated, related, and protected from inconsistent states. It should be used alongside the ER diagrams and the database implementation.

5.1. Data Model Overview

The data model stores user accounts, sensor nodes, assets, rooms or location values, movement requests, detection events, alerts, activity events, and system settings. These entities support both the system's operational state and the historical record of important events.

The data model should support traceability from an RFID detection to the node that produced it, the asset it matched, the alert it generated, and the activity event shown to users.

5.2. Entity-Relationship Diagrams

Entity-relationship diagrams should be used to describe how the main data entities relate to one another. To keep the diagrams readable, the document may use several smaller ER diagrams, such as user/settings, hospital-room-node, asset-detection, asset-movement, alerts, and activity-events.

A complete logical ERD may also be included in an appendix or as a landscape figure. The smaller diagrams should remain in the main section because they are easier to read and explain.

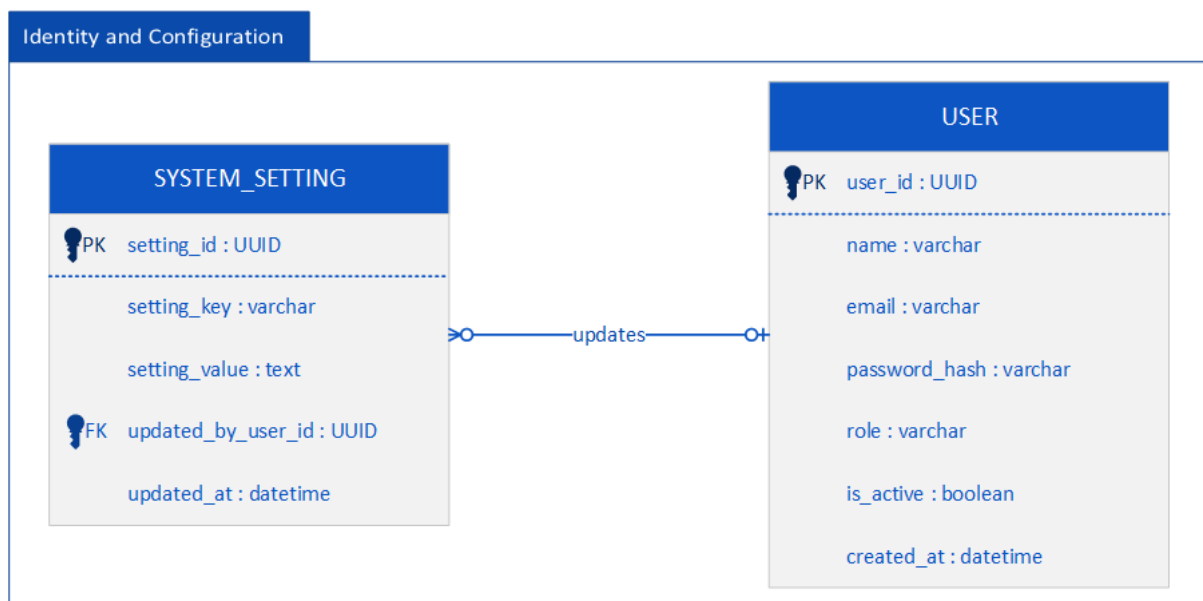


Figure 5. 2. 1 Identity and configuration logical ERD.

Hospital Location and Nodes

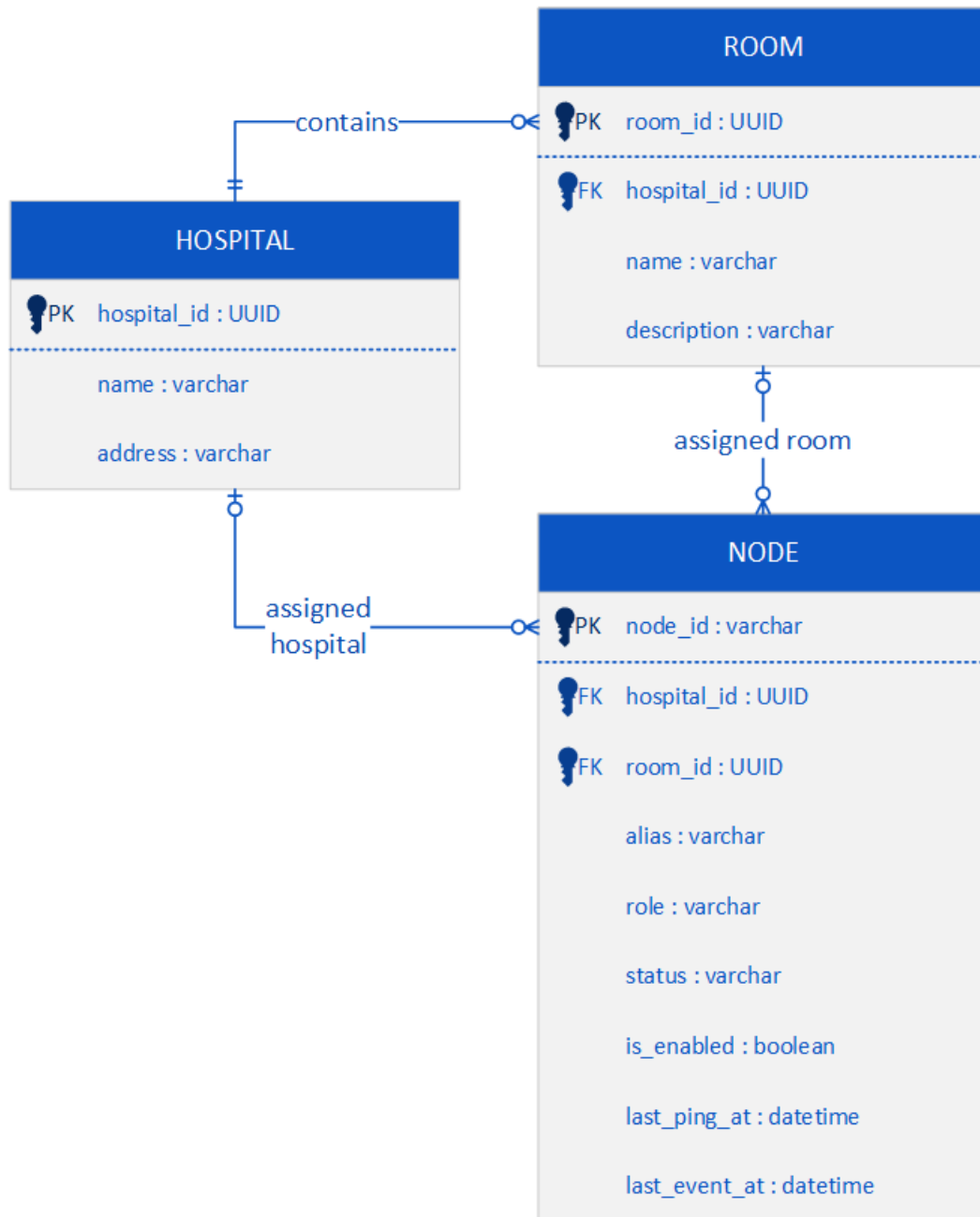


Figure 5. 2. 2 Hospital, room, and node logical ERD.

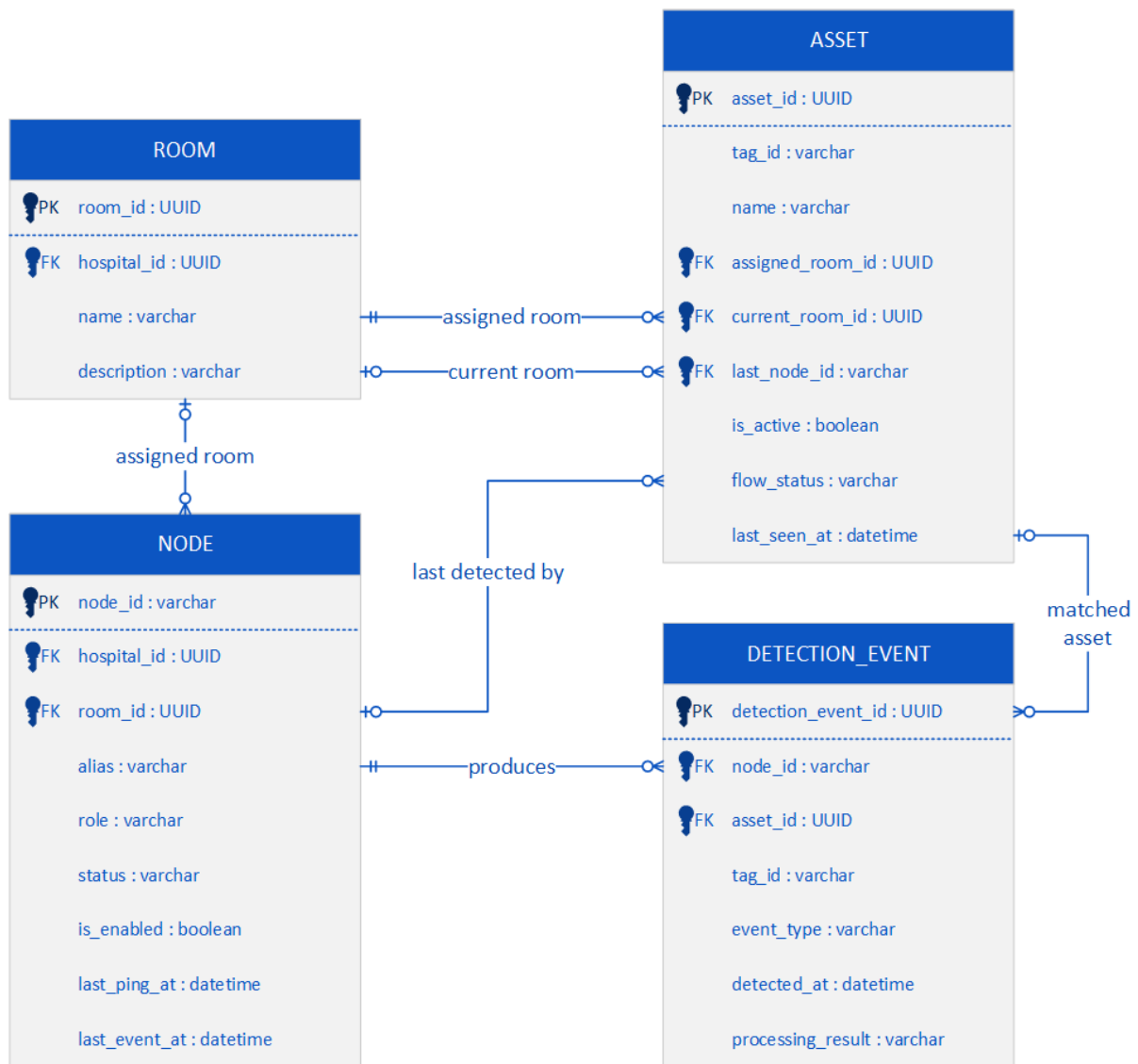


Figure 5. 2. 3 Asset location and detection logical ERD.

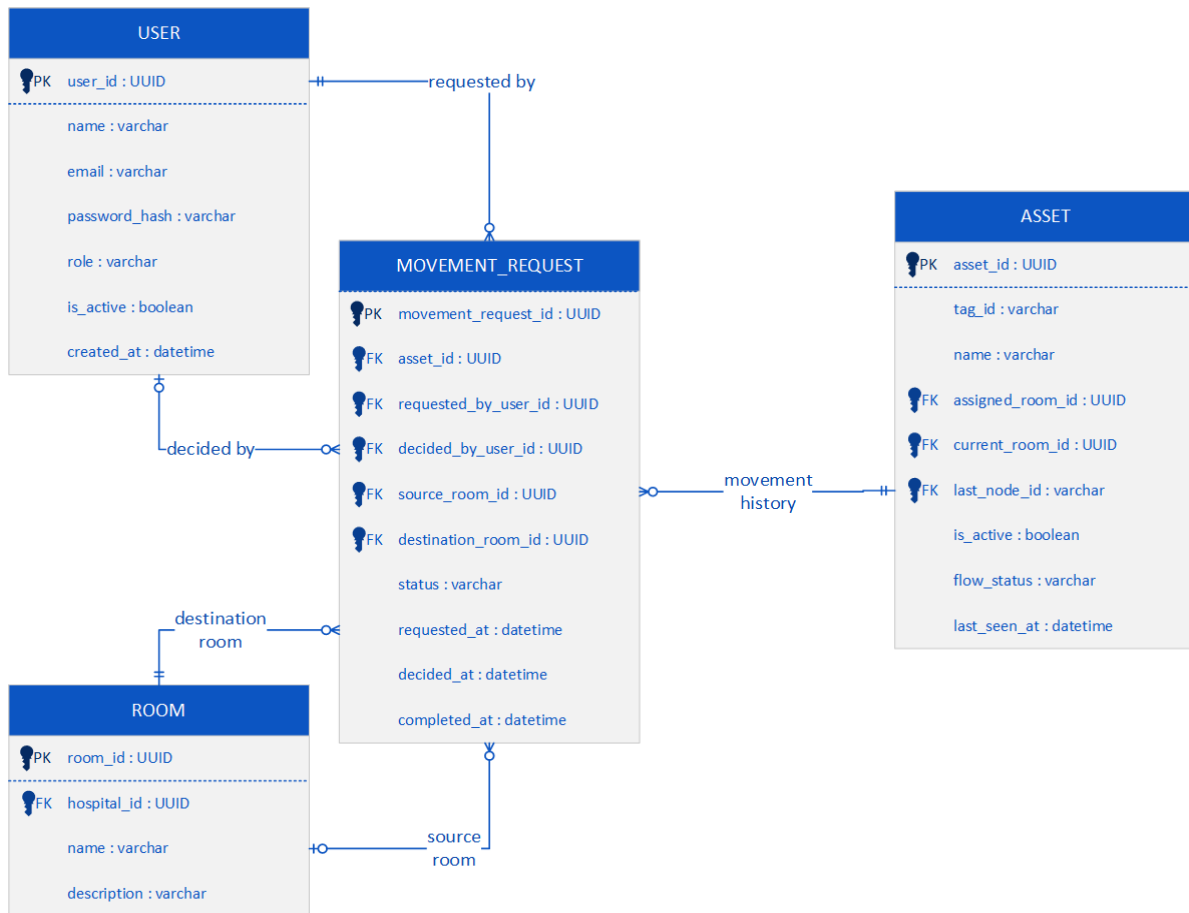


Figure 5.2.4 Movement request logical ERD.

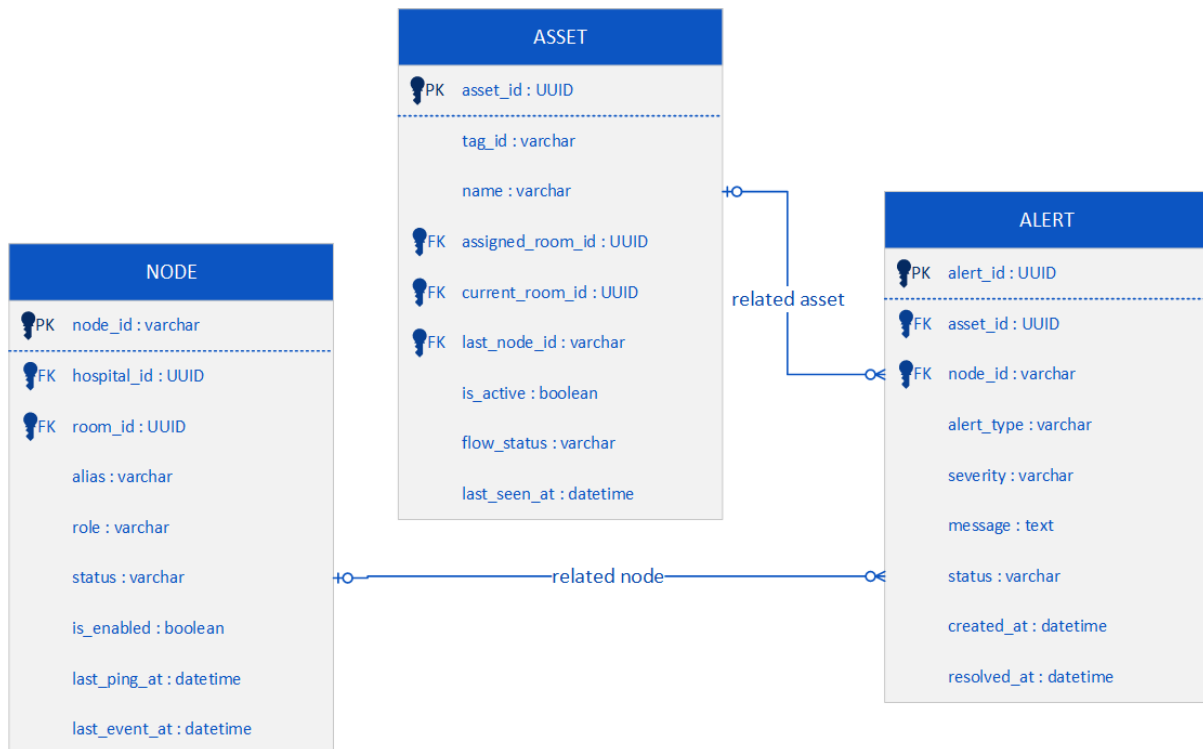


Figure 5.2.5 Asset and alert logical ERD.

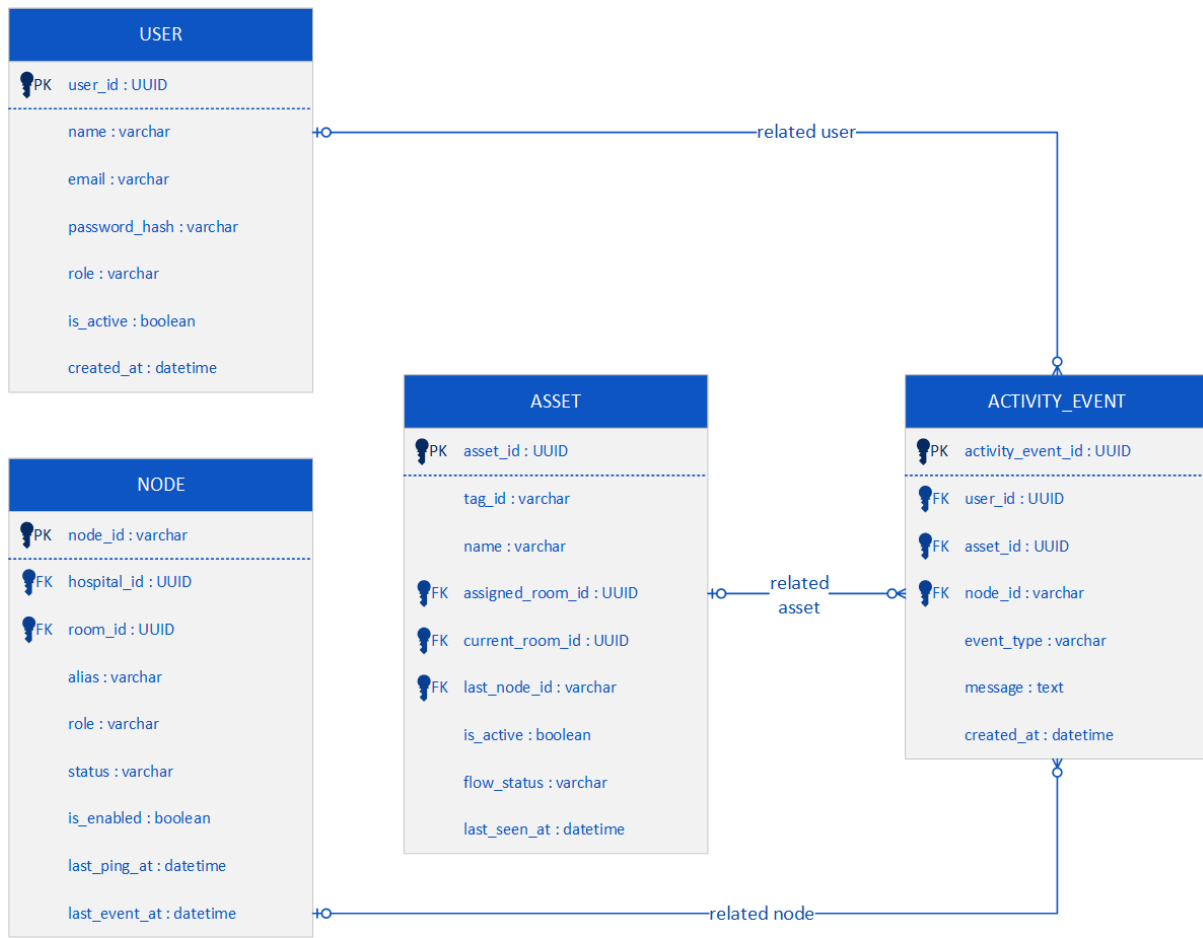


Figure 5.2. 6 Activity event logical ERD.

5.3. Main Data Entities

The main data entities include User, System Setting, Hospital, Room, Node, Asset, Detection Event, Movement Request, Alert, and Activity Event. If the hospital and Room are not physically separate tables in the current database, they may be presented as logical entities or configuration values.

Each entity should identify required fields, optional fields, primary keys, foreign keys, unique values, and conditionally required values. For example, a Movement Request requires a destination room but may not have a decision-maker user until it is approved or rejected.

5.4. Database Relationships

Important relationships include: one hospital containing many rooms; one room containing many nodes or assets; one asset containing many detections; one asset containing many movement requests; and one user requesting or deciding on many movement requests.

Some relationships are optional. For example, a detection event may not match an asset when the RFID tag is unknown, and an activity event may not have a user when it is generated automatically by the system.

5.5. Data Validation Rules

Data validation should be applied at both the frontend and backend, but backend validation is authoritative. The backend must validate required fields, allowed status values, role permissions, tag uniqueness, node assignment rules, movement-request rules, and state transitions.

The frontend validation improves user experience by catching simple errors early, while backend validation protects the database and ensures consistent business behavior.

5.6. Status and Enum Values

The system uses status values for nodes, assets, movement requests, alerts, and user accounts. Examples include DISCOVERED, ASSIGNED, ONLINE, OFFLINE, DISABLED, IN_PLACE, ON_THE_MOVE, UNAUTHORIZED, PENDING, APPROVED, REJECTED, CANCELLED, COMPLETED, OPEN, and RESOLVED.

Status values should be centralized in constants or enums so that backend logic, frontend display, tests, and documentation remain consistent.

Table 5. 6. 1 List of system's status and enum values.

No.	State owner	Value	Meaning	Storage or display note
1.	Node	DISCOVERED	The backend has accepted a previously unknown node.	Stored lifecycle/status value.
2.	Node	ASSIGNED	The node has an approved role and location configuration.	Stored lifecycle/status value.
3.	Node	ONLINE	A valid recent heartbeat or contact confirms availability.	Stored or derived operational value.
4.	Node	OFFLINE	The heartbeat timeout has been exceeded.	Stored or derived operational value.
5.	Node	DISABLED	The node has been administratively disabled.	Stored lifecycle/status value.
6.	Node	DELETED	The eligible node record has been removed according to policy.	Terminal lifecycle outcome, not necessarily a visible table filter.
7.	Node	UNKNOWN	Unexpected or unrecognized node status.	Frontend fallback only; not a normal stored state.
8.	Asset lifecycle	UNREGISTERED	No active tracked asset record exists for the tag.	Conceptual pre-registration state.
9.	Asset lifecycle	ACTIVE	The asset is registered and included in active tracking.	Stored active state.
10.	Asset lifecycle	DEREGISTERED	The asset is no longer actively tracked; history is retained.	Stored inactive/history state.
11.	Asset flow	ON_THE_MOVE	The asset is awaiting detection at its expected or approved destination.	Displayed in both applications.
12.	Asset flow	IN_PLACE	The latest valid detection matches the expected room.	Displayed in both applications.
13.	Asset flow	PENDING_APPROVAL	A movement request exists and awaits a decision.	Displayed separately from the request status where needed.
14.	Asset flow	UNAUTHORIZED	The latest detection is in an unexpected room without a valid completed movement.	May generate an alert.
15.	Movement request	PENDING	The request has been submitted and awaits approval or rejection.	Persistent request status.
16.	Movement request	APPROVED	Movement is authorized but not yet completed.	Persistent request status.
17.	Movement request	REJECTED	The request was denied.	Terminal request status.
18.	Movement request	CANCELLED	The eligible request was cancelled.	Terminal request status.
19.	Movement request	COMPLETED	The asset was detected at the approved destination.	Terminal request status.
20.	Alert	OPEN	The underlying warning condition currently requires attention.	Persistent alert status.
21.	Alert	RESOLVED	The underlying condition has cleared or was resolved by policy.	Persistent alert status.

22.	Session	SIGNED_OUT / AUTHENTICATING / AUTHENTICATED / TOKEN_EXPIRED / ACCESS_DENIED	User-facing authentication and authorization states.	Frontend session state.
23.	WebSocket	DISCONNECTED / CONNECTING / CONNECTED / RECONNECTING / STOPPED	Browser live-update connection states.	Frontend connection state.

5.7. Time and Date Handling

The backend should store timestamps in a consistent format, and the frontend should display them in the configured local timezone. This is important because heartbeat timeout logic, last-seen values, movement decisions, and activity records all depend on correct time interpretation.

The design should avoid mixing server-local time, browser-local time, and manually formatted timestamps without a clear conversion rule.

5.8. Data Integrity Rules

Data integrity rules ensure that assets, nodes, movements, and events remain consistent. One active tag should not be assigned to multiple active assets. One asset should not have more than one pending movement request. A movement should not be completed unless the asset is detected at the approved destination.

Node assignment rules should prevent invalid combinations of roles and rooms. For example, a Checkpoint Node requires a room assignment, whereas a Registration Desk Node may not.

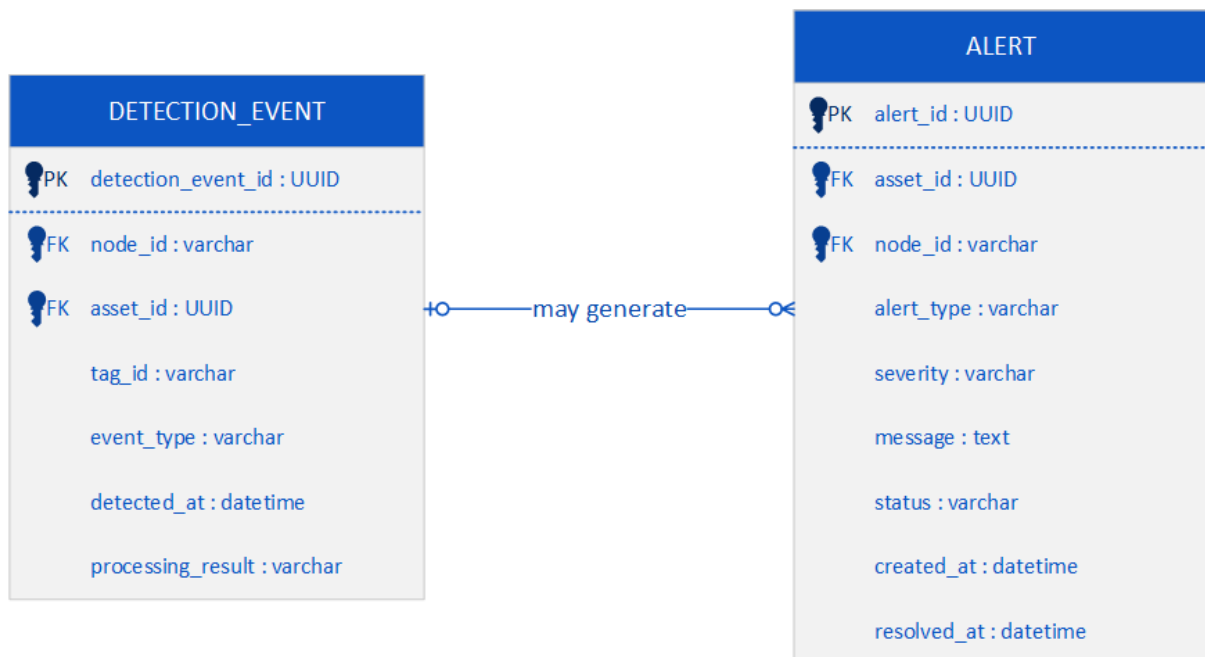


Figure 5. 8. 1 Detection event and alert relationship.

6. Software Class and Component Design

This section documents the code-level structure of the embedded firmware, the backend, the Registration App, and the Monitor App. The firmware and backend are represented with compact class or module-relation diagrams. React applications are represented using component-structure diagrams because their implementation is based primarily on function components, custom hooks, services, contexts, and utility modules rather than constructor-based classes.

6.1. Firmware Class and Interface Design

The firmware diagrams describe the principal ESP32 modules, configuration structures, and abstraction interfaces. The compact main relation diagram should be used as the primary firmware overview. Individual class and interface diagrams provide implementation detail and may be placed in the relevant subsection or moved to the appendix if the main report becomes too long.

The firmware class design should show the most important classes and interfaces involved in node operation. Important classes include NodeManager, MyNodeWiFi, MyNodeNetworkConfig, MyNodeMQTT, MyNodeRFID, NodeStatusLED, NodeBoardControl, and NodeClock.

The diagram should avoid listing every private helper method. It should focus on the main constructor, public responsibilities, important member variables, and relationships between firmware modules.

6.1.1. Core Firmware Classes and Main Relations

The image below explains the class that coordinates node setup, runtime behavior, communication, and configuration.

NodeManager

```
-my_node_wi_fi: IMyNodeWiFi&
-my_node_net_config: IMyNodeNetworkConfig&
-my_node_mqtt : IMyNodeMQTTConnector&
-my_node_rfid : IMyNodeRFID&
-my_node_status_led : INodeStatusLED&
-my_clock : INodeClock&
-my_board : INodeBoardControl&
-active_instance : NodeManager*
-----
+NodeManager(IMyNodeWiFi&, IMyNodeNetworkConfig&, IMyNodeMQTTConnector&,
  IMyNodeRFID&, INodeStatusLED&, INodeClock&, INodeBoardControl&)
+~NodeManager()
+beginNodeSystem() : void
+runNodeSystem() : void
-myNodeMQTTCallback(char*, uint8_t*, unsigned int) : void
```

Figure 6. 1. 1. 1 Node manager class.

The image below shows the status indications and the physical blink-identification responsibilities.

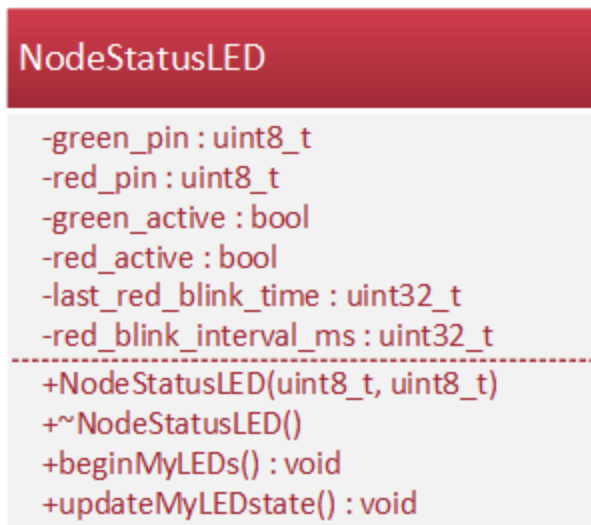


Figure 6. 1. 1. 2 Node status LED class.

The image below shows board-level control responsibilities and the reset or hardware-control boundary.

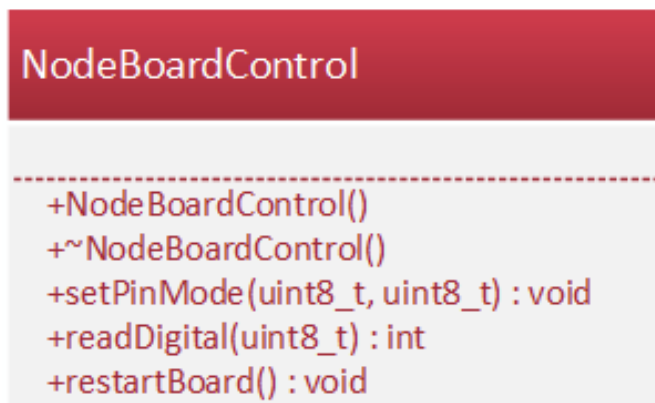


Figure 6. 1. 1. 3 Node board controller class.

The image below shows the time abstraction used for intervals, timeouts, and heartbeat scheduling.

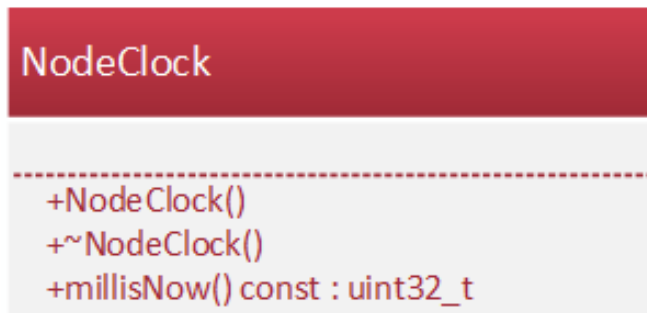


Figure 6. 1. 1. 4 Node clock class.

The diagram below shows the MQTT connection, publishing, subscribing, command, and acknowledgment responsibilities.

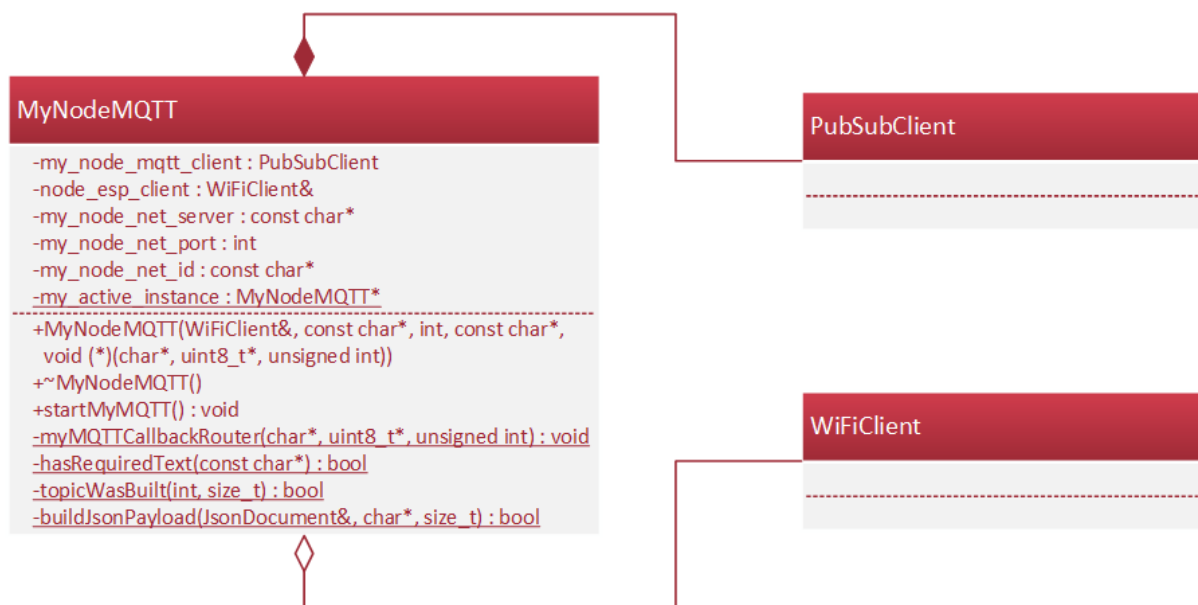


Figure 6. 1. 1. 5 Node MQTT class dependencies.

The diagram below shows the configuration responsibilities for the persistent Wi-Fi, broker, and node.

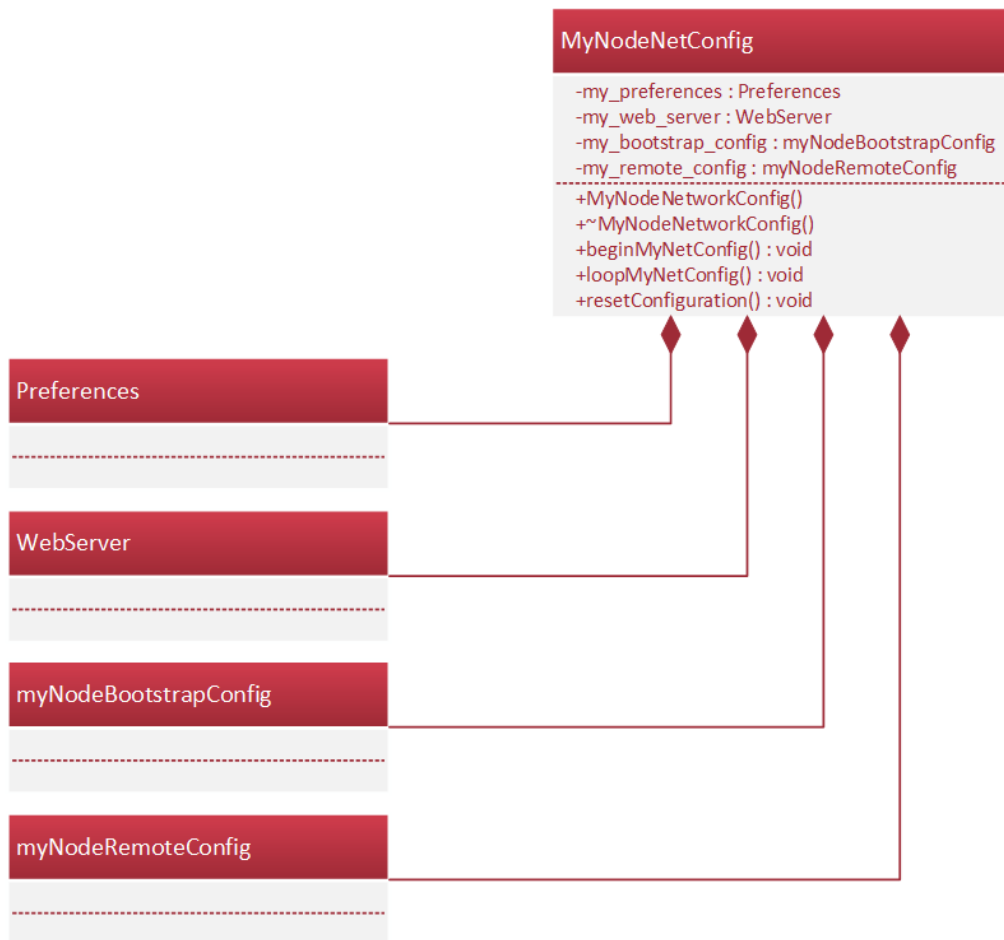


Figure 6. 1. 1. 6 Node network configuration class dependencies.

The image below shows Wi-Fi connection, captive portal, and reconnection behavior.

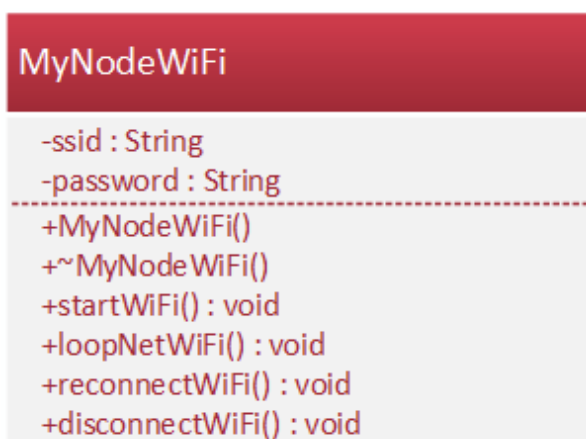


Figure 6. 1. 1. 7 Node Wi-Fi class.

The diagram below shows the RFID reader boundary, tag-reading flow, and scan-result handling.

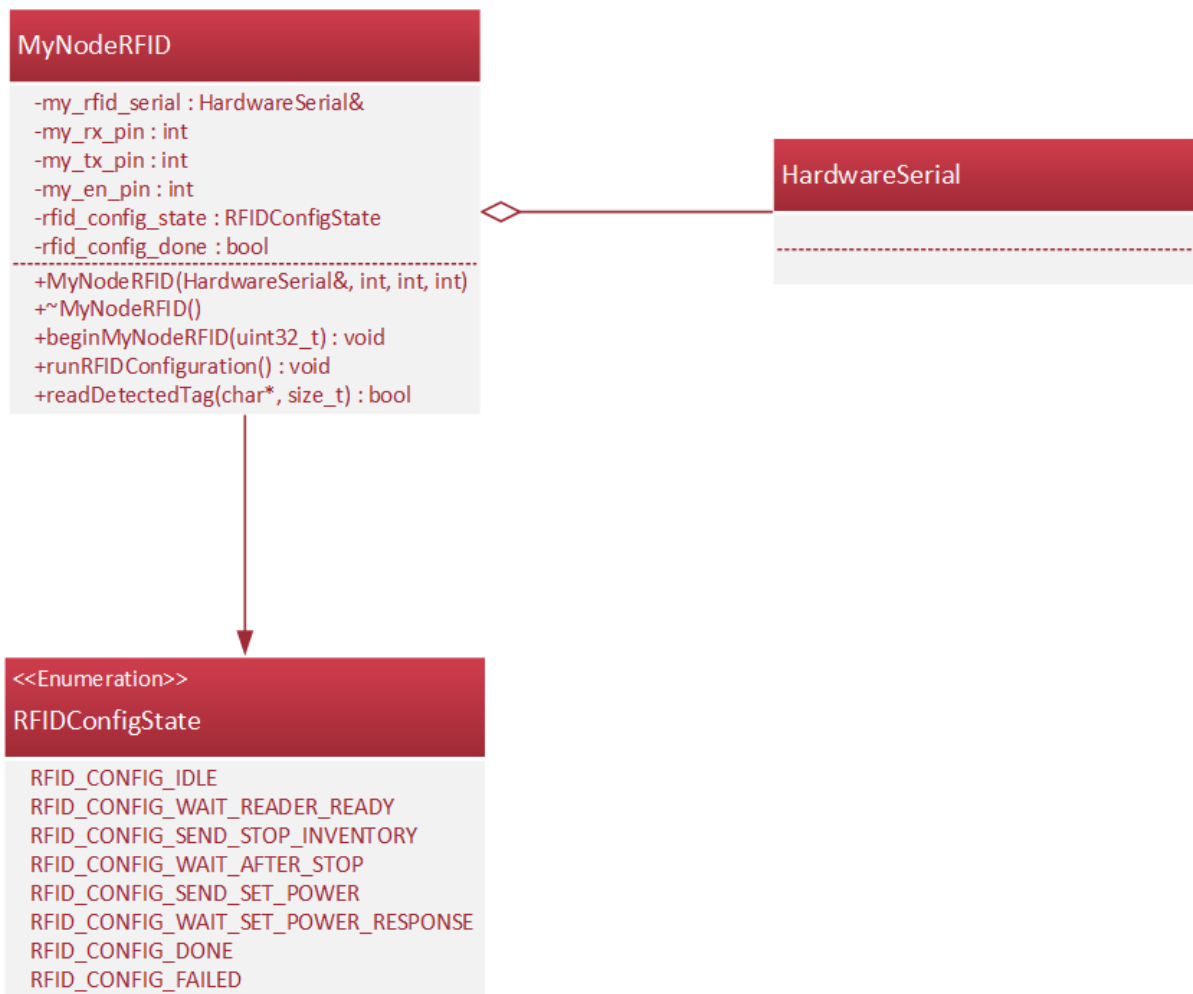


Figure 6. 1. 1. 8 Node RFID and config state relation with hardware serial dependency.

The diagram below shows how the main firmware coordinates the network, MQTT, RFID, LED, clock, and board-control modules.

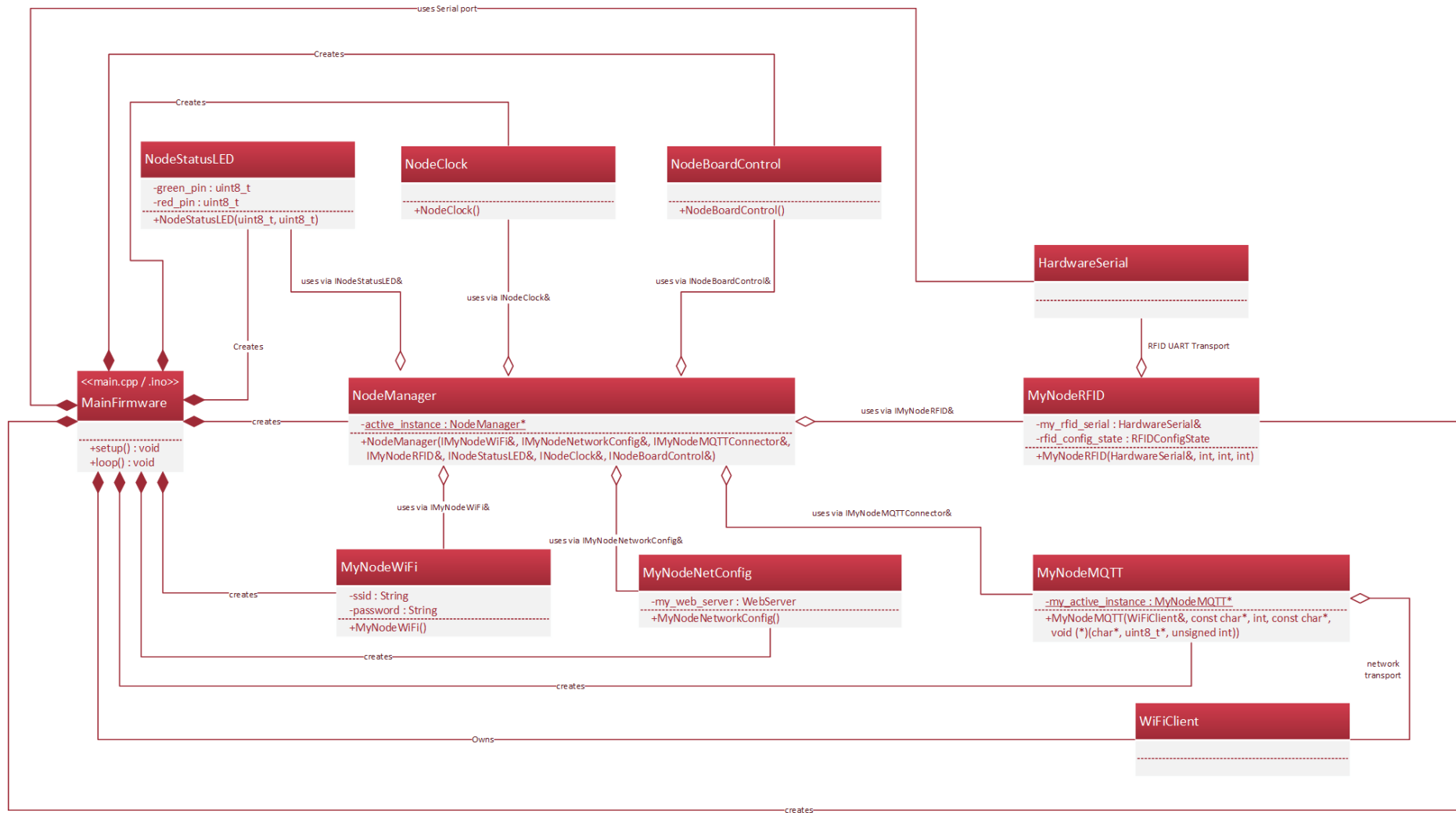


Figure 6. 1. 1. 9 Main module firmware coordination.

The diagram below emphasizes behavioral dependencies between the main firmware and its modules.

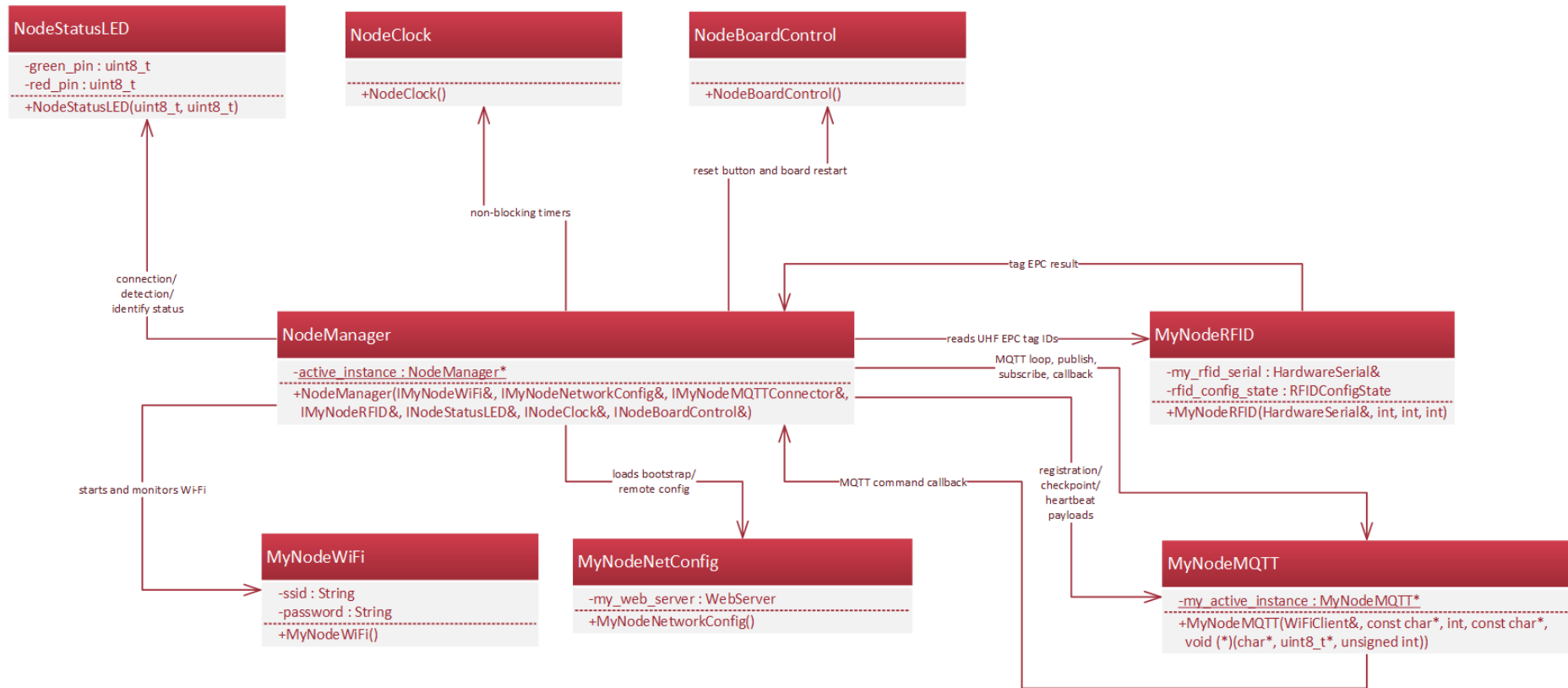


Figure 6. 1. 10 Firmware module dependencies.

6.1.2. Firmware Configuration and Data Structures

Firmware configuration and data structures describe startup configuration, remote configuration, MQTT broker settings, node identity, node role, RFID state, and enum values. These structures should be documented because they explain how the same firmware can support different node roles through configuration.

The image below shows the local configuration values required before the node can connect and identify itself.

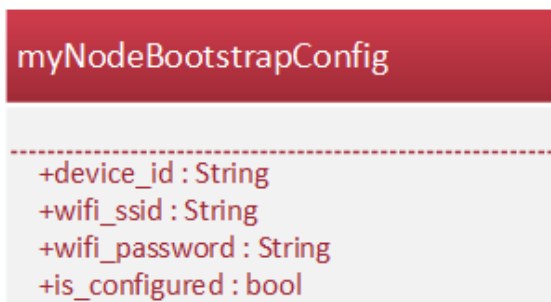


Figure 6. 1. 2. 1 Bootstrap configuration class.

The image below shows configuration values that were received or refreshed from the backend.

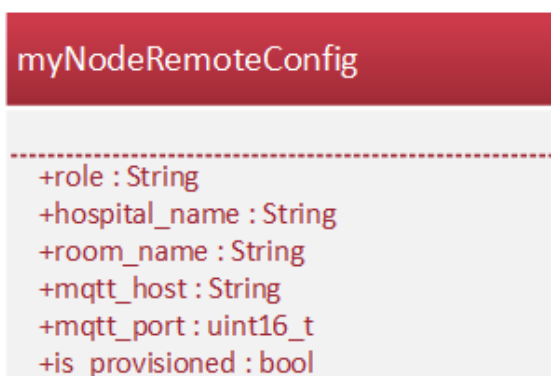


Figure 6. 1. 2. 2 Remote configuration class.

The image below lists important firmware roles, connection states, and configuration states.

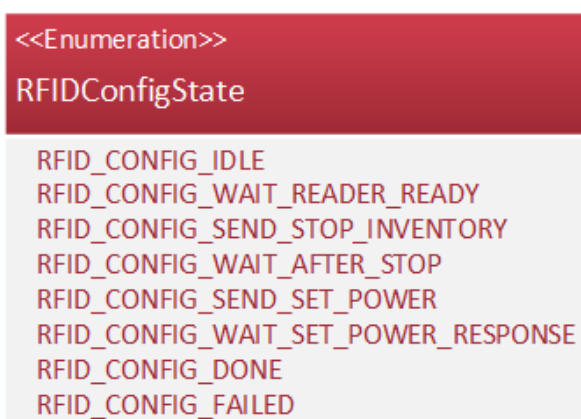


Figure 6. 1. 2. 3 Firmware roles and state enumerations.

6.1.3. Firmware Interface Diagrams

Firmware interface diagrams show stable boundaries between modules. They are useful for testing because mocked implementations can replace real Wi-Fi, MQTT, RFID, clock, LED, or board-control behavior during unit tests.

The diagram below shows the connector, publisher, and subscriber interfaces used by the MQTT implementation.

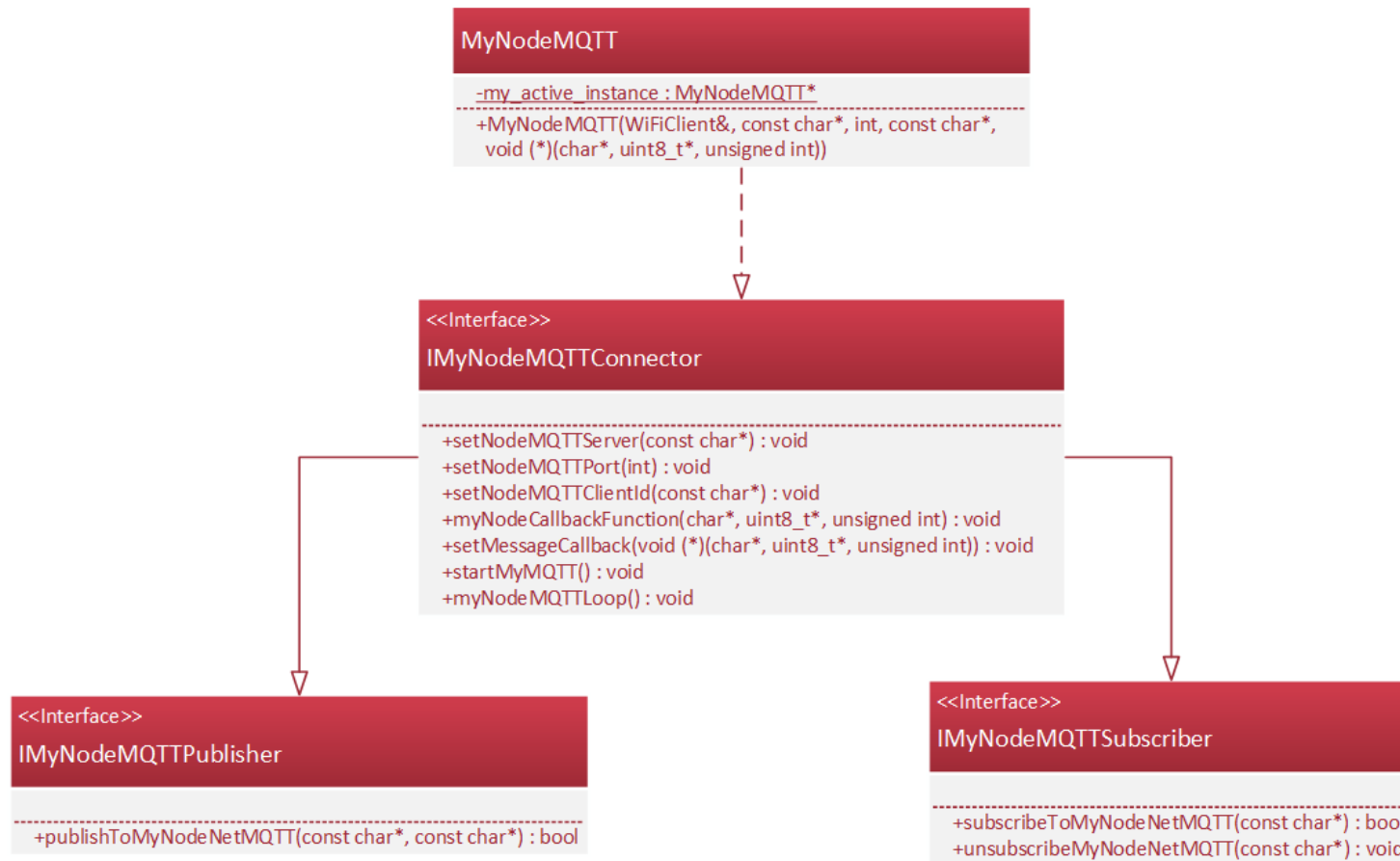


Figure 6. 1. 3. 1 MQTT interface class diagram.

The diagram below shows the abstraction used for Wi-Fi status, connection, and setup behavior.

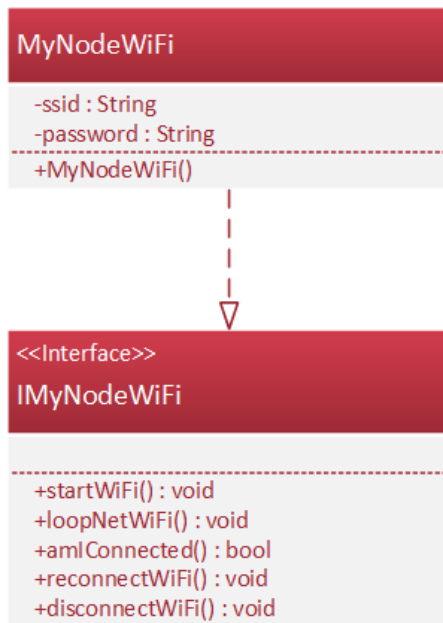


Figure 6. 1. 3. 2 Wi-Fi interface class diagram.

The diagram below shows the interface between runtime logic and stored network or node configuration.

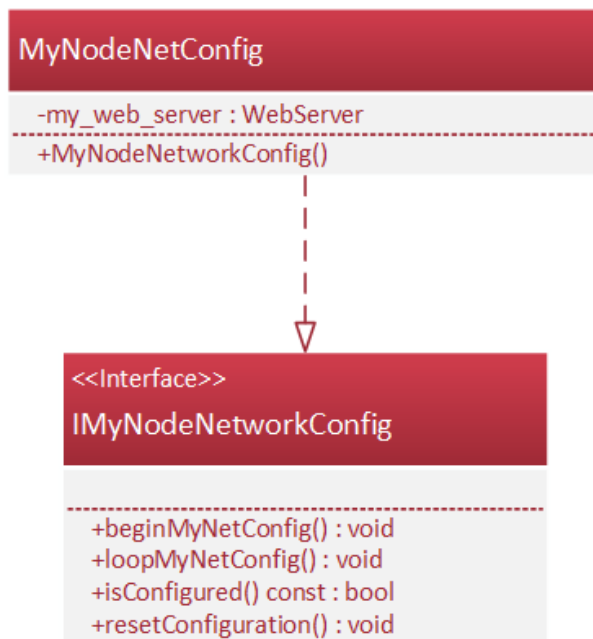


Figure 6. 1. 3. 3 Network configuration interface class diagram.

The diagram below shows the abstraction used by the firmware to read RFID tags.

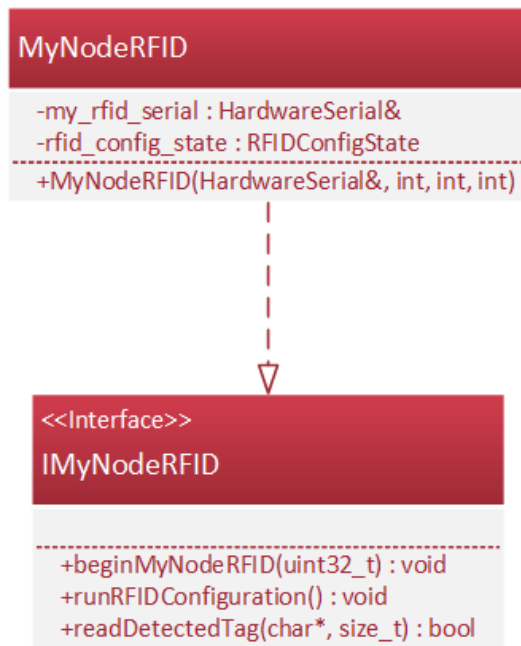


Figure 6. 1. 3. 4 RFID interface class diagram.

The diagram below shows the status LED and blink control abstraction.

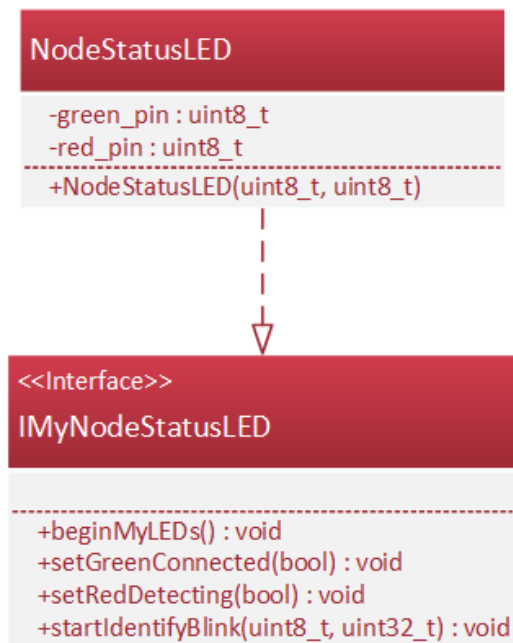


Figure 6. 1. 3. 5 Status LED interface class diagram.

The diagram below shows the time and timer abstraction used by scheduling and timeout logic.

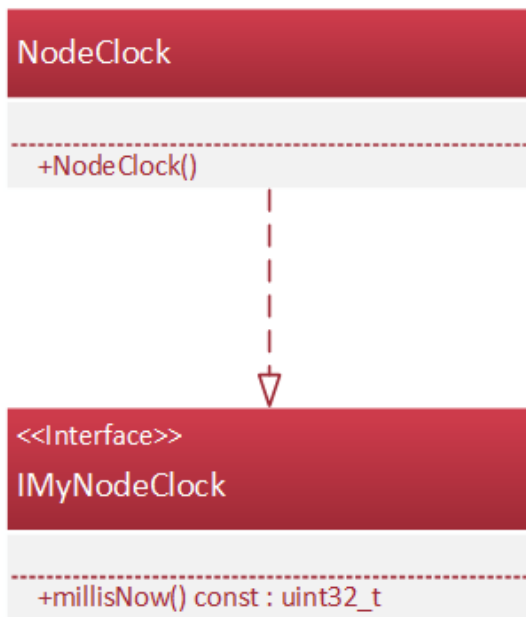


Figure 6. 1. 3. 6 Node clock interface class diagram.

The diagram below shows the hardware-control abstraction used for board-level actions.

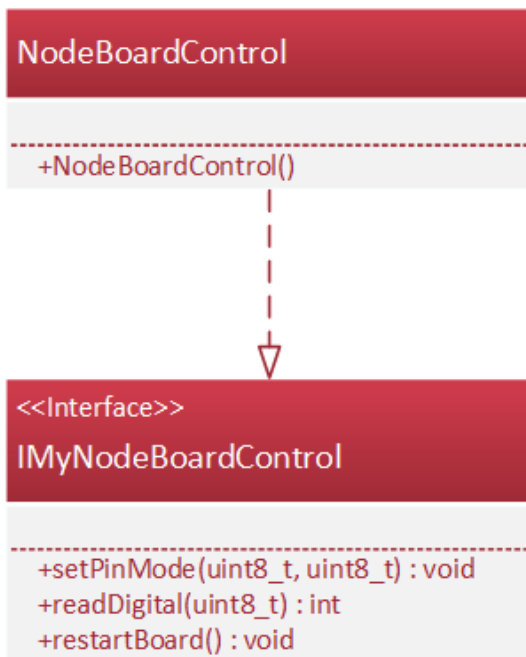


Figure 6. 1. 3. 7 Node board control interface class diagram.

The diagram below combines the main firmware interfaces and their concrete implementations.

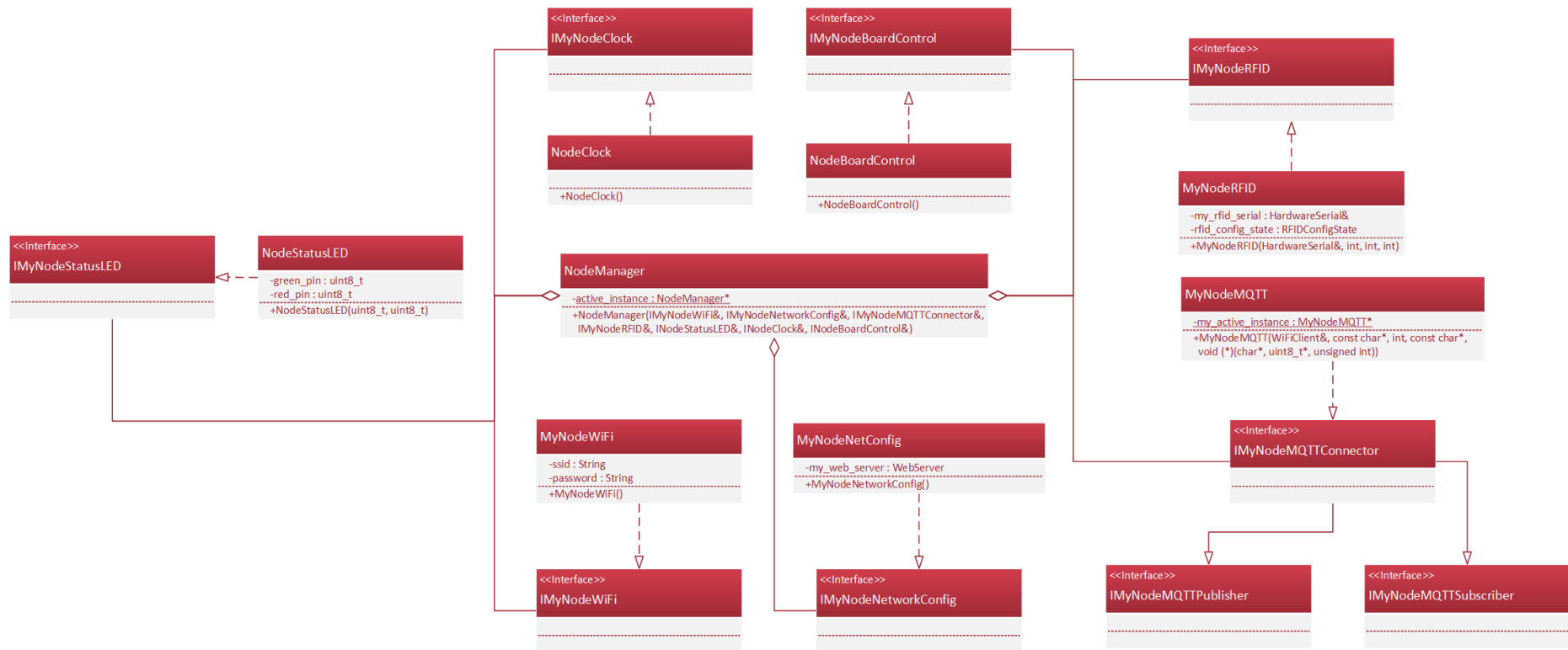


Figure 6. 1. 3. 8 System firmware interface implementation.

6.1.4. Firmware Responsibility Summary

The responsibility summary translates the firmware class and interface diagrams into concise implementation roles. NodeManager coordinates startup and recurring node behavior; MyNodeWiFi maintains network access; MyNodeNetworkConfig loads and saves bootstrap or remote configuration; MyNodeMQTT manages broker connection, subscriptions, publishing, and acknowledgments; MyNodeRFID reads and filters tag observations; NodeStatusLED presents operational status and temporary identification patterns; NodeClock supplies timing and interval checks; and NodeBoardControl isolates reset and board-specific operations. The final summary table should state only these major responsibilities and dependencies rather than reproducing every private helper method.

In a structured way, the system responsibility can be defined as follows:

Table 6. 1. 4. 1 List of the system's firmware responsibilities.

No.	Module	Primary responsibility	Principal dependencies
1.	NodeManager	Coordinates startup, configuration, recurring runtime behavior, MQTT callback routing, RFID polling, heartbeat timing, LED state, and recovery.	Wi-Fi, network configuration, MQTT, RFID, LED, clock, and board-control interfaces.
2.	MyNodeWiFi	Maintains Wi-Fi setup, connection, captive portal behavior, and reconnection.	Stored network configuration and ESP32 Wi-Fi services.
3.	MyNodeNetworkConfig	Loads, validates, saves, refreshes, and resets bootstrap and remote node configuration.	Preferences storage, configuration web server, bootstrap data, and remote configuration data.
4.	MyNodeMQTT	Connects to the broker, subscribes to command topics, publishes scans, detections, heartbeats and acknowledgments, and reconnects after interruption.	Wi-Fi client, MQTT client library, node configuration, and callback routing.
5.	MyNodeRFID	Initializes the UHF reader, reads and normalizes tags, manages reader configuration state, and suppresses repeated observations.	Hardware serial interface, reader pins, configuration state, and MQTT publishing path.
6.	NodeStatusLED	Displays setup, connection, online, offline, error, and temporary identification patterns.	GPIO pins, current node status, blink timing, and clock.
7.	NodeClock	Provides monotonic timing for intervals, retries, cooldowns, timeouts, and heartbeat scheduling.	ESP32 timing service.

6.2. Backend Class and Service Structure

The backend uses SQLAlchemy model classes together with function-oriented service, repository, policy, validation, event-processing, and FastAPI route modules. The diagrams, therefore, use UML class notation with stereotypes such as service module, repository module, policy module, background service, and SQLAlchemy model. This presents the implementation accurately without pretending that every Python module is an object-oriented class.

6.2.1. Backend Domain Models and Layer Overview

This subsection presents the backend as a set of cooperating layers. The compact layer overview shows how FastAPI routes delegate business work to service modules, how repositories and SQLAlchemy models provide persistence, and how MQTT processing and WebSocket broadcasting connect asynchronous events to user-visible updates. The domain-model diagram complements that overview by showing the principal persistent entities and the associations needed for users, nodes, assets, detections, movement requests, alerts, activity records, and settings.

The diagram below illustrates the relationships among API routes, core services, event processing, real-time communication, and database models.

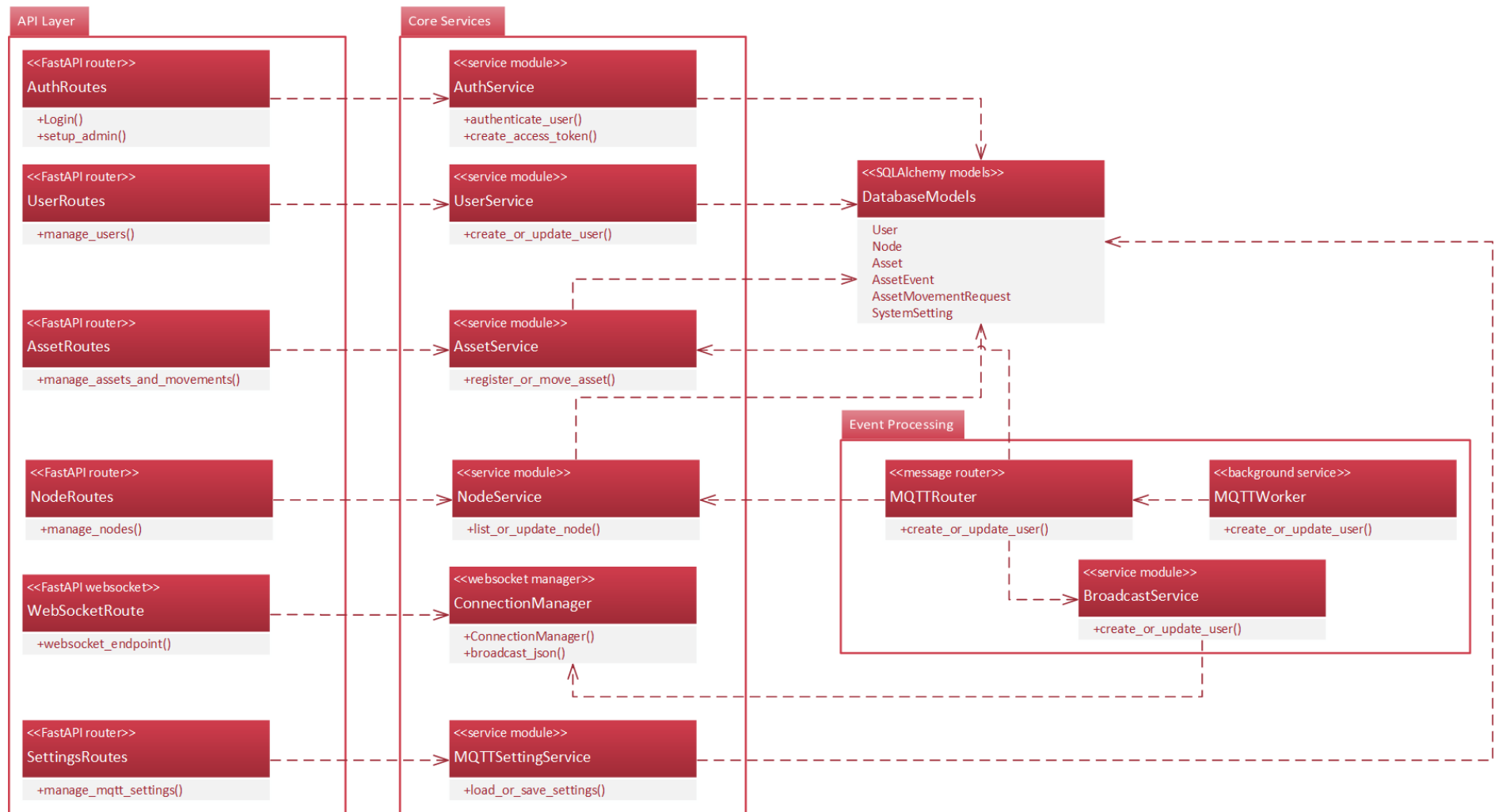


Figure 6. 2. 1. 1 Backend layer overview.

The diagram below shows the important persistent models and their principal fields and associations.

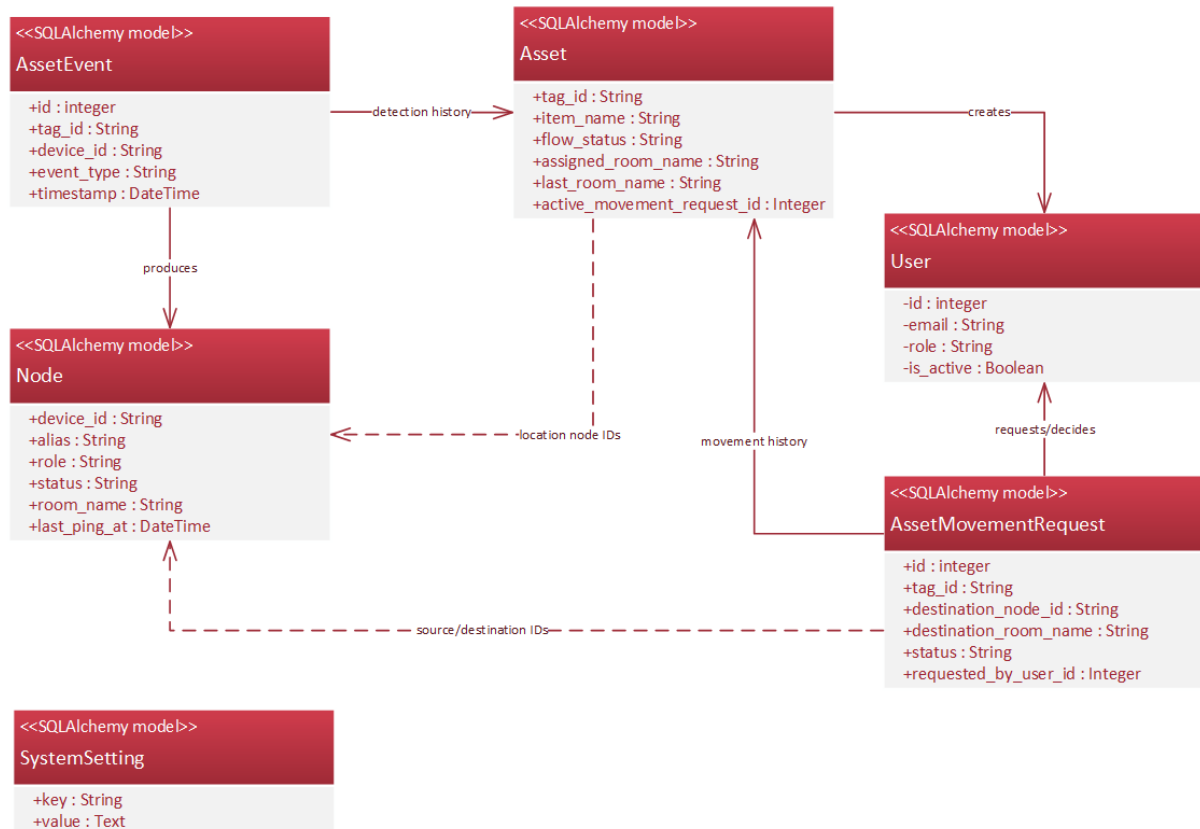


Figure 6.2. 1. 2 Backend domain models.

6.2.2. Authentication, User, and Node Services

The diagrams in this subsection explain two protected areas of the backend. The authentication and user diagram covers credential verification, password hashing, token creation, first-admin setup, account changes, role validation, and the policy that protects the last active administrator. The node-management diagram covers discovery, assignment, location and role validation, un-assignment, deletion eligibility, heartbeat processing, offline detection, and exceptions used for controlled dummy-node testing. Together, these diagrams show where authorization and operational node rules are enforced.

The diagram below shows authentication dependencies, first-admin setup, user validation, password handling, and last-active-admin safety policy.

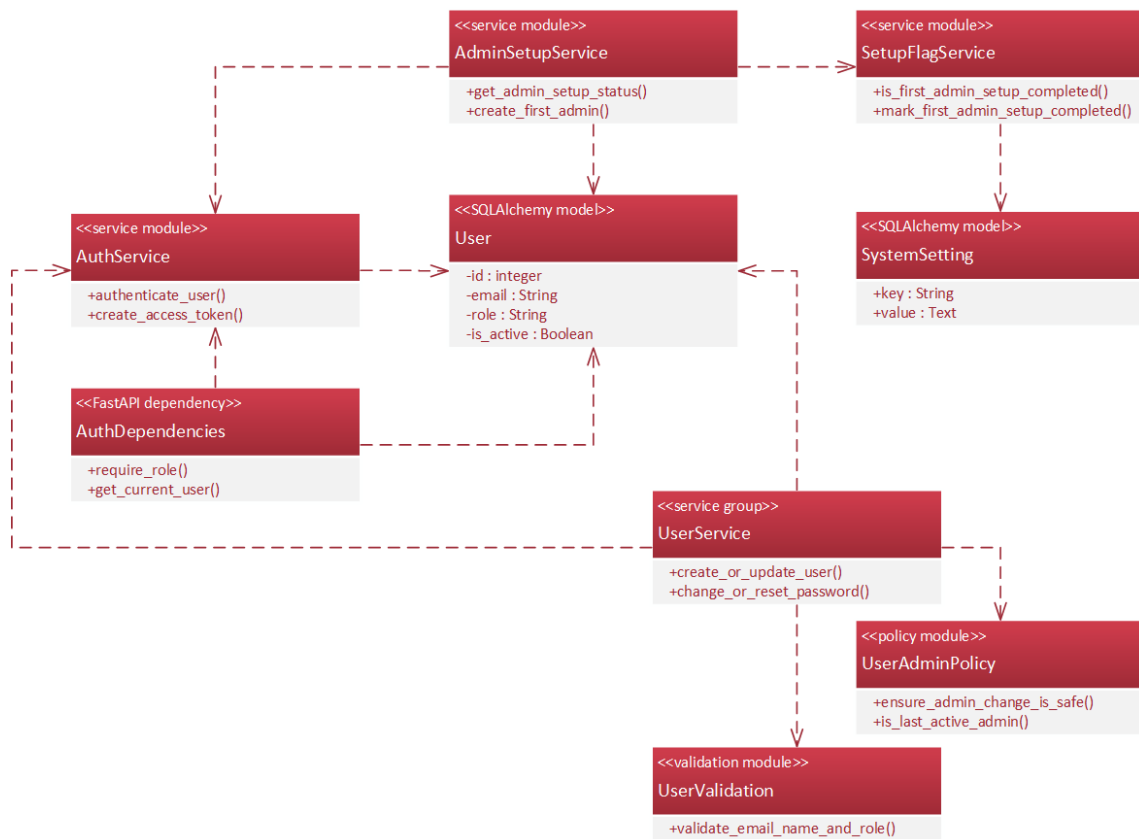


Figure 6. 2. 2. 1 System authentication and user services.

The diagram below shows assignment, validation, repository access, reference checks, heart-beat/offline monitoring, and demo-node policy.

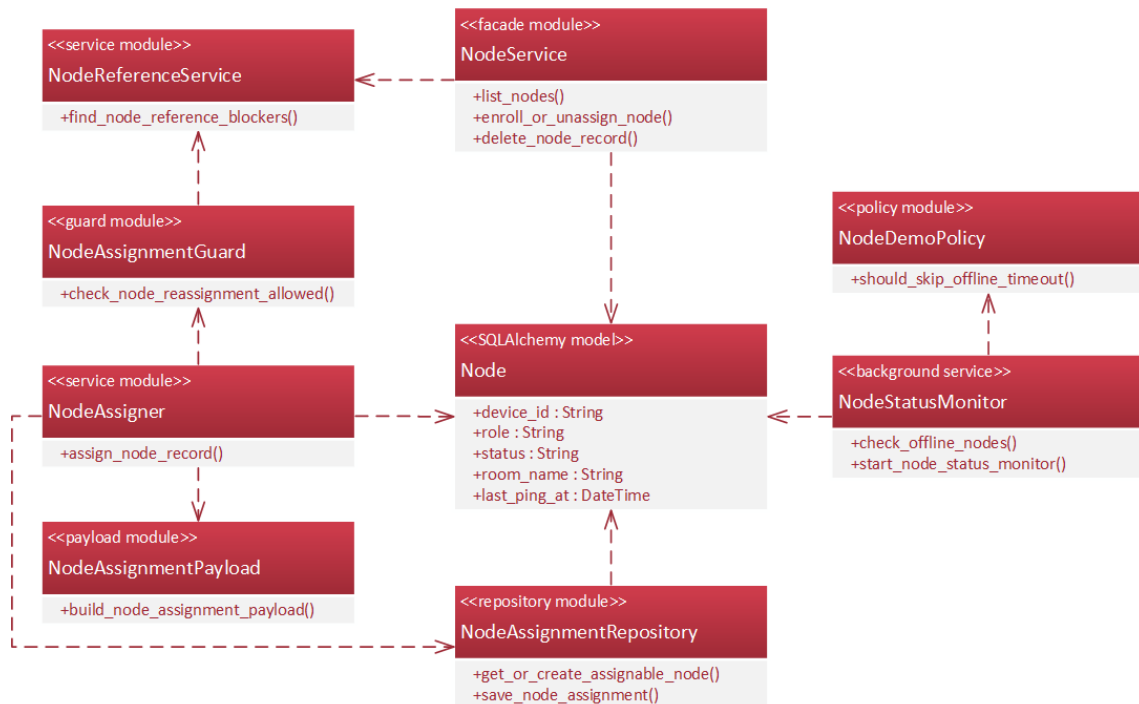


Figure 6. 2. 2. 2 System management services.

6.2.3. Asset Registration, Movement, and Detection Services

This subsection describes the backend services that control the asset lifecycle and movement workflow. Registration services validate a scanned tag, active-tag uniqueness, registration-node state, asset data, and assigned-room information before creating or deregistering an asset. Movement services create, cancel, approve, reject, and complete movement requests while preventing duplicate pending requests. Detection and flow-resolution services process checkpoint observations, update the latest location, determine whether the asset is in place or unauthorized, complete an approved movement at its destination, and generate related events or alerts.

The diagram below shows the registration facade, validation, repository, Asset model, and Node dependency.

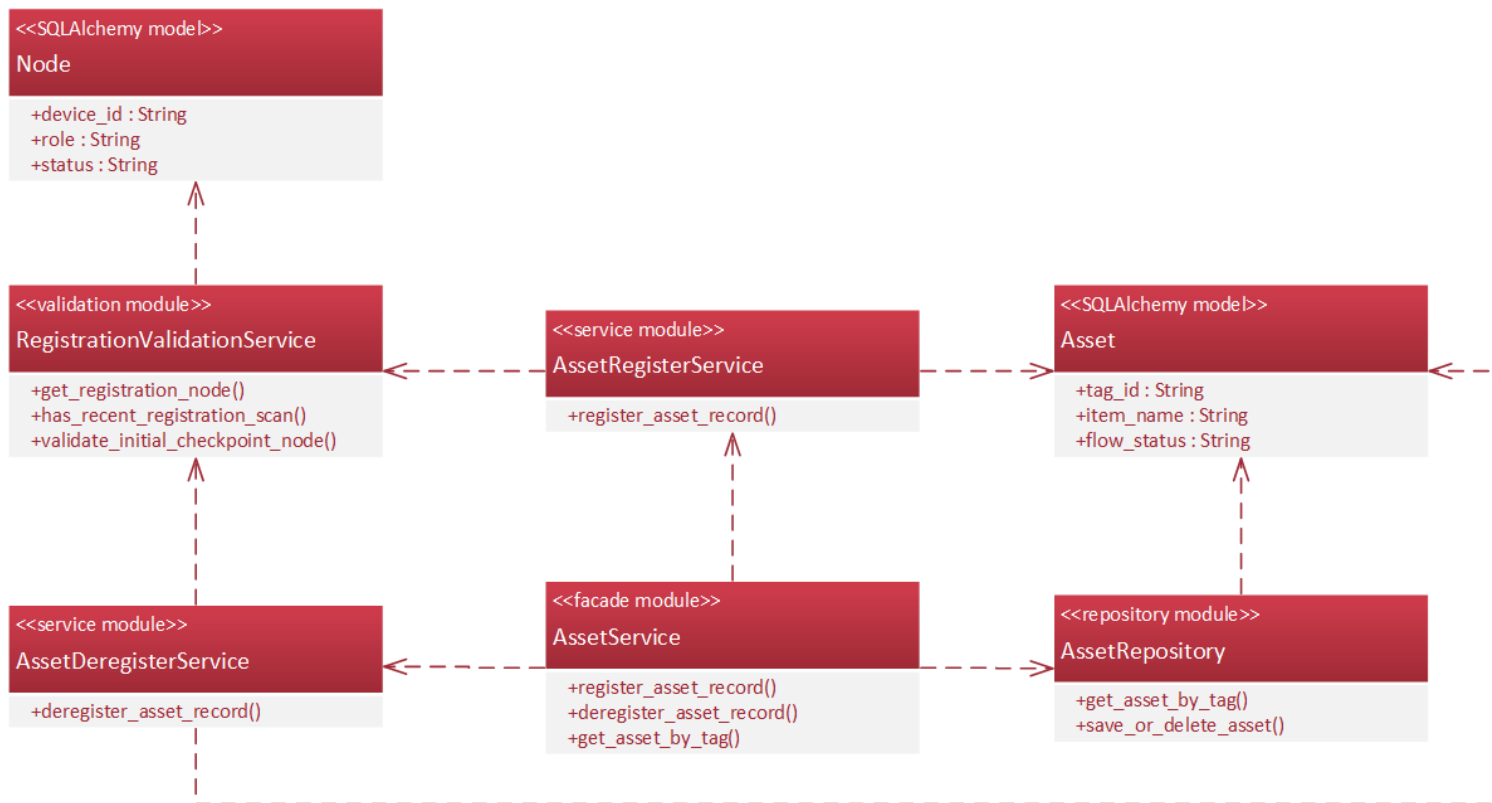


Figure 6. 2. 3. 1 Asset registration services.

The diagram below shows the creation, approval, rejection, cancellation, validation, execution, and persistence of movement.

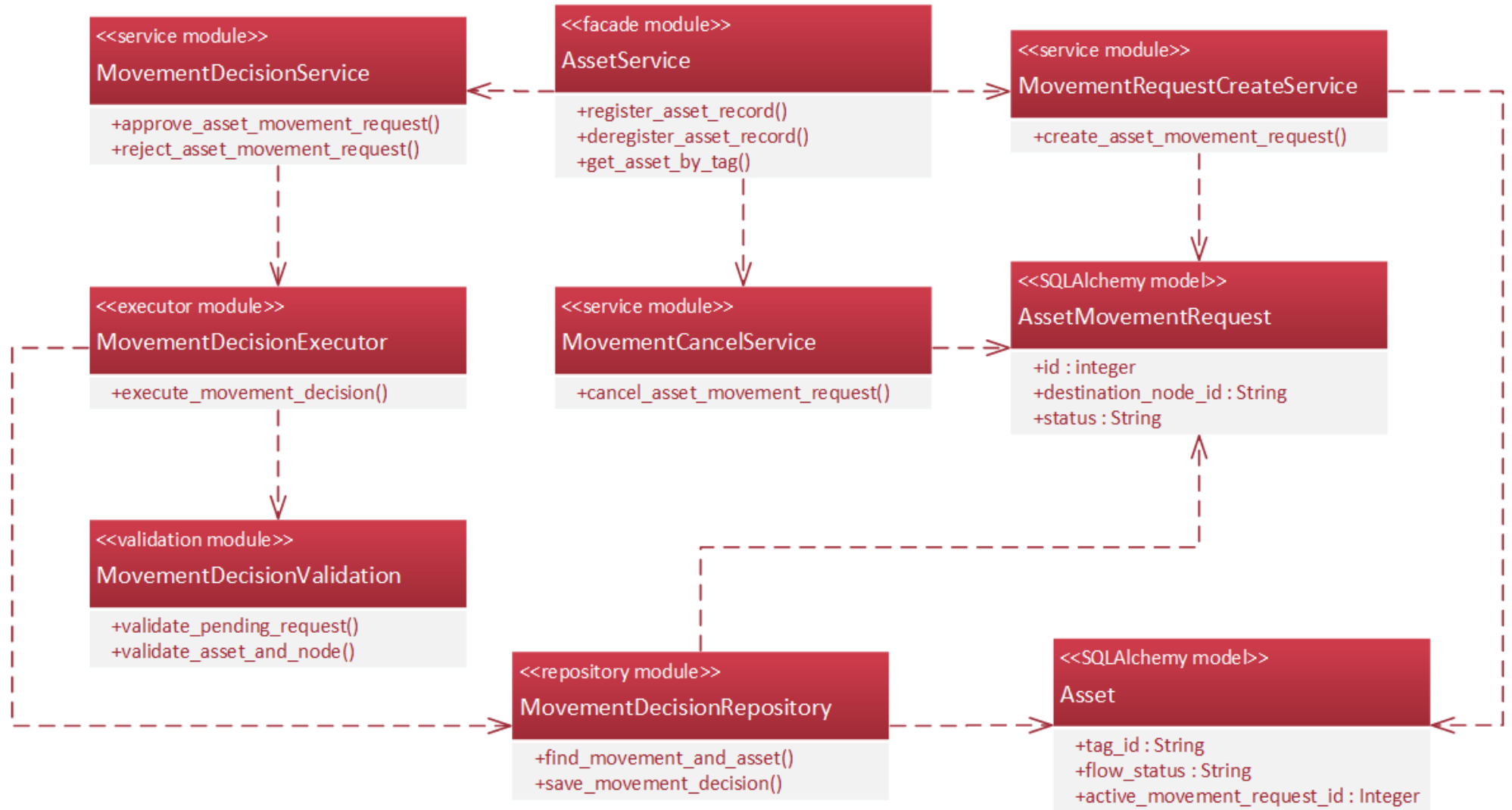


Figure 6. 2. 3. 2 Asset movement services.

The diagram below shows how an MQTT detection updates location, resolves flow state, completes approved movement, and writes events.

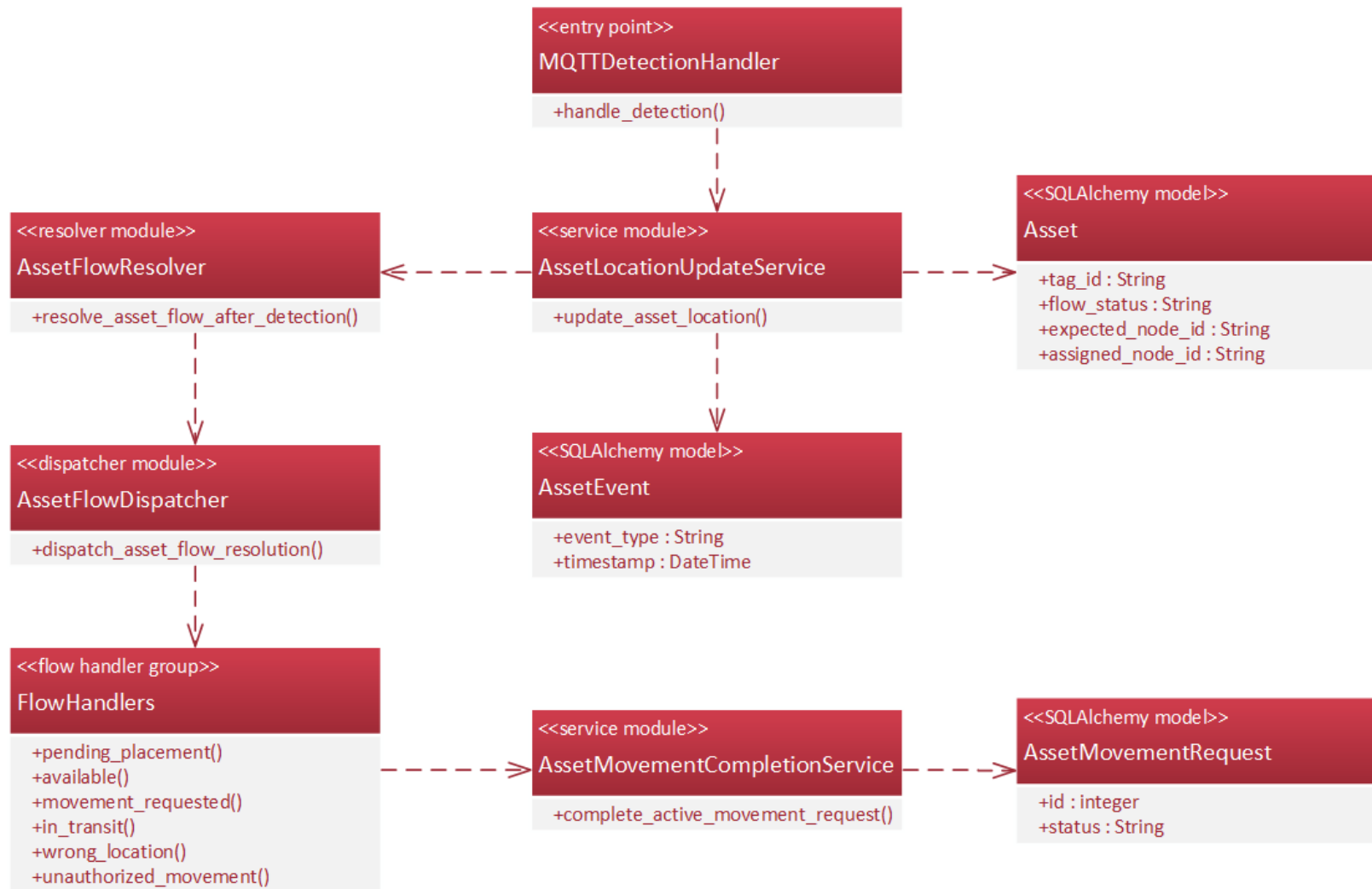


Figure 6. 2. 3. 3 Asset detection and flow resolution

6.2.4. MQTT Processing and API Schemas

The MQTT diagram shows how the backend subscribes to configured topics, validates message structure, routes registration scans, RFID detections, heartbeats, and acknowledgments to the correct handlers, and publishes supported commands to sensor nodes. It also shows the connection from processed MQTT events to database services and WebSocket broadcasts. The API-schema diagram documents the main Pydantic request and response objects used at the HTTP boundary, ensuring that route inputs, validation, and returned data remain consistent across authentication, node, asset, movement, and configuration features.

The diagram below shows MQTT settings, workers, routing, topic helpers, event handlers, publishing, asset location updates, and WebSocket broadcasting.

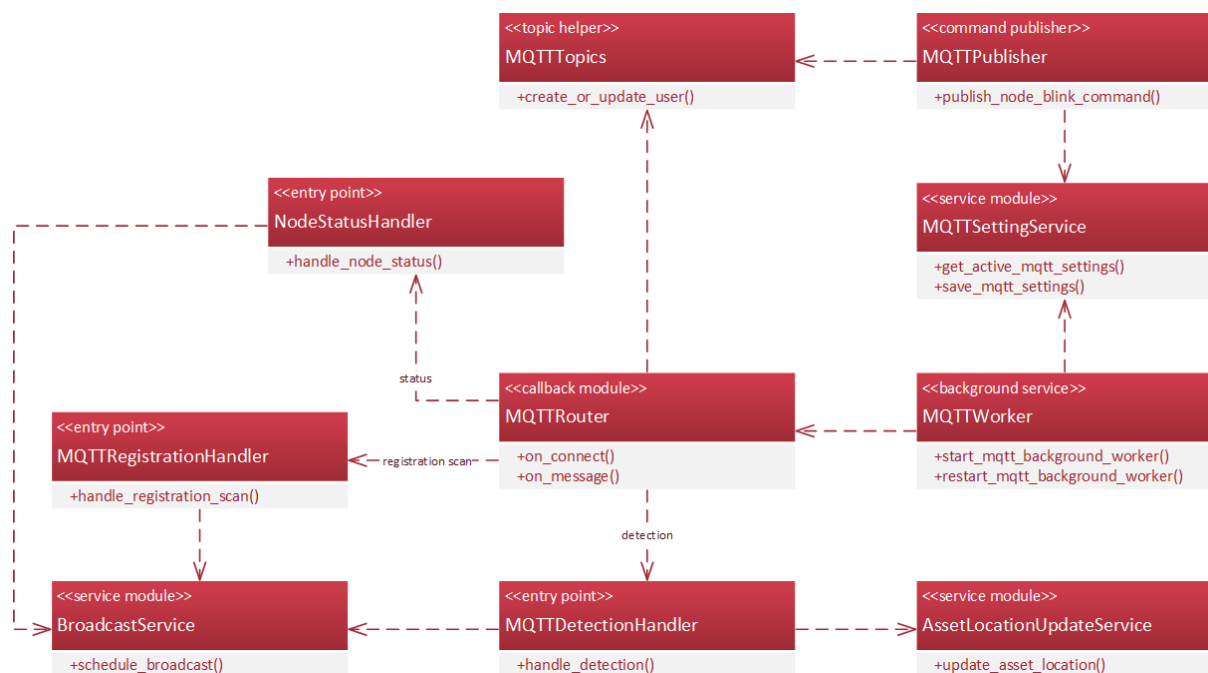


Figure 6. 2. 4. 1 Backend MQTT processing.

The diagram below shows the main Pydantic request and response schemas grouped by authentication, users, nodes, assets, movement, and configuration.

This schema groups the request and response schemas used for login, first-admin setup, token/session results, and user account information.

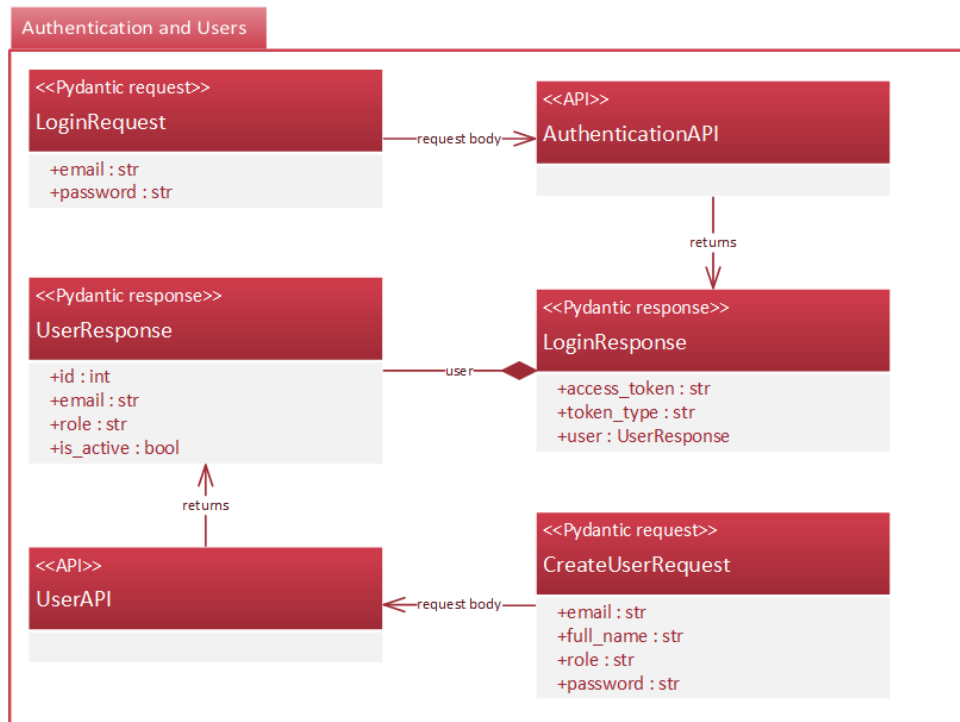


Figure 6. 2. 4. 2 Authentication and user API schemas.

The schema below groups node discovery, assignment, update, status, blink-command, and node response schemas.

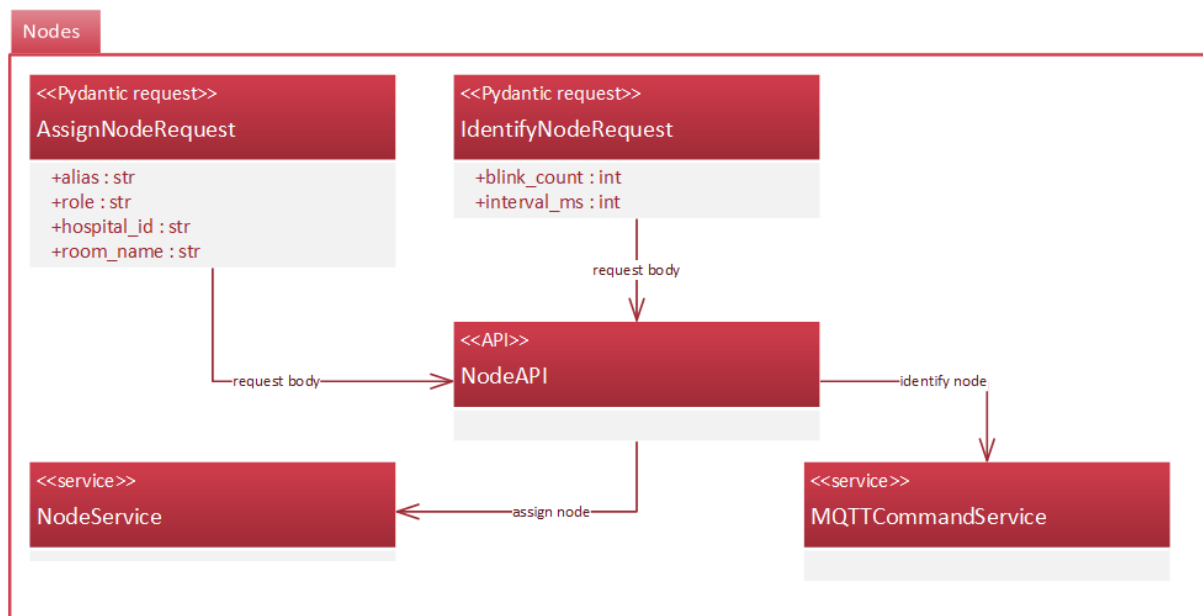


Figure 6. 2. 4. 3 Node API schemas.

The schema below groups asset registration, deregistration, list, location, scan, and asset response schemas.

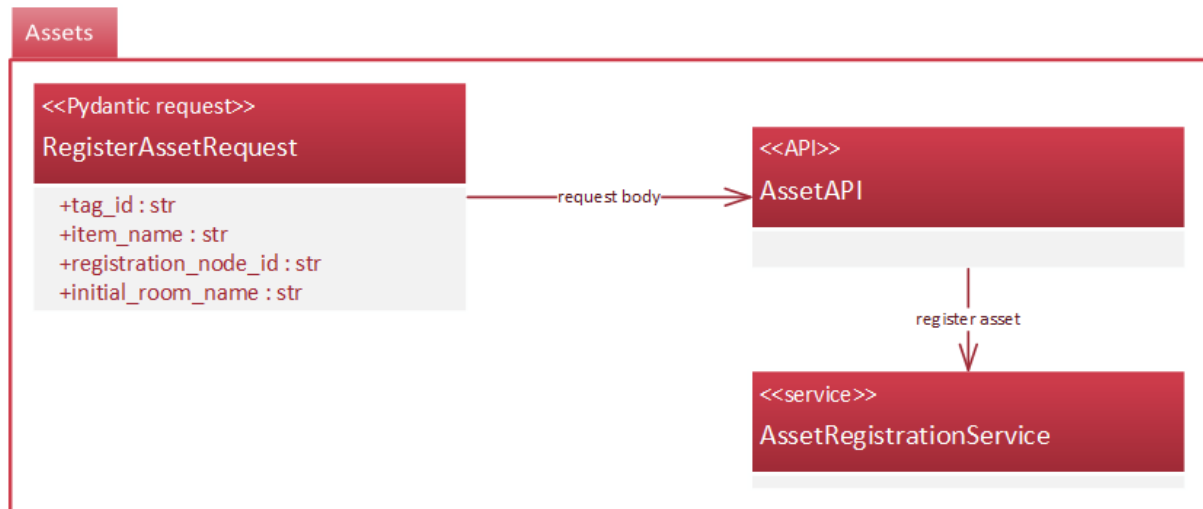


Figure 6. 2. 4. 4 Asset API schemas.

The schema below groups movement request creation, decision, cancellation, completion, and response schemas.

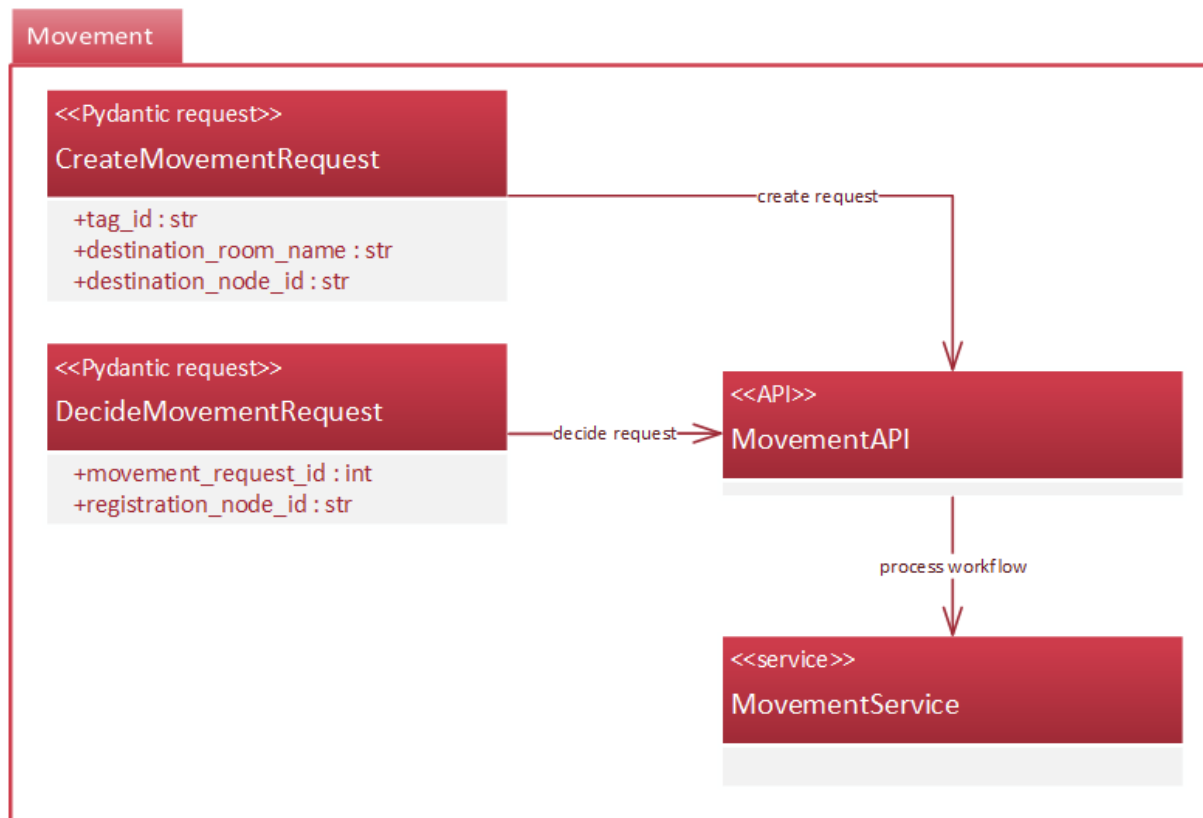


Figure 6. 2. 4. 5 Movement request API schemas.

The schema below groups system-setting and configuration request/response schemas used at the HTTP boundary.

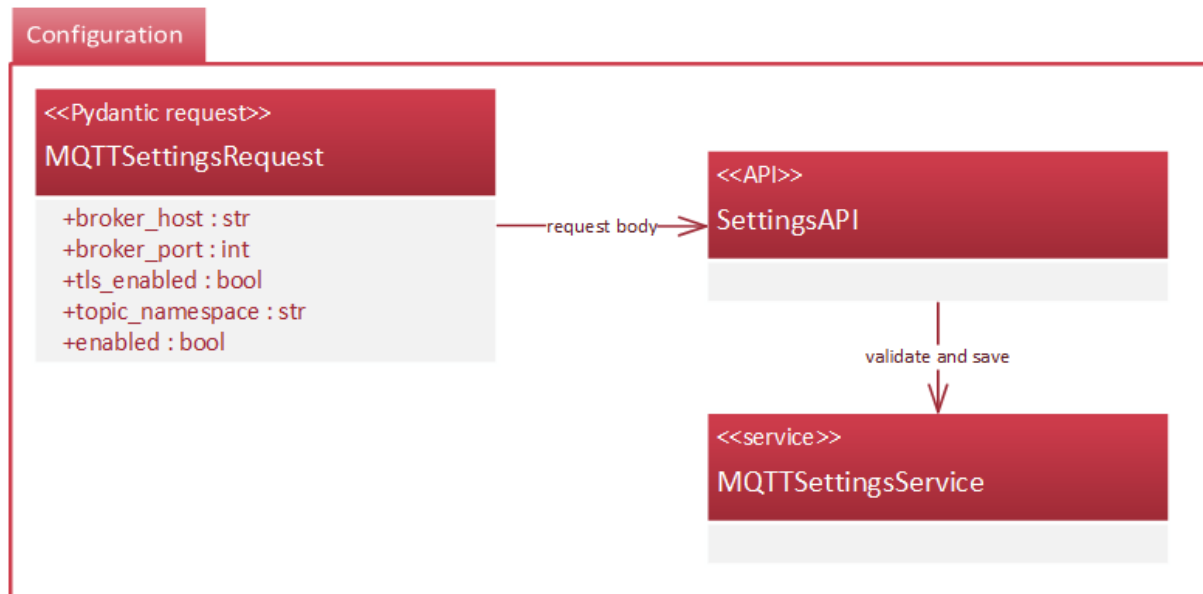


Figure 6. 2. 4. 6 Configuration API schemas.

6.3. Registration App Component Structure

The Registration App is built with React function components and custom hooks. Component diagrams are therefore used instead of traditional class diagrams. The diagrams show page composition, routing, context, state hooks, API services, real-time communication, and shared UI modules.

6.3.1. Registration App Architecture and Authentication

The architecture overview shows how the Registration App is divided into the application shell, routed pages, feature components, custom hooks, service modules, utilities, and shared UI components. The authentication diagram focuses on login, first-admin routing, session restoration, role-aware navigation, protected routes, logout, and redirect behavior after token expiry or backend access denial. These diagrams should make clear that the frontend improves navigation and usability, while the backend remains authoritative for authentication and authorization.

The diagram below shows the application shell, pages, feature components, hooks, services, context, and external backend connection.

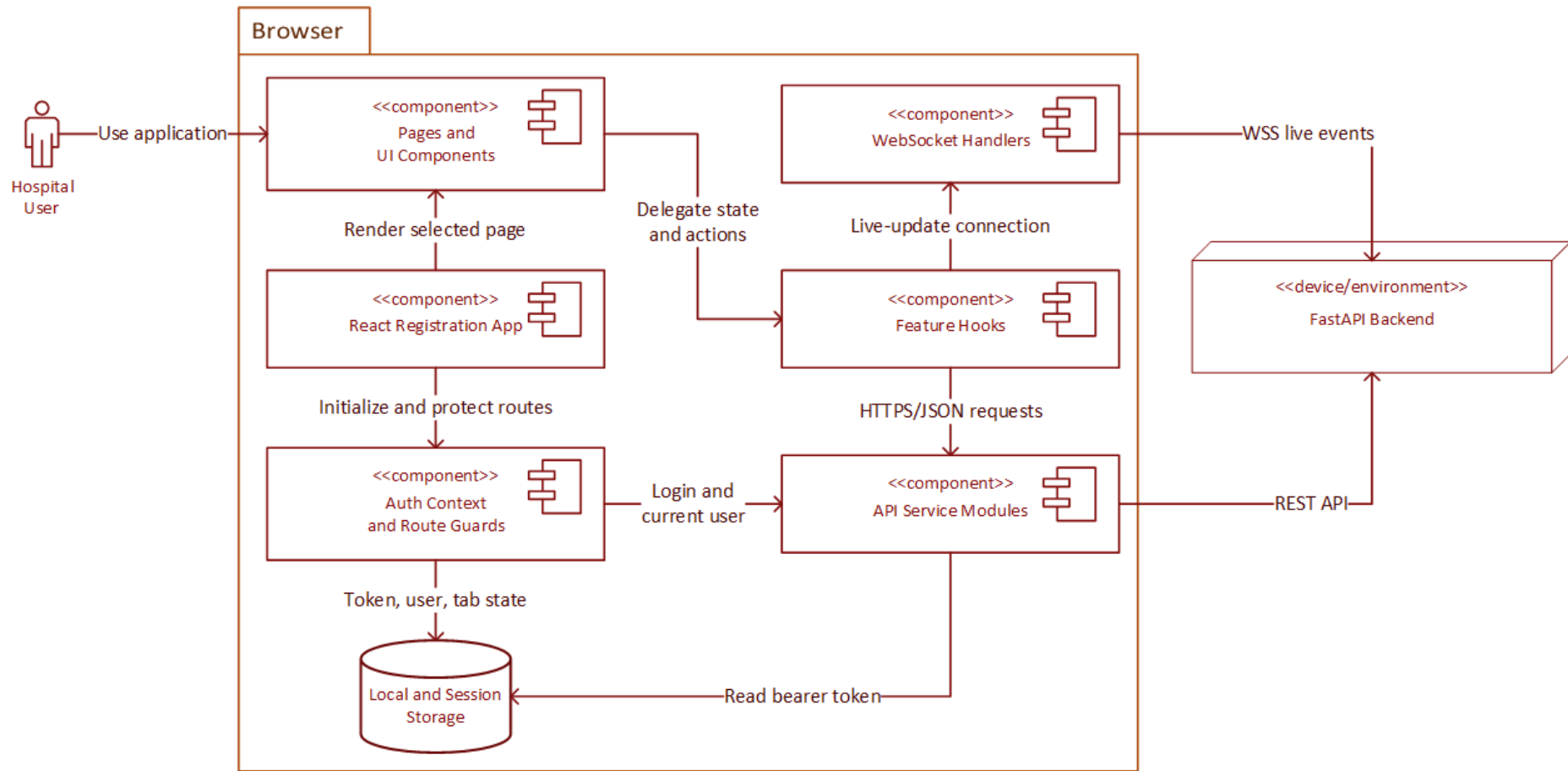


Figure 6. 3. 1. 1 Registration app architecture.

The diagram below shows route protection, authentication context, session helpers, login flow, role-aware navigation, and backend authentication service.

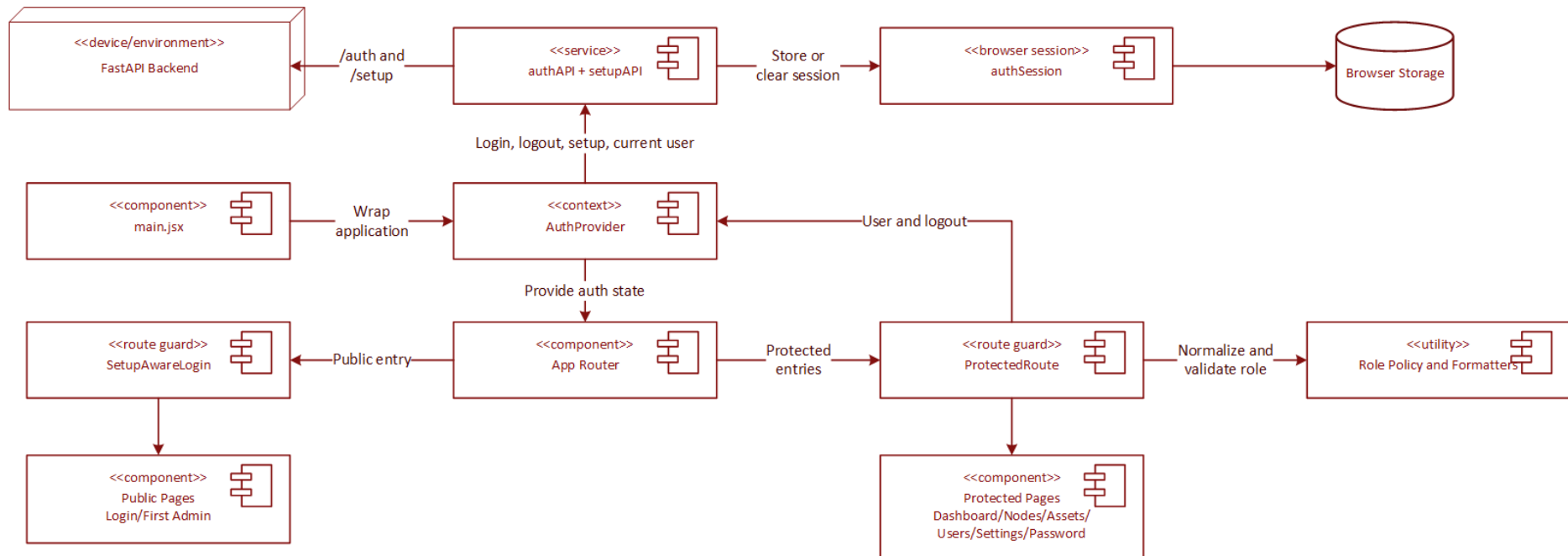


Figure 6. 3. 1. 2 Registration app authentication and routing.

6.3.2. Registration App Feature Components

The feature diagrams decompose the Registration Dashboard, Node Registration page, Asset Registration page, movement-decision area, User Management page, Account Security page, and System Settings page. They show which page components compose forms and tables, which hooks own loading and interaction state, which API services perform requests, and which utility modules normalize status, date, text, sorting, and pagination behavior. This structure keeps page JSX focused on composition while moving reusable logic into hooks, services, and utilities.

The diagram below shows dashboard page composition, role-aware cards, summary data, and navigation dependencies.

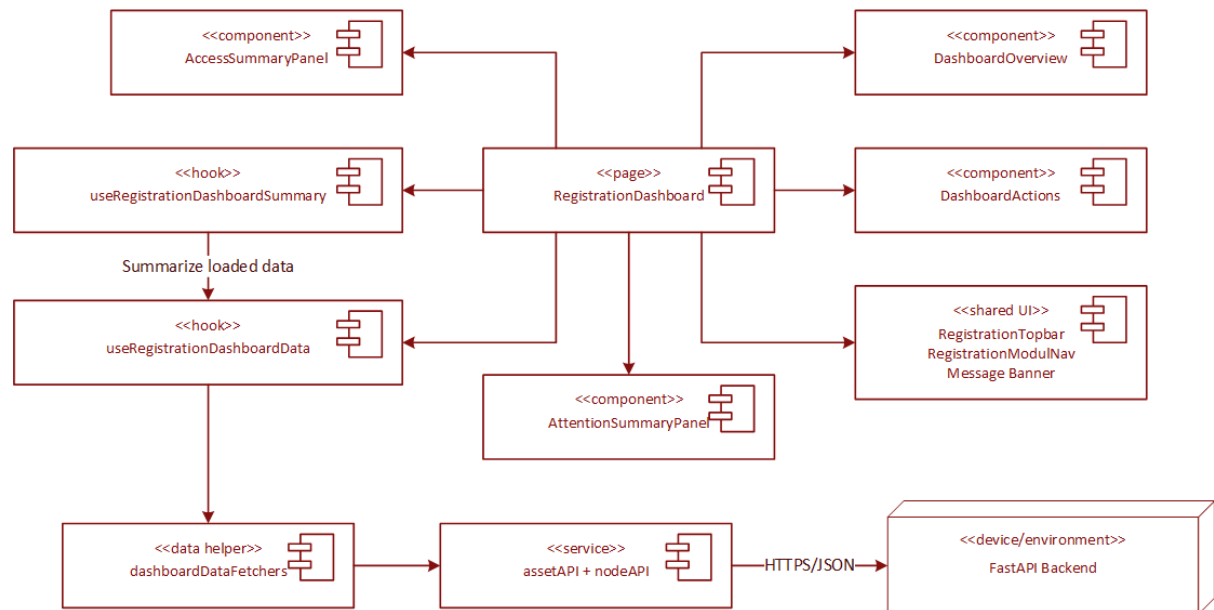


Figure 6. 3. 2. 1 Registration dashboard components.

The diagram below shows the node page, toolbar, assignment form, node table, hooks, node API, status utilities, and live updates.

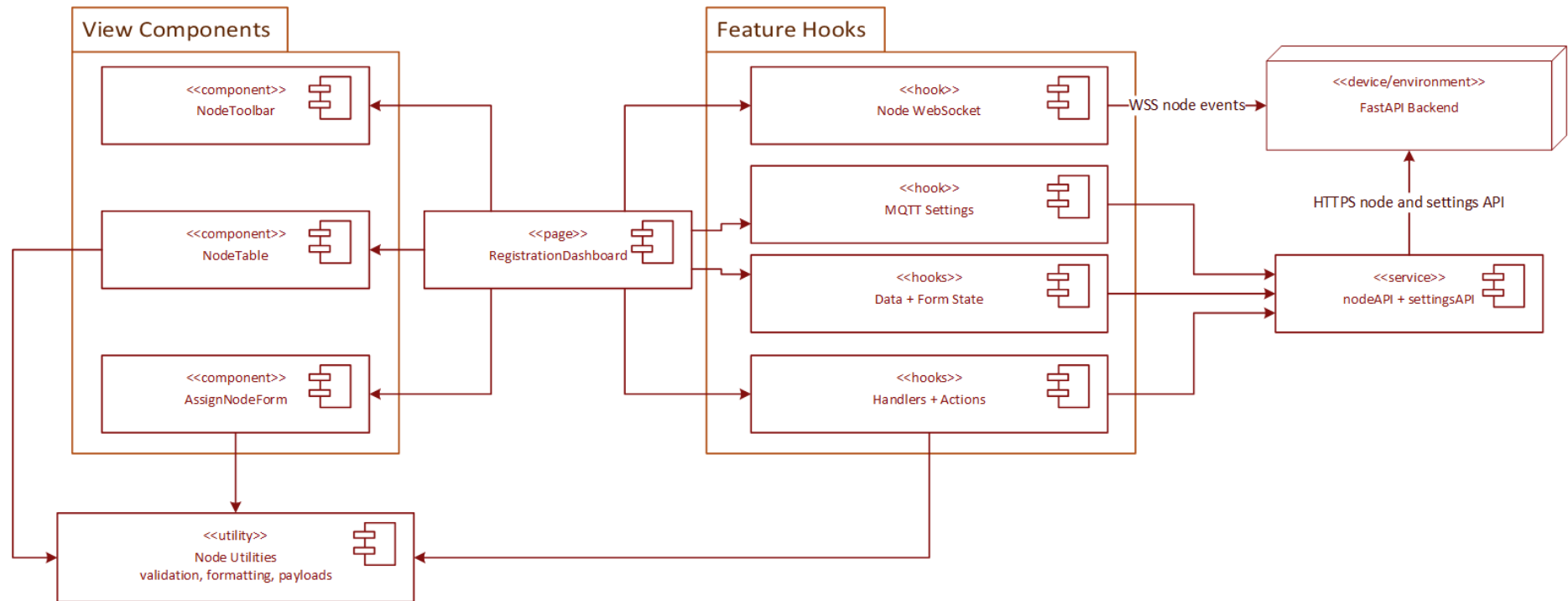


Figure 6. 3. 2. 2 Node registration components.

The diagram below shows the asset registration and deregistration forms, the asset table, the movement request panel, hooks, services, and real-time events.

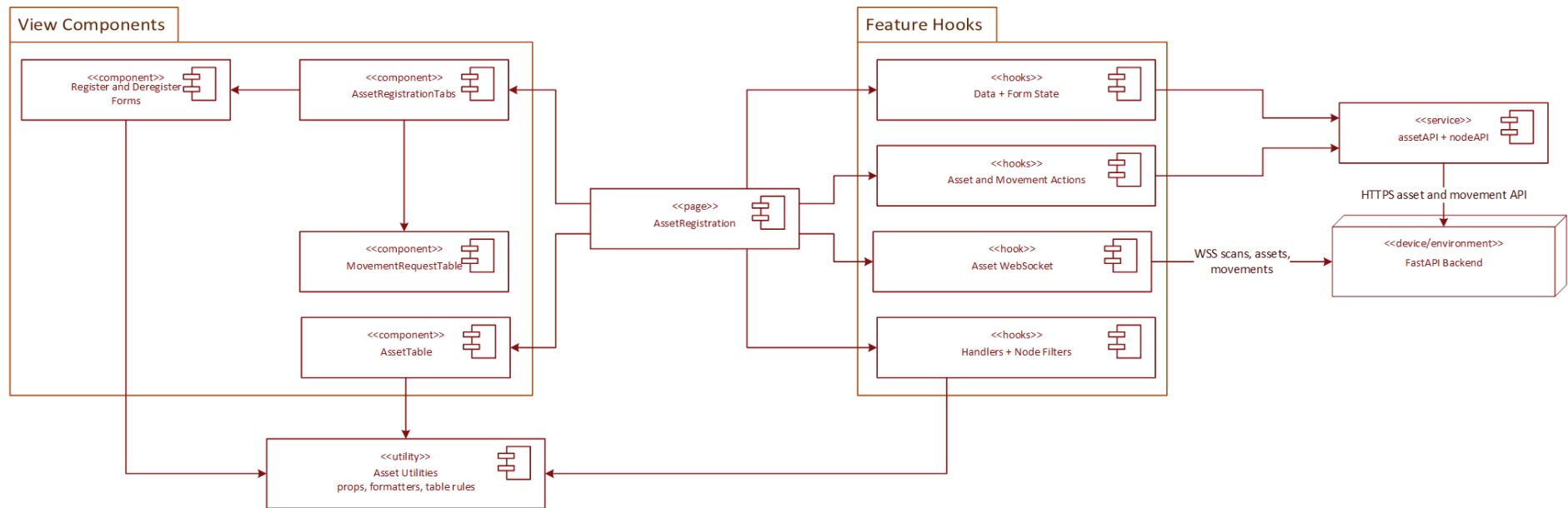


Figure 6. 3. 2. 3 Asset registration and movement components.

The diagram below shows the user management page, the account form/table components, the user API, validation, and role protection.

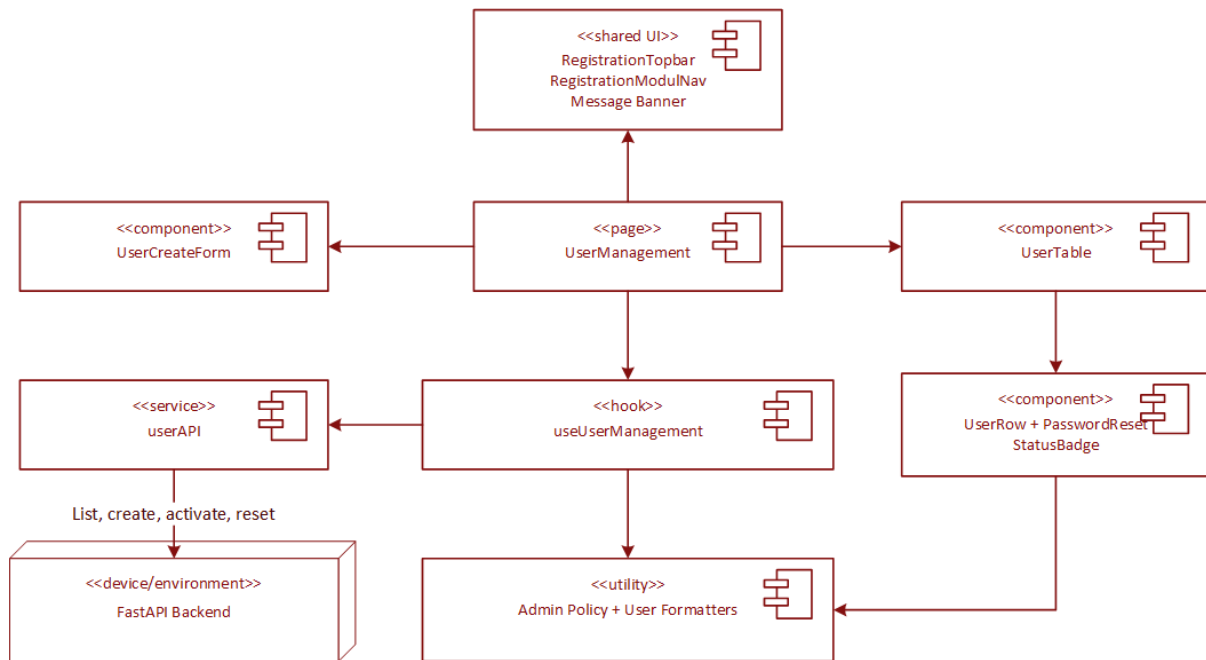


Figure 6. 3. 2. 4 User management components.

The diagram below shows the account-security and system-setting pages, along with authentication, validation, API services, and shared controls.

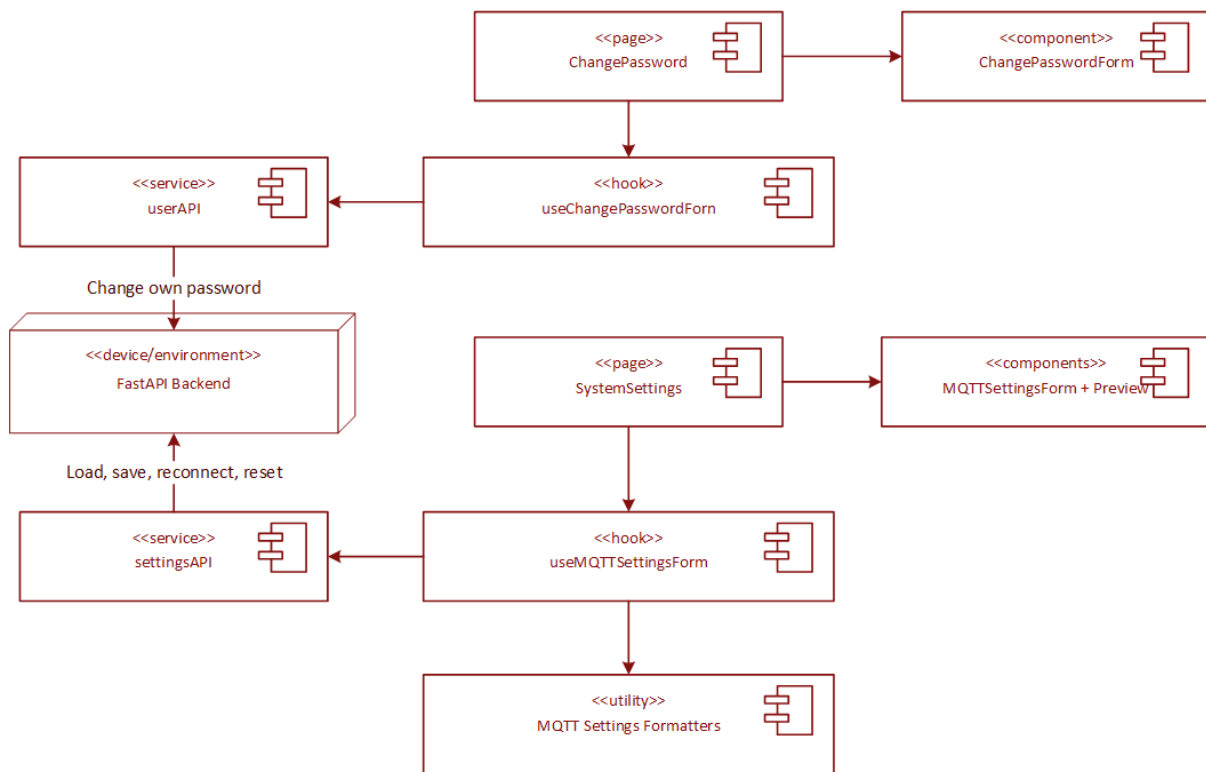


Figure 6. 3. 2. 5 Account security and settings components.

6.3.3. Registration App API and Realtime Structure

This diagram explains how registration-side features communicate with the backend. Feature-specific services call the shared API client, which attaches the current bearer token and applies common error handling. The WebSocket layer receives supported scan, asset, node, move-ment, alert, and activity events and passes them to the relevant hooks or page state. Initial and recovery data still come from HTTP APIs, so the WebSocket supplements rather than replaces the persistent backend source of truth.

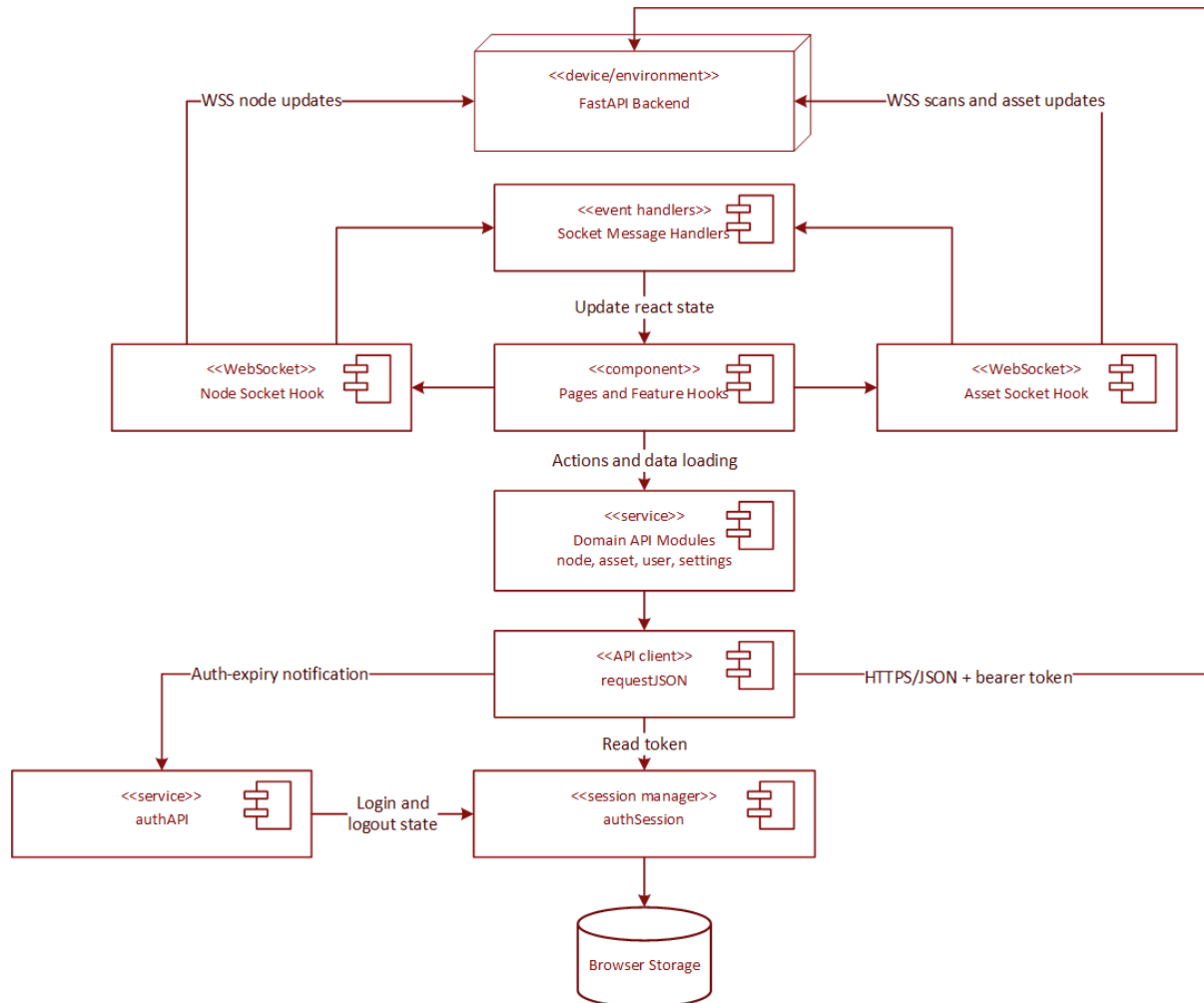


Figure 6. 3. 3. 1 Registration app API and real-time flow.

6.4. Monitor App Component Structure

The Monitor App also uses React function components, hooks, service modules, utilities, and shared components. The following diagrams document its page structure, movement workflow, operational monitoring features, and real-time data orchestration.

6.4.1. Monitor App Architecture and Authentication

The Monitor App overview shows the common shell, module navigation, monitoring pages, feature components, hooks, API services, socket service, and formatting utilities. The routing and authentication diagram explains how the login page establishes a session, how protected routes verify permitted monitoring roles, how the shell is displayed after authentication, and how logout, token expiry, or an access-denied response removes the session and returns the user to the login page.

The diagram below shows the application shell, monitoring pages, feature components, hooks, services, utilities, and backend connection.

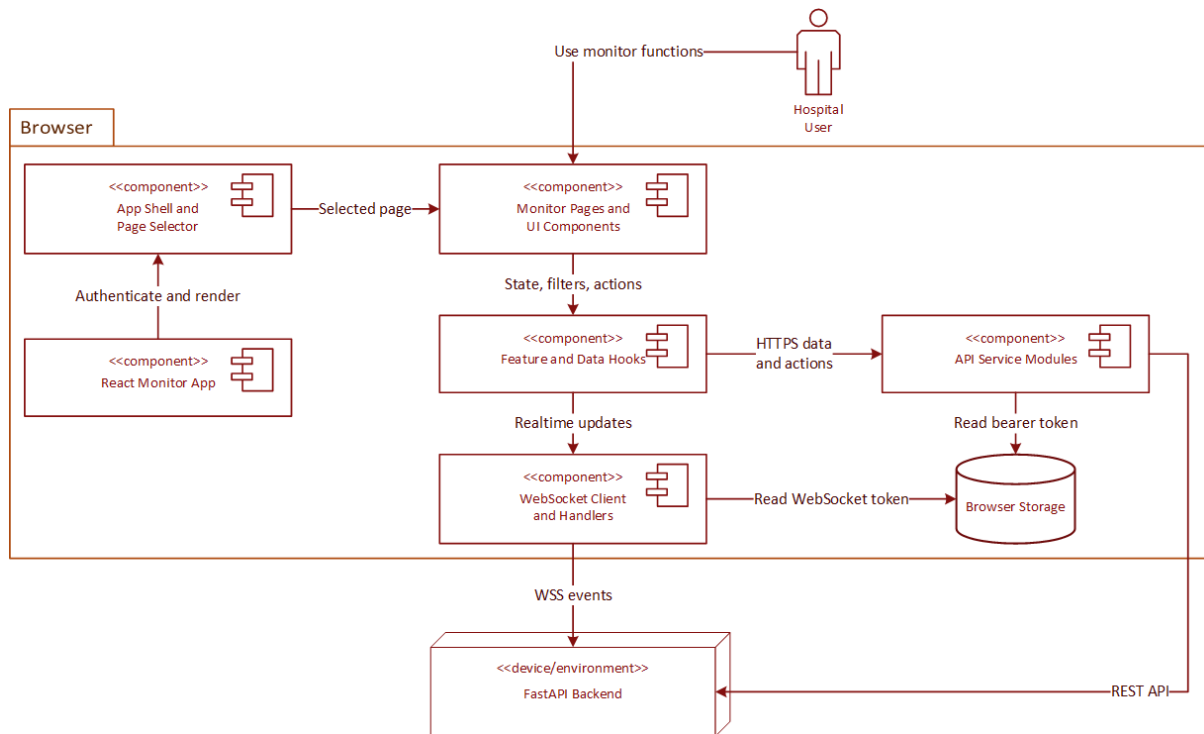


Figure 6.4.1.1 Monitor app architecture.

The diagram below shows route protection, login/session handling, shared shell components, role checks, and navigation.

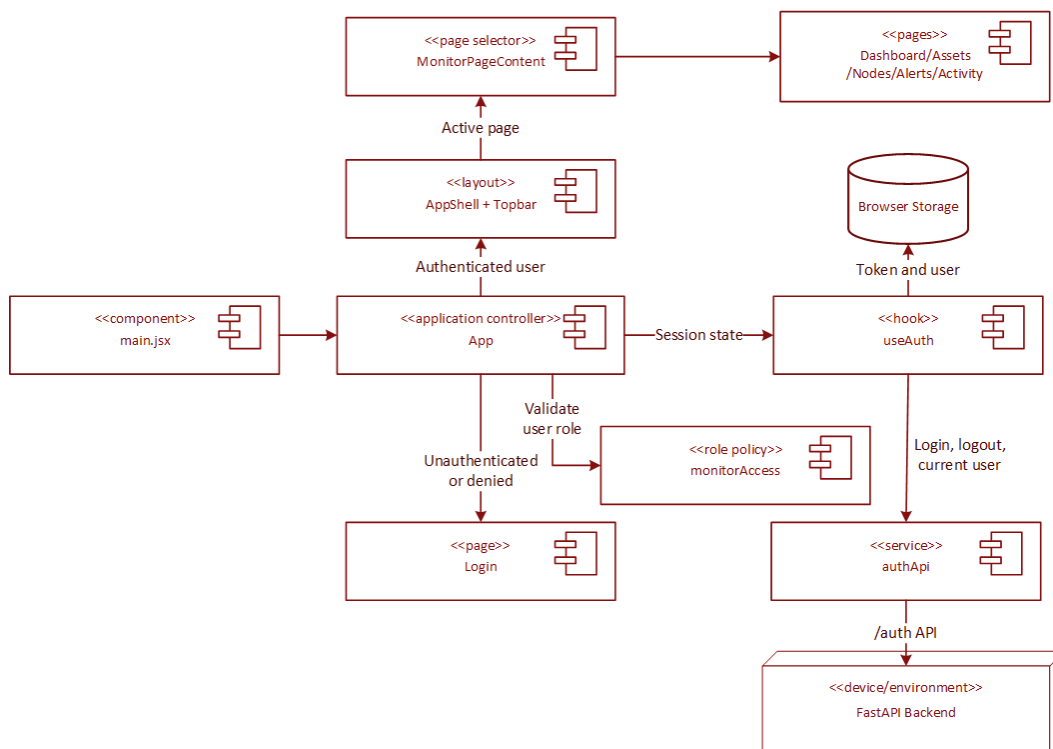


Figure 6.4.1.2 Monitor app authentication and routing.

6.4.2. Monitor App Feature Components

The feature diagrams describe the Dashboard, Asset Monitor, Movement Request modal, Node Monitor, Alerts page, and Activity page. They identify the tables, badges, search controls, sorting and pagination helpers, modal state, data hooks, and feature APIs used by each page. The diagrams also show that filtered counts and status labels are derived through shared logic, so the dashboard and detailed pages present consistent operational information.

The diagram below shows dashboard summaries, cards, hooks, filtered counts, and links to detailed monitoring pages.

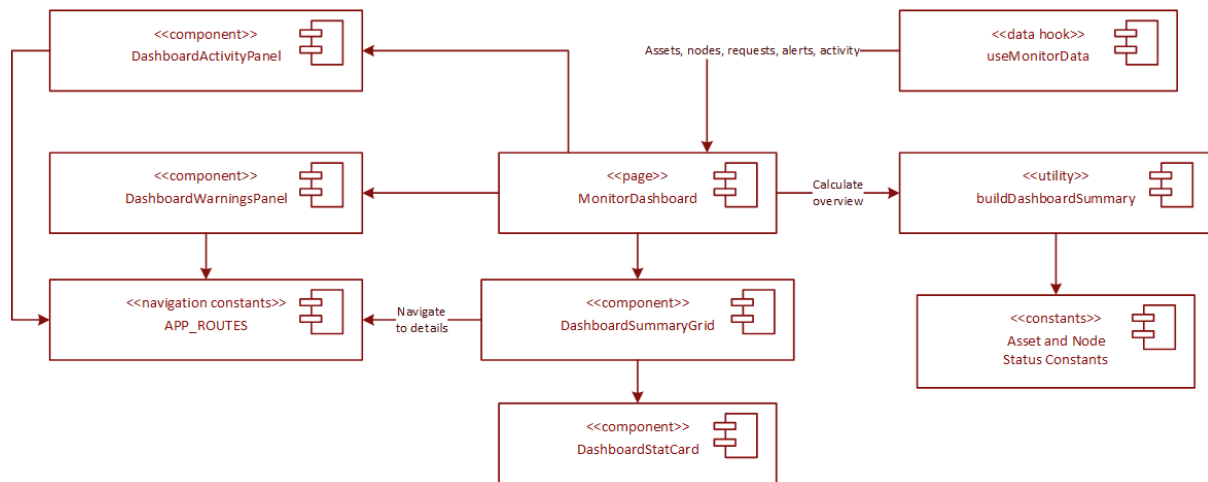


Figure 6.4.2.1 Monitor dashboard components.

The diagram below shows the asset page, table, badges, search, sorting, pagination, movement actions, hooks, and API services.

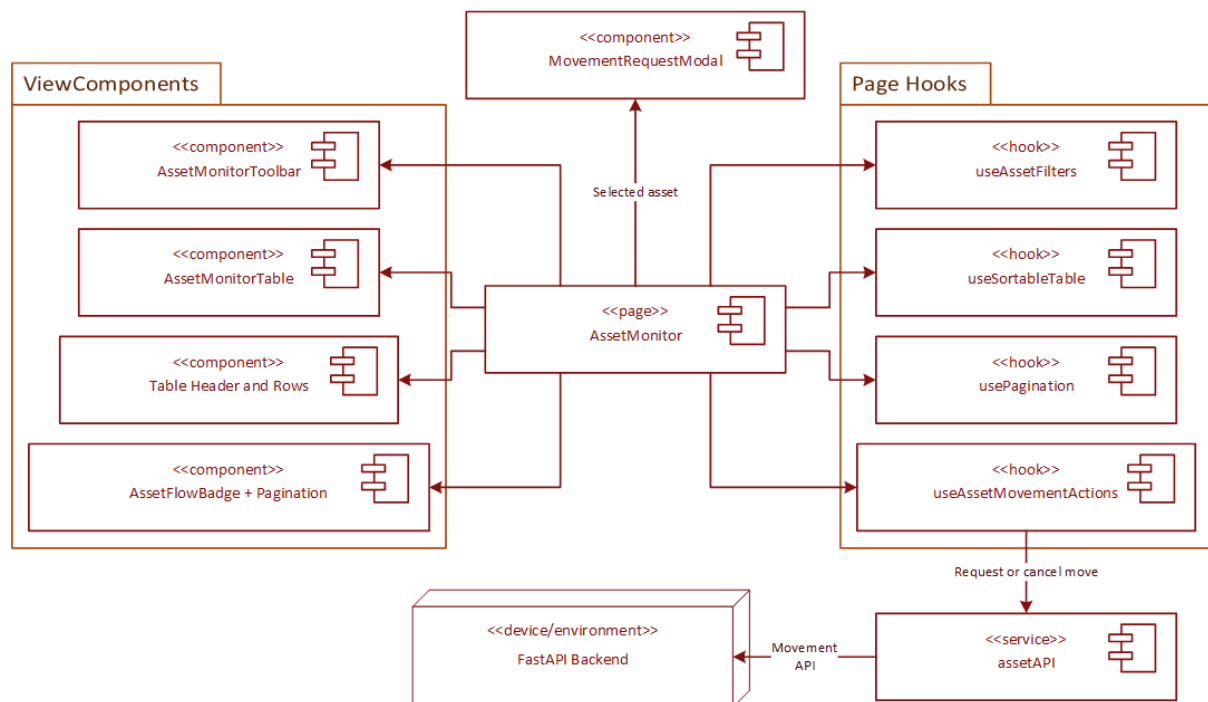


Figure 6.4.2.2 Asset monitor components.

The diagram below shows the movement modal, selection and validation state, request/cancel actions, and movement API dependencies.

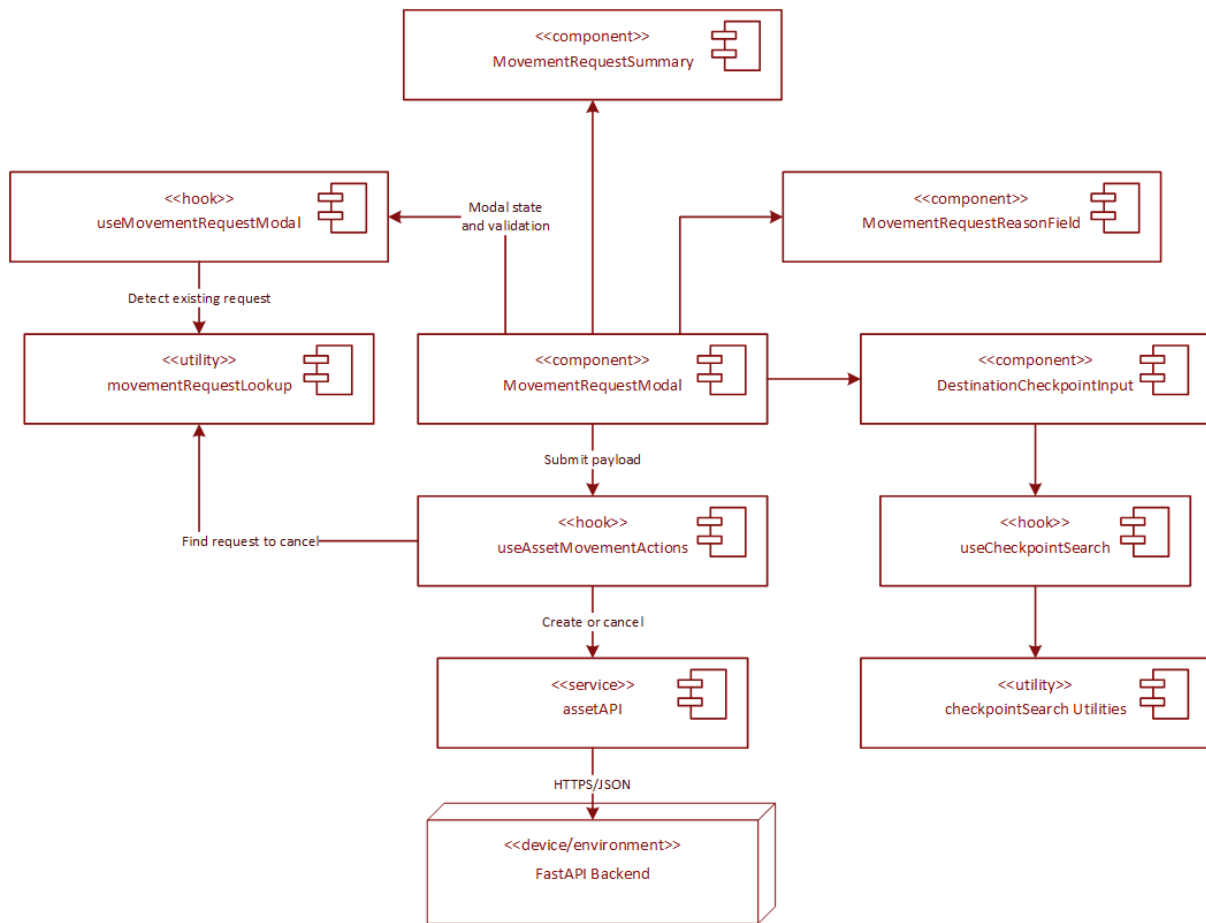


Figure 6. 4. 2. 3 Movement request modal components.

The diagram below shows the node page, table, status badge, search, sorting, pagination, and node data hook.

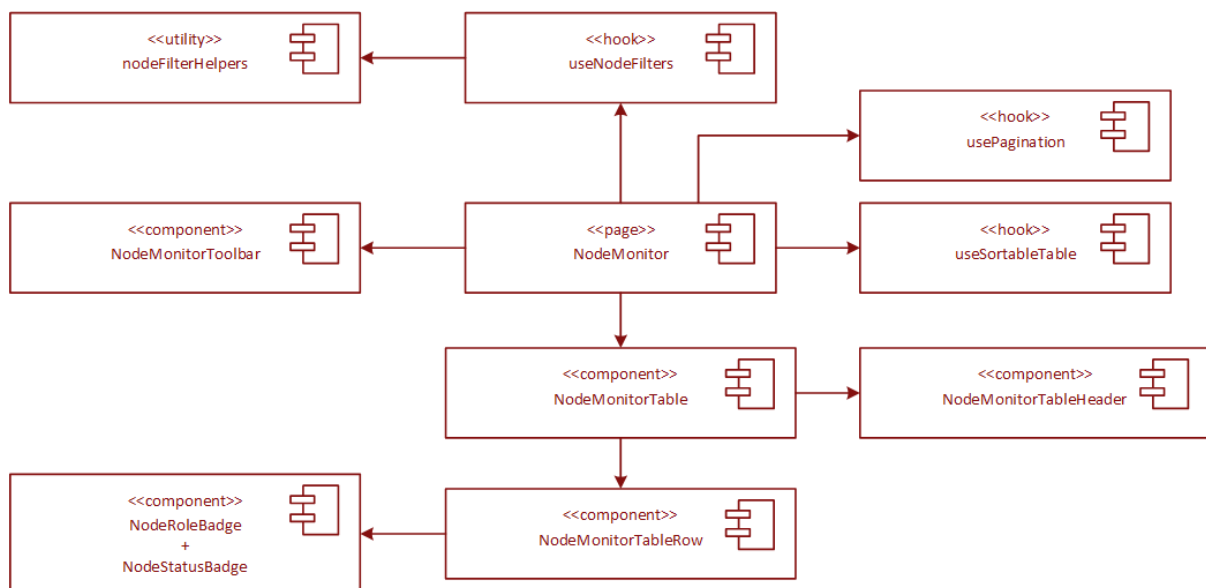


Figure 6. 4. 2. 4 Node monitor components.

The diagrams below show alert and activity pages, rows, filtering, sorting, pagination, data sources, and display utilities.

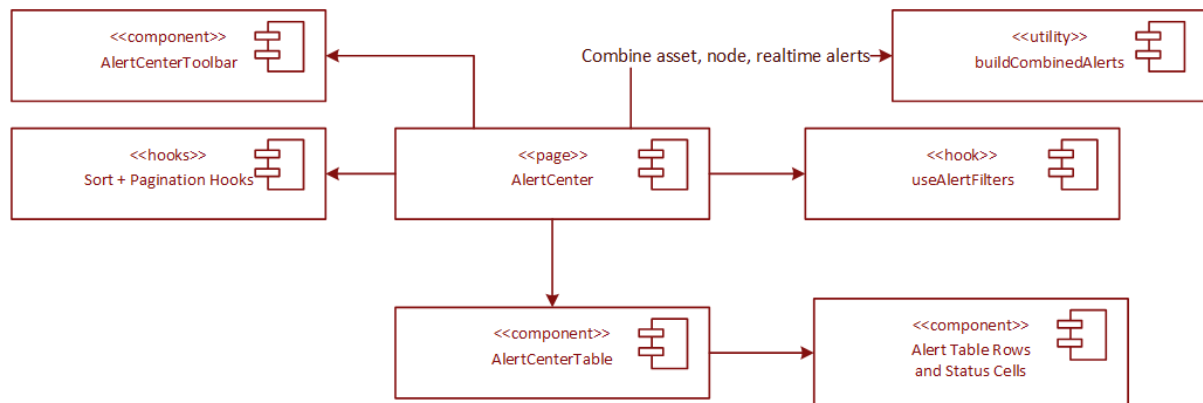


Figure 6.4.2.5 Alert page components.



Figure 6.4.2.6 Activity page components.

6.4.3. Monitor Data and Realtime Infrastructure

The data-orchestration diagram shows how the Monitor App performs initial API loading, stores current data on assets, nodes, alerts, activity, and movement, and applies incoming WebSocket events to the appropriate state collections. The infrastructure diagram shows the shared API client, authentication headers, feature-specific API modules, socket lifecycle, reconnect behavior, and backend boundaries. After a reconnection or a suspected missed event, the app can refresh the current API data to restore a consistent view.

The diagram below shows how monitor data hooks coordinate initial API loading, local state, WebSocket events, alerts, asset flow, sorting, and pagination.

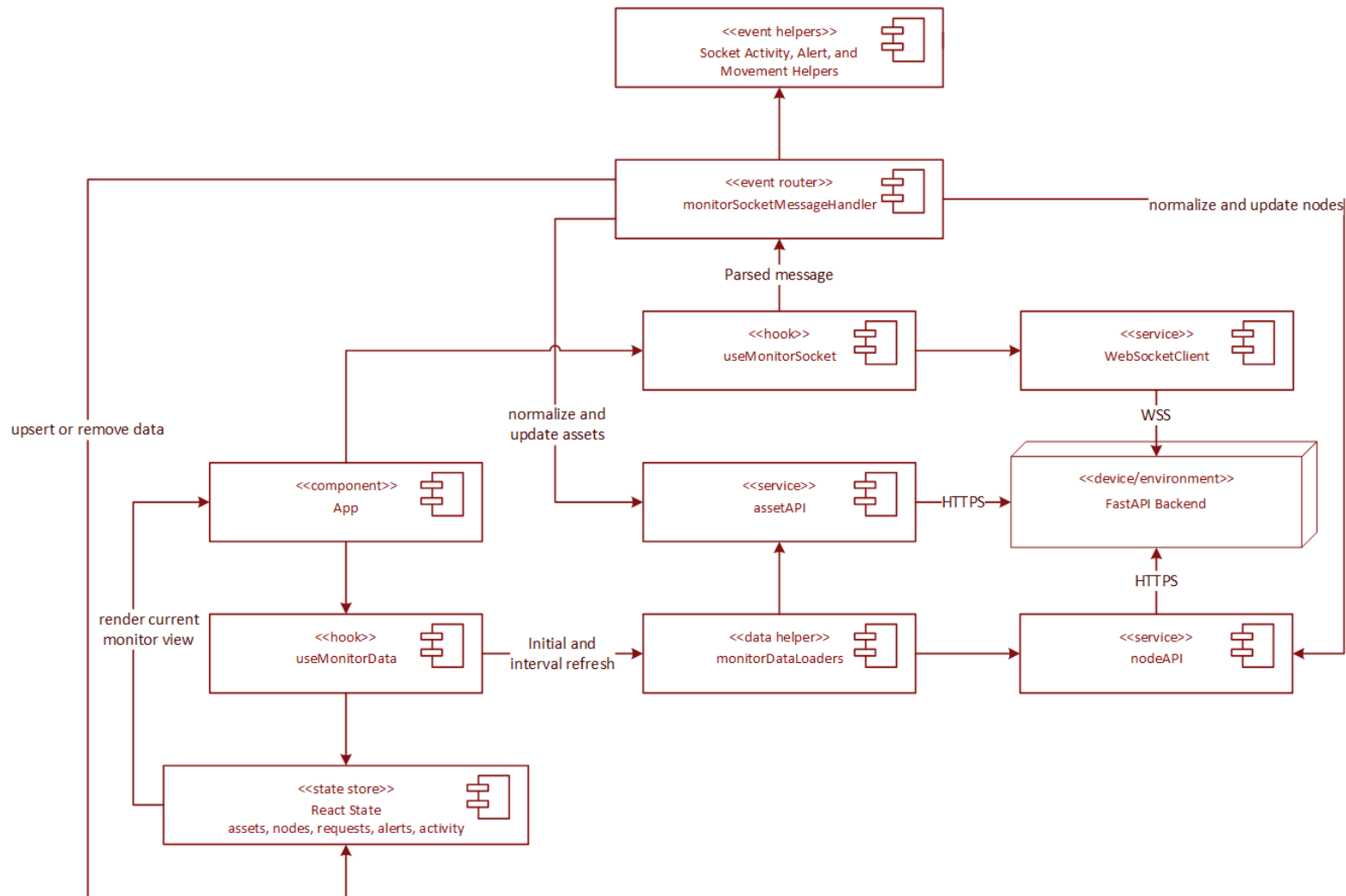


Figure 6. 4. 3. 1 Monitor data orchestration.

The diagram below shows API client configuration, authentication headers, monitor API modules, socket lifecycle, and backend boundaries.

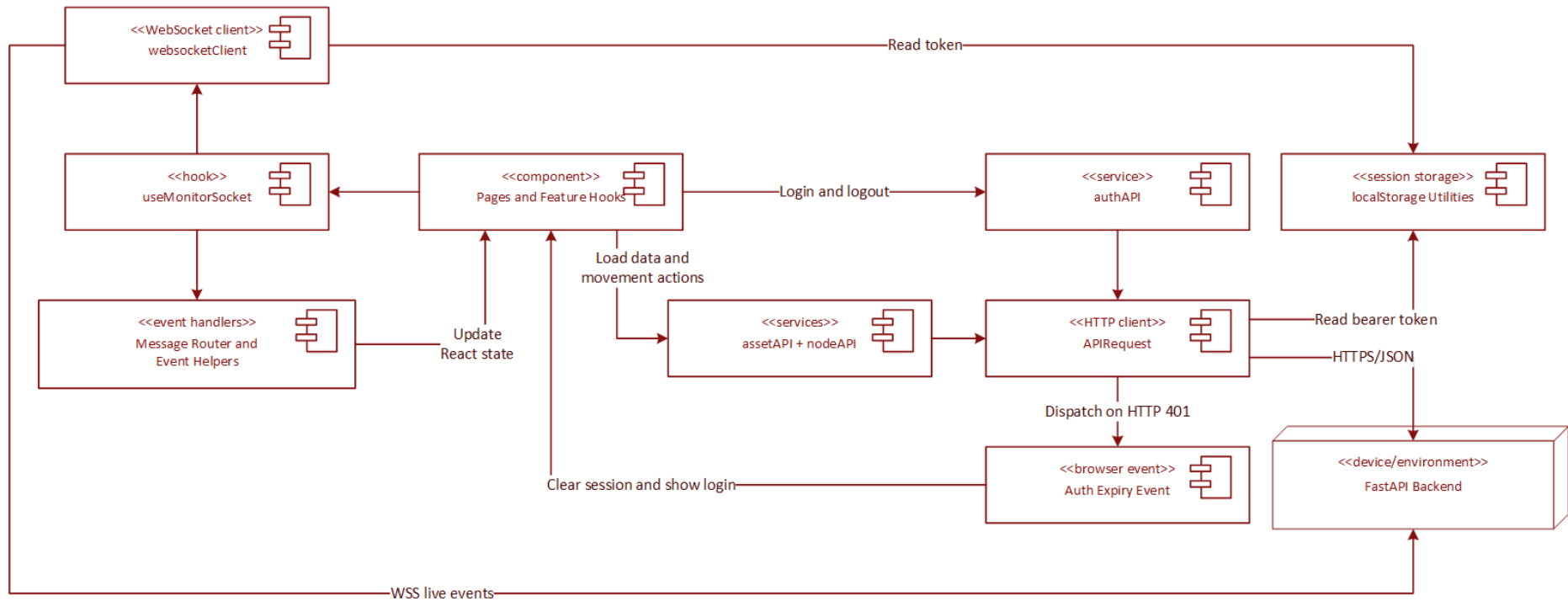


Figure 6. 4. 3. 2 Monitor API and WebSocket infrastructure.

7. Behavioral Design

This section explains the system's behavior over time using sequence and state-machine diagrams. It connects use cases to the system's internal interactions and state transitions.

7.1. Sequence Diagrams

Sequence diagrams describe how actors and system components exchange messages over time. They are useful for workflows that involve multiple components, validation steps, database updates, MQTT messages, or WebSocket broadcasts.

This section includes sequence diagrams for the most important workflows rather than every simple page-opening use case. Each sequence diagram should state its related use case and purpose.

7.1.1. User Login

The login sequence should show the user entering credentials, the frontend sending a login request, the backend verifying the user and password hash, and the frontend storing the returned token and role.

Alternative behavior should include invalid credentials, disabled accounts, and roles not allowed to access the selected application.

The diagram below shows the login request, backend verification, token response, and role-based result.

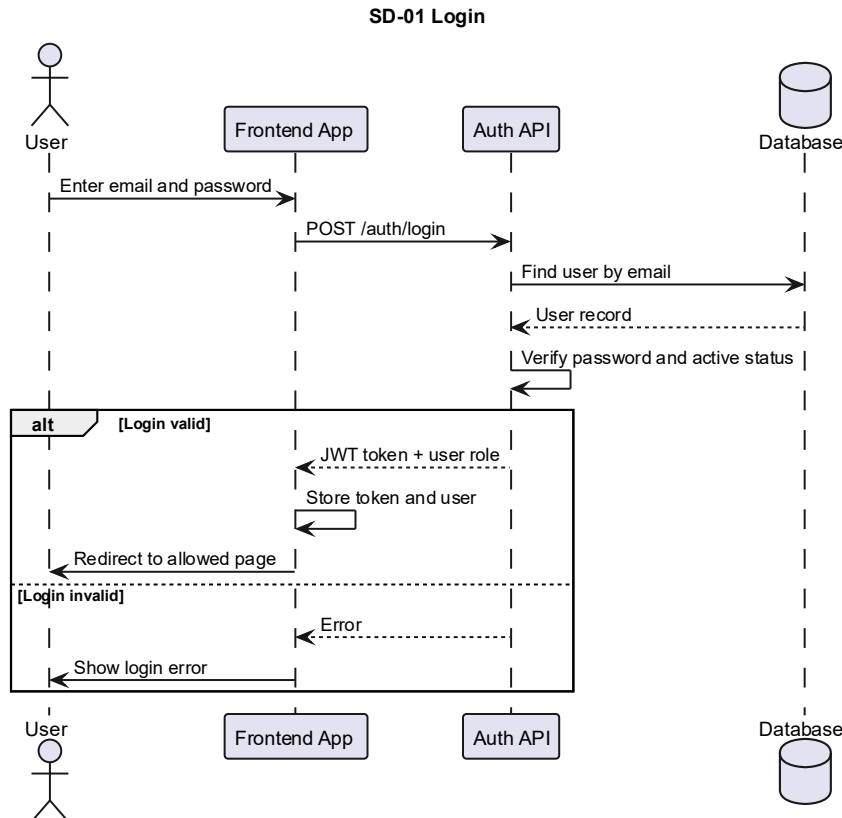


Figure 7. 1. 1 User login sequence diagram.

7.1.2. Load Registration Dashboard

This sequence shows how the Registration App restores an authenticated session and prepares the dashboard after navigation or page refresh. The route guard checks the locally stored session; the app requests dashboard summaries and any role-dependent data from protected backend endpoints; and the backend validates the token and role before querying the database. The frontend displays loading feedback while the requests are pending, then renders only the cards and actions allowed for the current role. An authentication failure clears the session, whereas an ordinary loading failure produces a retryable error message.

The diagram below shows initial data loading for the registration-side dashboard.

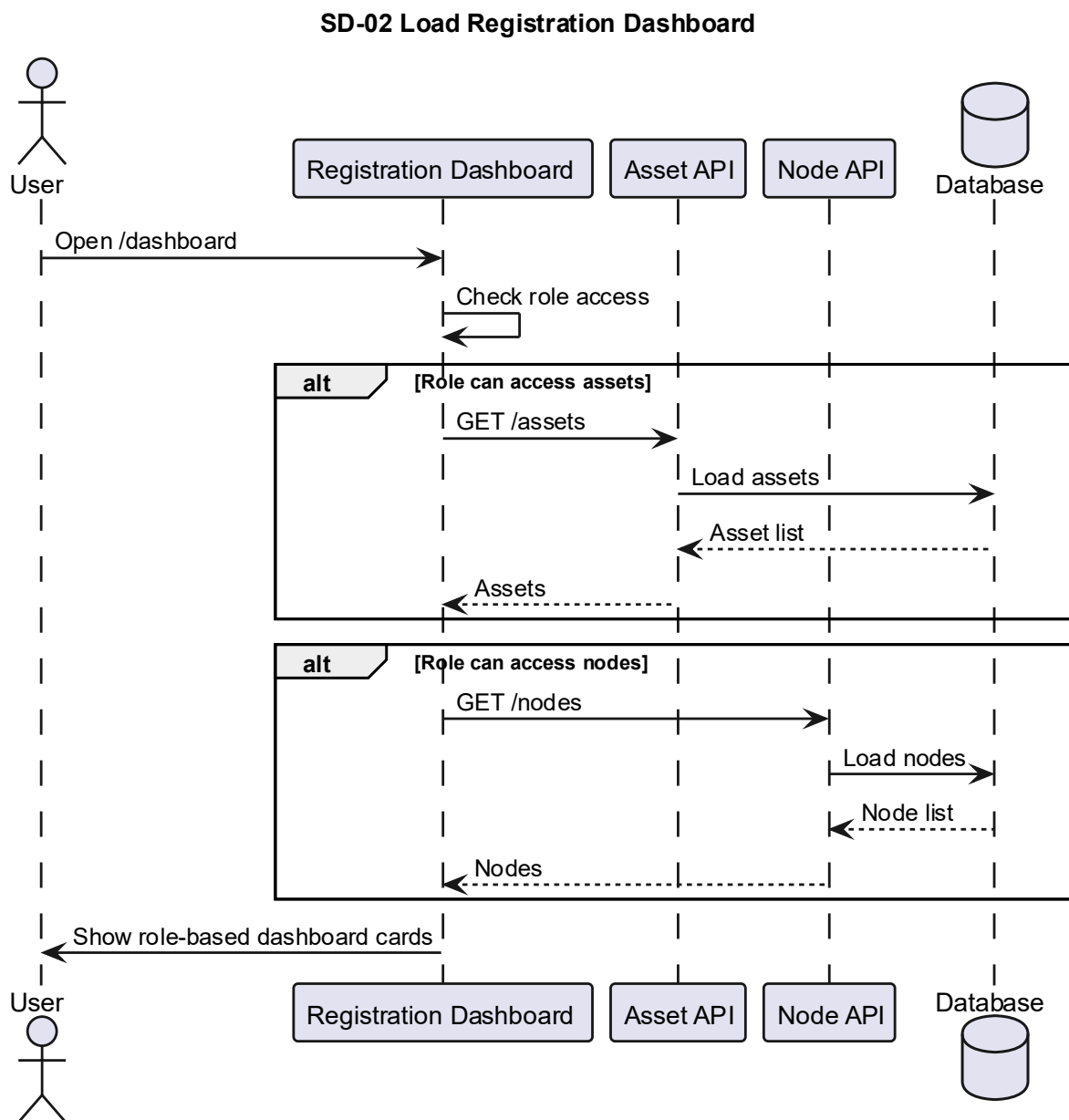


Figure 7. 1. 2 Registration dashboard loading sequence diagram.

7.1.3. Assign Node

The assign-node sequence should show an authorized user selecting a discovered node, submitting hospital, role, and room information, and the backend updating the node record.

The assign-node sequence begins when an authorized user selects a discovered node and submits an alias, role, hospital, and room, where applicable. The Node Registration page validates the visible form, then sends the assignment request to the backend. The node service verifies permission, stable node identity, allowed role and location combinations, and current assignment state before updating the node record. After the transaction succeeds, the backend returns the updated node and broadcasts a node update, so the open Registration and Monitor pages display the new assignment without manual refresh.

The diagram below shows how to assign a discovered node to a role/location.

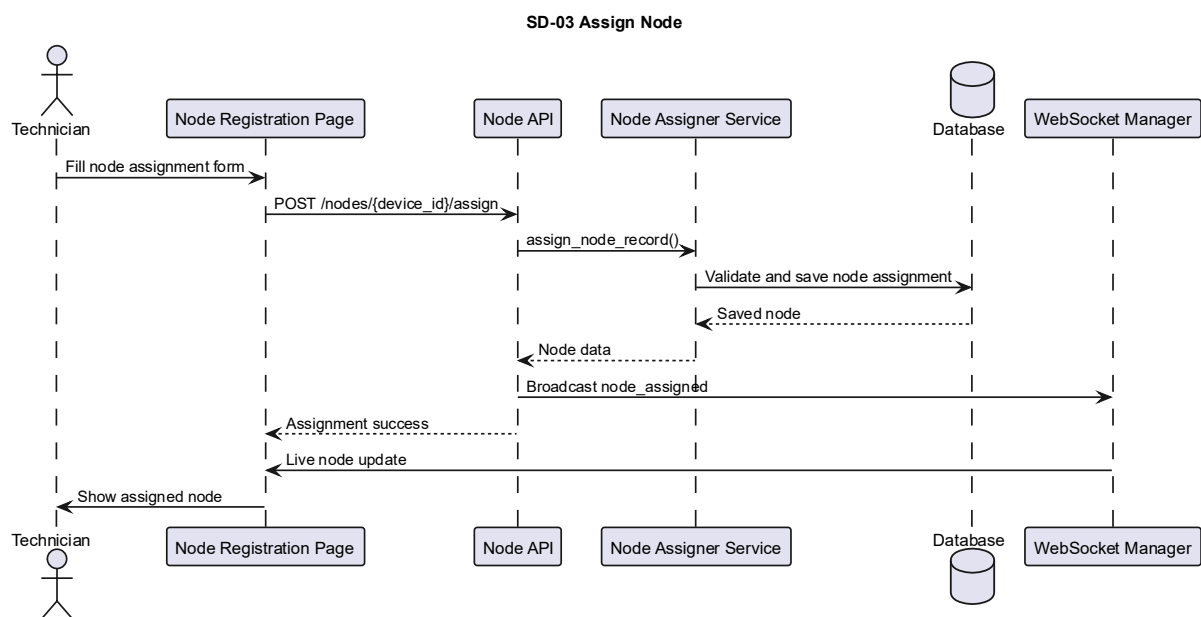


Figure 7. 1. 3 Assign node sequence diagram.

7.1.4. Identify Node Blink

This sequence explains physical node identification. An authorized user chooses Blink for an assigned, reachable node; the frontend requests the command through the backend rather than publishing directly to MQTT. The backend checks permission and node eligibility, builds the node-specific command topic and payload, and publishes the command through its MQTT client. The firmware validates the target command, temporarily drives the red and green LEDs, and publishes an acknowledgment. The backend records or broadcasts the result so the user can distinguish a successful command from an acknowledgment timeout or an offline node.

The diagram below shows backend command publishing and node acknowledgment.

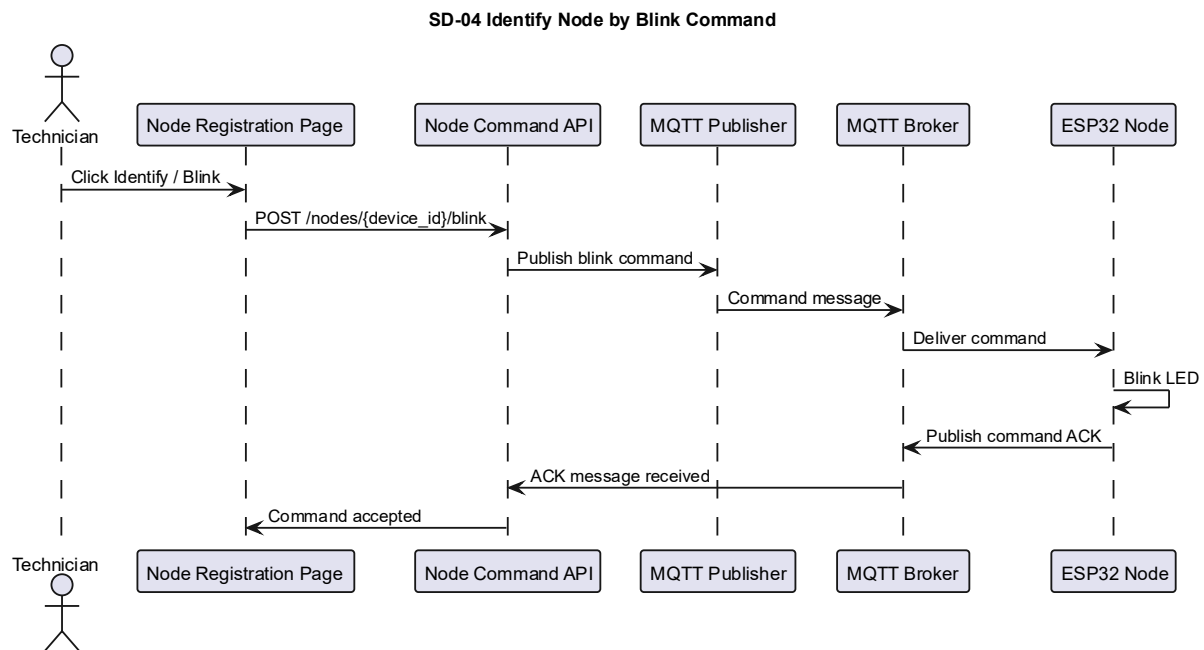


Figure 7. 1. 4 Identify node blink sequence diagram.

7.1.5. Scan Tag at Registration Desk

The registration-scan sequence starts when the UHF RFID reader detects a tag at a node configured for the Registration Desk role. Firmware normalizes the tag identifier and publishes a registration-scan message through MQTT. The backend validates the topic, payload, node identity, role, and enabled or online conditions, then makes the accepted tag ID available to the registration workflow and sends a live event to connected Registration App clients. The page displays the scanned tag for registration or deregistration; unknown, malformed, duplicate, or ineligible-node events are rejected or reported without directly changing asset records.

The diagram below shows the registration node scan via MQTT and the UI update.

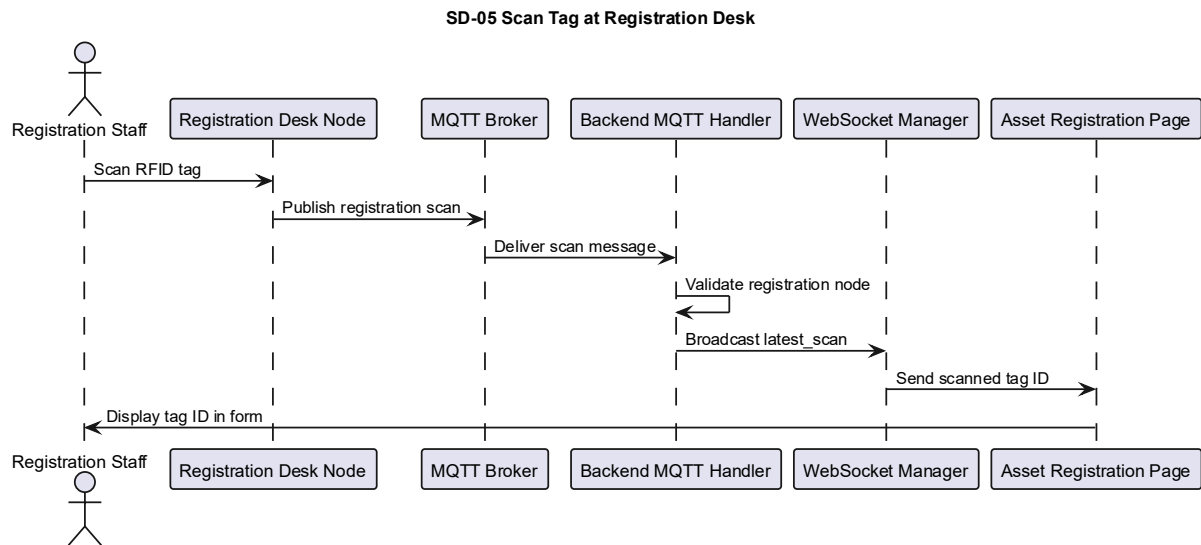


Figure 7. 1. 5 Registration desk scan sequence diagram.

7.1.6. Register Asset

After a tag has been scanned, the user enters the asset name and selects the assigned room and registration context. The frontend submits the registration request to the backend, where the asset-registration service verifies the user role, tag format, the absence of another active asset with the same tag, a valid assigned room, and an eligible registration node. The backend creates the asset, records the initial event, and sets the flow state to “on the move” until the asset is detected at its assigned checkpoint. It then returns the created record and broadcasts asset and activity updates to both applications.

The diagram below shows the user submitting asset registration after a scanned tag is available.

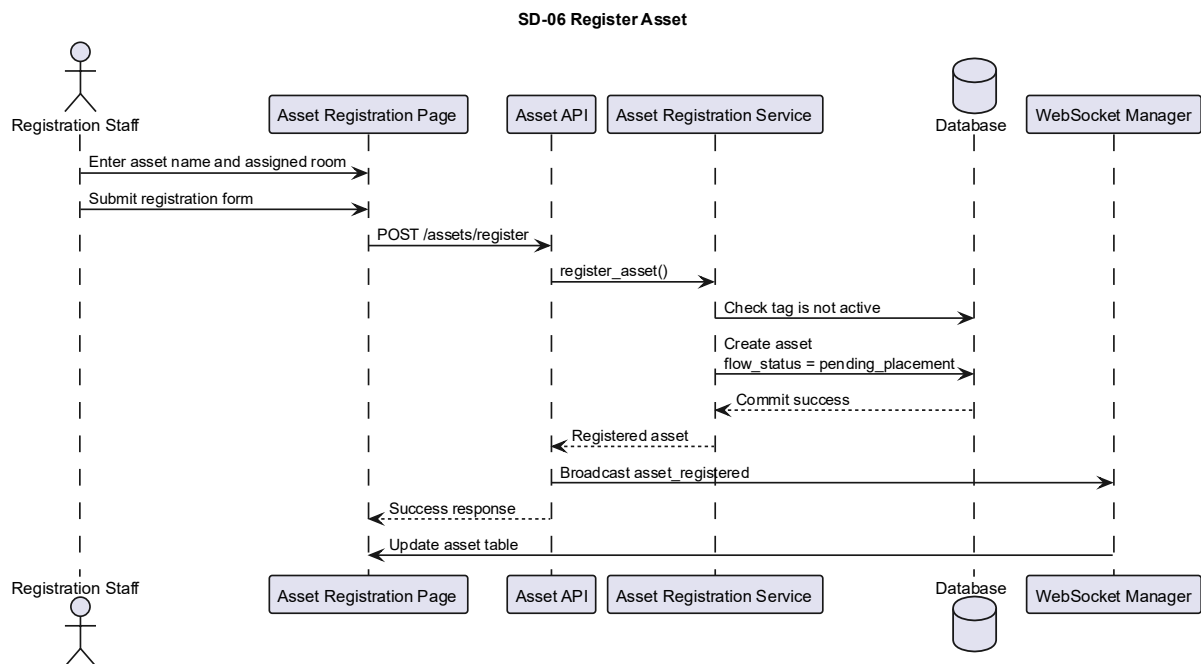


Figure 7. 1. 6 Register asset sequence diagram.

7.1.7. Request Asset Movement

The movement-request sequence begins in the Monitor App when an authorized user selects an active asset and a destination room. The modal validates the destination and submits the request to the backend. The movement service verifies that the asset is eligible, that the destination is valid, and that no other pending request exists for the same asset. A new request is stored with PENDING status, the asset flow is presented as pending approval, and a live update makes the request visible to registration-side users. A duplicate or invalid request is rejected without changing the existing workflow.

The diagram below illustrates how movement requests are created in the Monitor App.

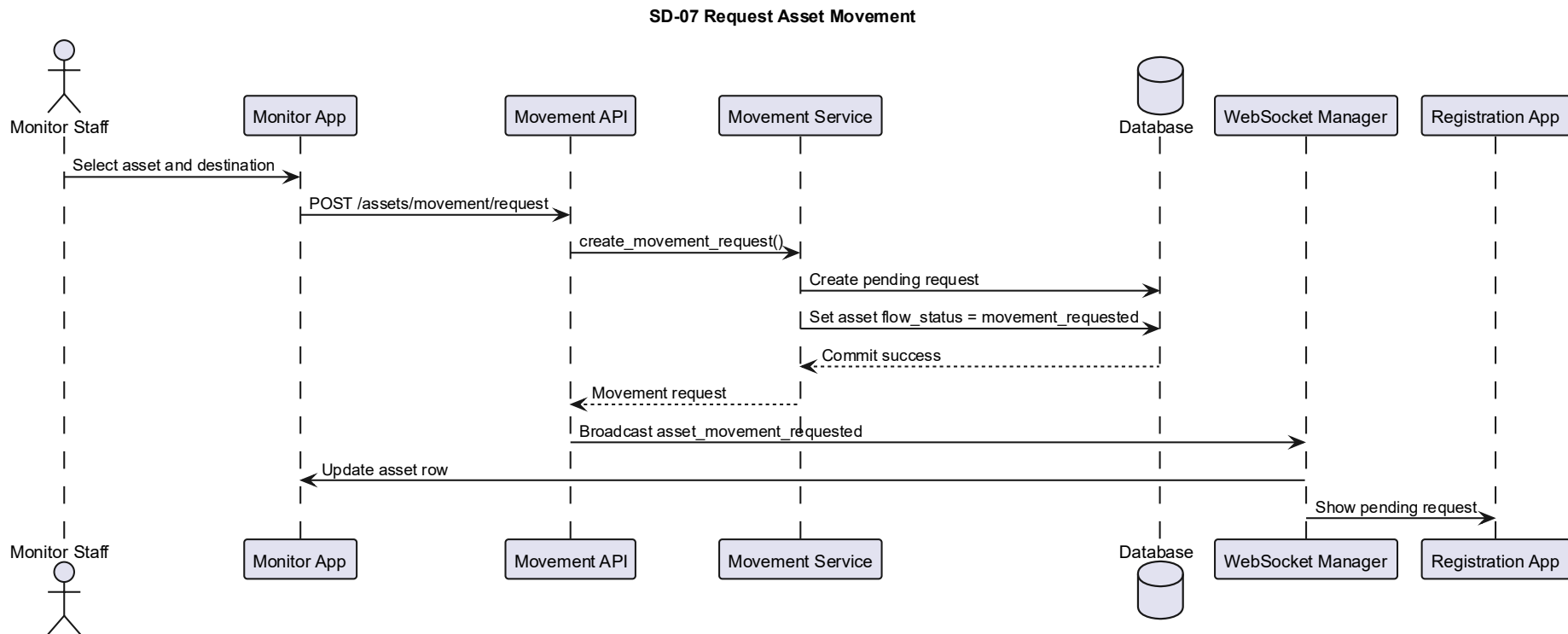


Figure 7. 1. 7 Request asset movement sequence diagram.

7.1.8. Approve Movement Request

This sequence starts when an authorized registration-side user reviews a pending movement request and chooses Approve. The backend confirms that the request is still pending, the asset remains active, the destination is valid, and the deciding user has permission. It records the decision and timestamp, changes the request to APPROVED, and places the asset in an on-the-move flow state. The updated request and asset are then broadcast to connected clients. Approval authorizes movement but does not complete it; completion requires a later detection at the approved destination. A rejection follows the same validation boundary but closes the request without authorizing movement.

The diagram below shows registration-side approval and live update.

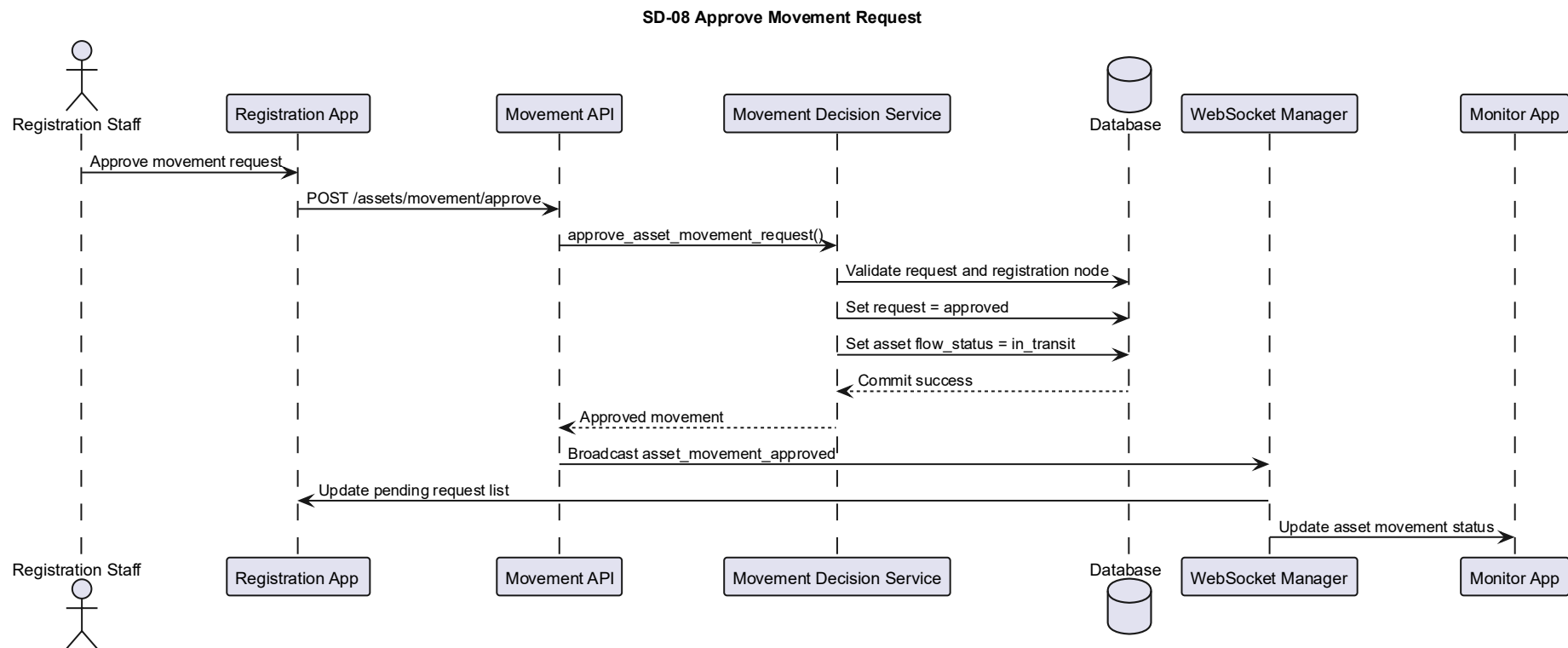


Figure 7. 1. 8 Approved movement request sequence diagram.

7.1.9. RFID Detection Updates Asset Location

A Checkpoint Node reads an RFID tag and publishes a detection that includes the tag, the node identity, and the event time. The backend MQTT router validates the message, identifies the configured node and active asset, records the detection, and updates the asset's last node, current room, and last-seen timestamp. The flow resolver compares the detected room with the assigned room and any approved destination to determine whether the asset is in place, on the move, or unauthorized. Related activity and alert records are created as required, and the backend broadcasts the resulting changes to the web applications.

The diagram below shows checkpoint detection and asset location update.

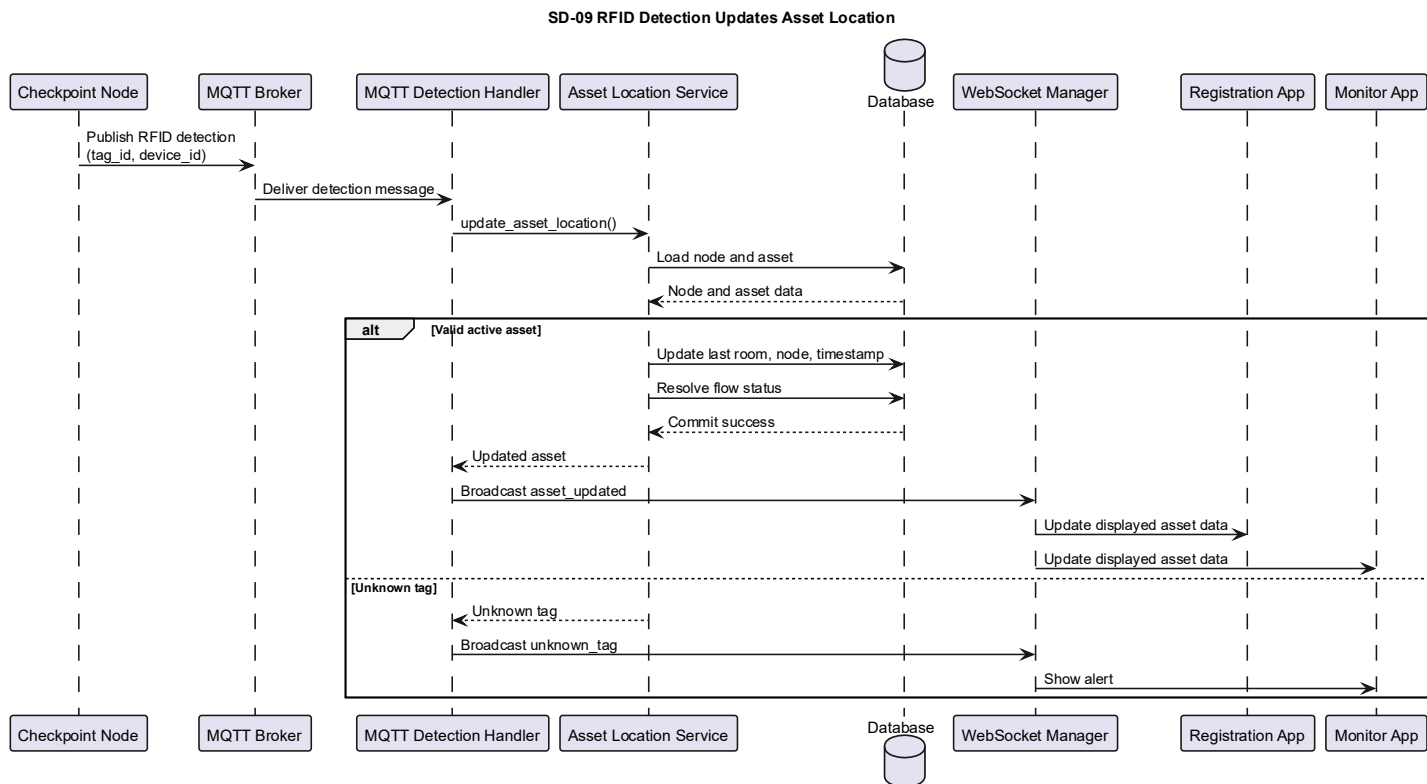


Figure 7. 1. 9 RFID detection location update sequence diagram.

7.1.10. Complete Approved Movement

This sequence is triggered by a checkpoint detection after a movement request has been approved. The backend finds the active approved request for the detected asset and verifies that the detecting node belongs to the approved destination room. It then marks the request COMPLETED, updates the asset's assigned and current room, sets the asset flow to in place, and records the completion event. Asset, movement, activity, and any resolved alert updates are persisted and broadcast. A detection in another room does not complete the request and may instead create an unauthorized-movement condition.

The diagram below shows approved movement completion at the destination node.

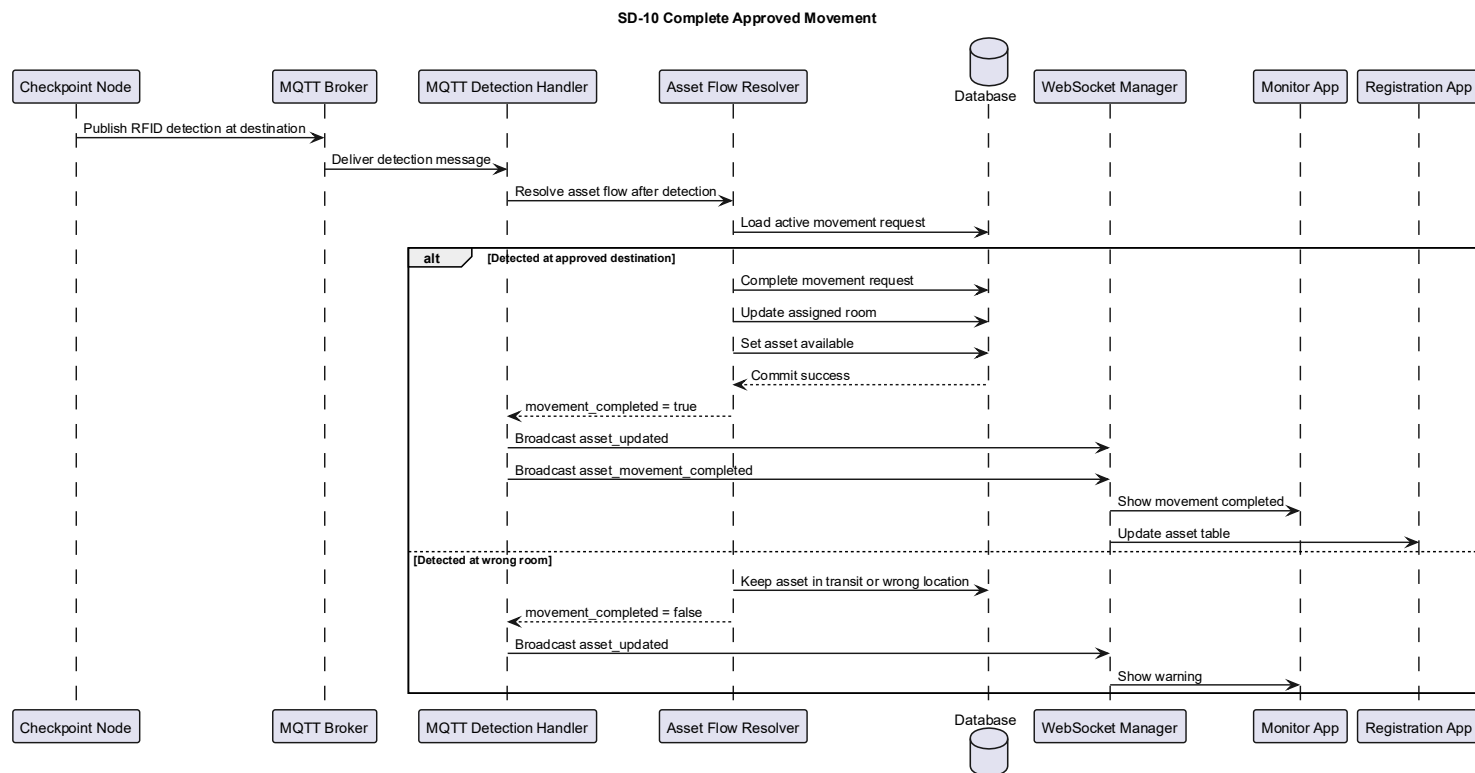


Figure 7. 1. 10 Approved movement completion sequence diagram.

7.1.11. Node Heartbeat

The heartbeat sequence runs periodically while the sensor node is operational. Firmware builds a heartbeat payload containing stable node identity and current status information, then publishes it to the configured MQTT topic. The backend validates the message and updates the node's last-ping timestamp and online status. A live node update is broadcast when the visible state changes or when the interface needs refreshed timing data. Publishing failure does not permanently block the firmware; a normal reconnect and subsequent heartbeat cycles provide recovery.

The diagram below shows heartbeat publishing and node status update.

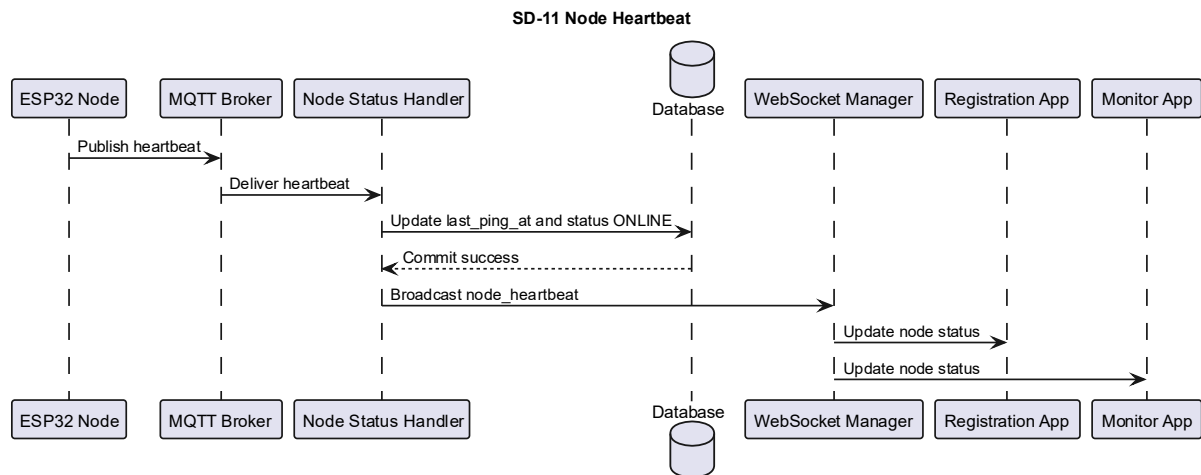


Figure 7. 1. 11 Node heartbeat sequence diagram.

7.1.12. Offline Node Detection

Offline detection is initiated by a background task rather than by a user. At a configured interval, the task compares each eligible node's last heartbeat with the offline timeout and applies any explicit skip policy used for controlled dummy nodes. When the elapsed time exceeds the threshold, the service changes the node to OFFLINE, records an activity event, creates or updates an offline-node alert, and broadcasts the new state. A subsequent valid heartbeat returns the node to ONLINE and allows the related alert to be resolved per the implemented policy.

The diagram below shows backend offline timeout detection.

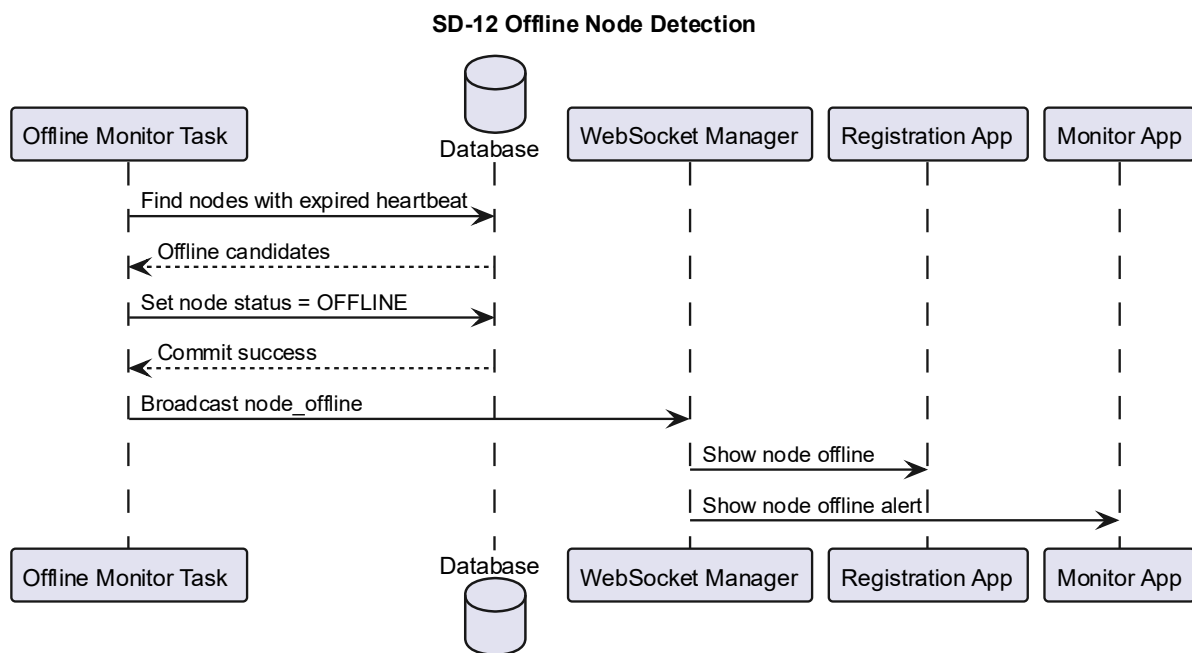


Figure 7. 1. 12 Offline node detection sequence diagram.

7.2. Firmware State Machine Diagrams

State-machine diagrams describe how important objects move between states. They are useful for node status, asset lifecycle, asset flow status, movement request status, user account status, WebSocket connection state, and MQTT connection state.

State-machine diagrams define the allowed states, transition events, guards, and terminal outcomes for behavior that cannot be explained reliably by a static structure diagram. Firmware state machines model internal ESP32 runtime control. System-level state machines model backend-managed node and asset records, movement requests, authenticated sessions, browser live-update connections, and alerts. Keeping these state owners separate prevents firmware connection states from being confused with the operational states displayed to users.

Each state machine should define valid states, events that trigger transitions, and invalid transitions that the system must reject.

7.2.1. Firmware State Machines

The firmware state machines describe non-blocking control behavior inside an ESP32 node. Together, they explain how the node starts, obtains configuration, establishes Wi-Fi and MQTT connectivity, reads RFID tags, publishes heartbeat status, executes a remote blink command, refreshes configuration, handles a physical reset request, and presents LED status. Each subsection identifies the event that initiates the behavior, the states it passes through, and the recovery path used when an operation fails.

7.2.1.1. *Main Firmware State Machine*

The Main Firmware Lifecycle diagram provides the top-level view of node execution. After boot, the firmware initializes board services and loads the stored bootstrap configuration. Missing or invalid configuration directs the node to setup mode; a valid configuration allows Wi-Fi connection, provisioning, or remote configuration retrieval, MQTT startup, and normal RFID and heartbeat operation. Temporary network or broker failure enters a recovery path rather than restarting the full device, while a confirmed reset request clears configuration and returns the node to setup.

The diagram below shows the node startup and main firmware lifecycle.

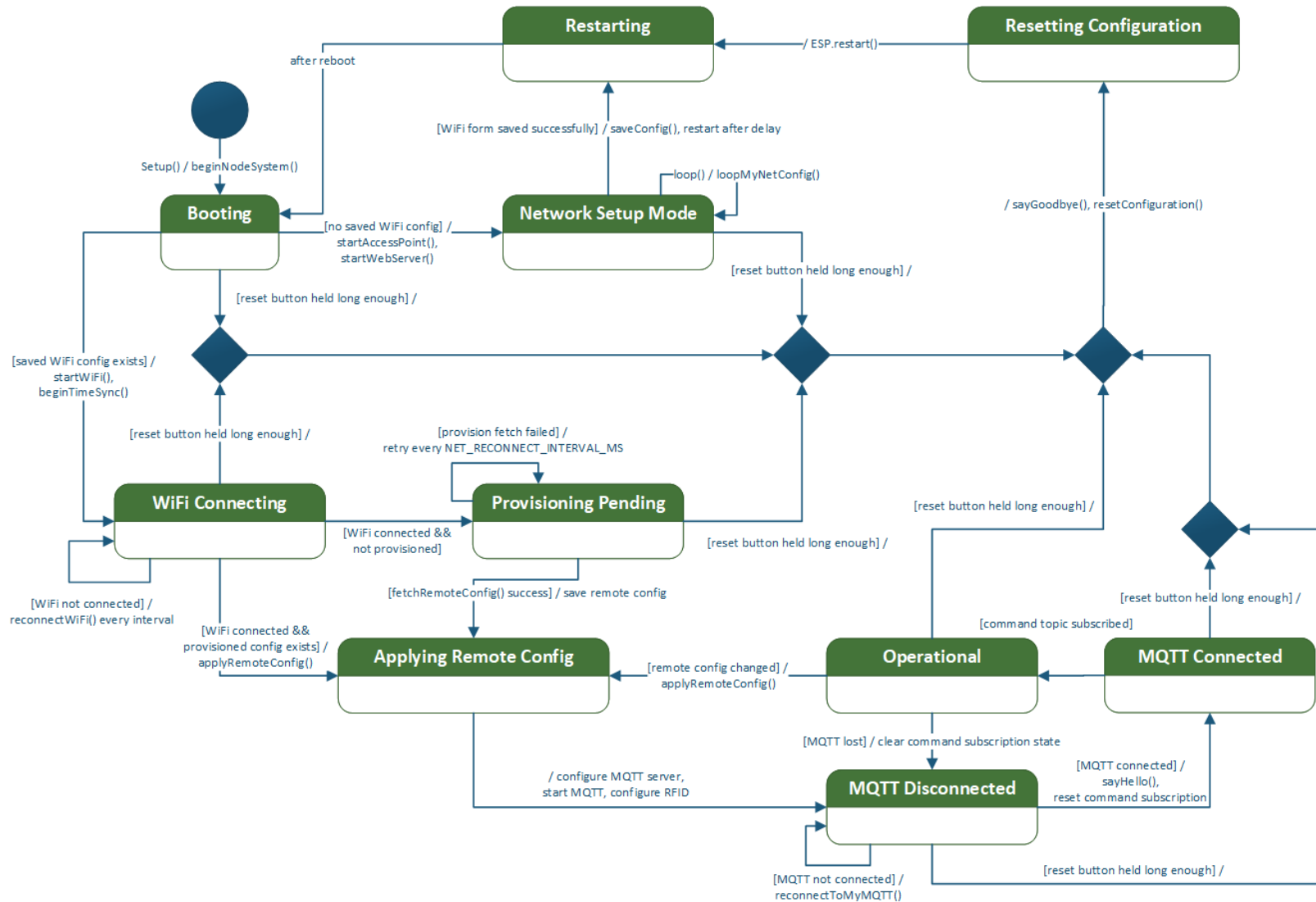


Figure 7. 2. 1. 1 Main firmware lifecycle state machine.

7.2.1.2. Captive Portal State Machine

The Captive Portal state machine describes local setup when the node cannot use a valid stored network configuration. The node starts an access point and configuration web server, waits for a technician to submit Wi-Fi and node values, validates the required fields, and saves an accepted configuration. Invalid input keeps the portal active with an error response. After a successful save, the portal closes and the device restarts or continues into the normal connection flow using the new values.

The diagram below shows the configuration portal behavior when setup is needed.

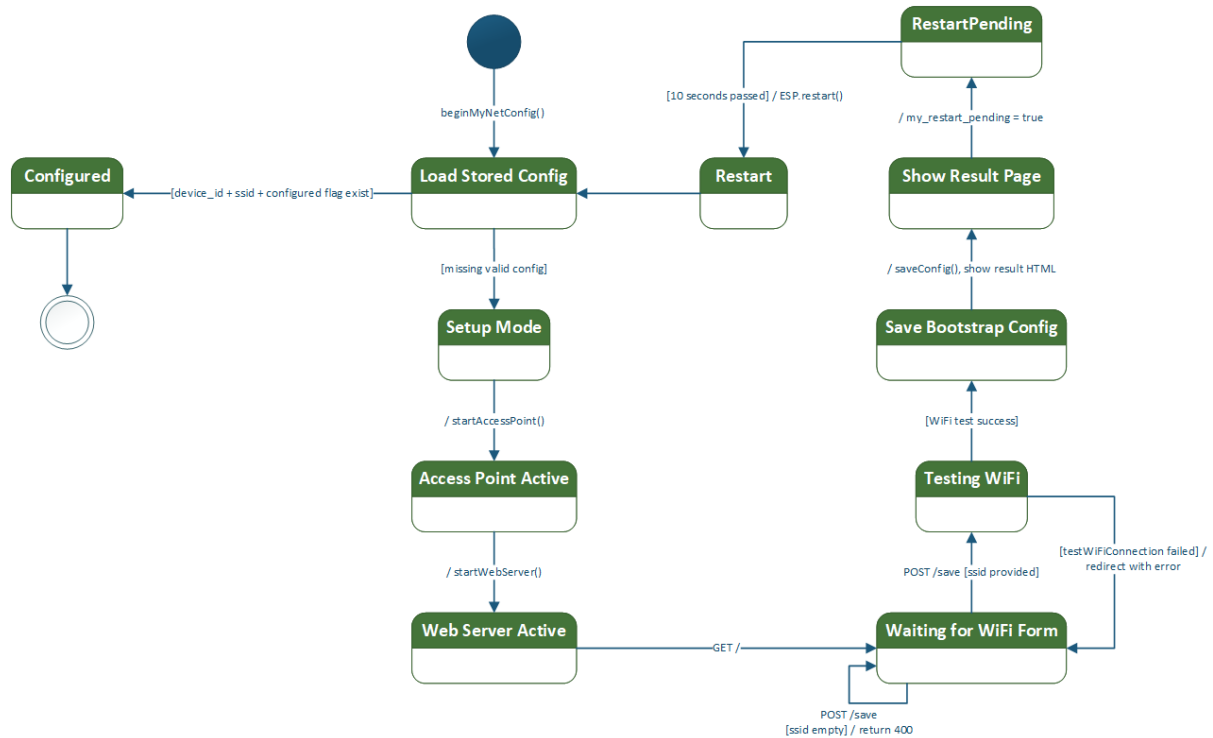


Figure 7. 2. 1. 2 Captive portal state machine.

7.2.1.3. Provisioning State Machine

The Provisioning state machine describes how a newly configured or unassigned node obtains the backend-controlled information needed for normal operation. Once network access is available, the firmware submits its stable identity and bootstrap data, waits for a valid response, validates the received role, hospital, room, and topic configuration, and stores the accepted remote configuration. A rejected or unavailable response leaves the node unprovisioned and schedules a later retry; successful provisioning enables normal MQTT subscriptions and event publishing.

The diagram below shows how node provisioning/configuration is handled.

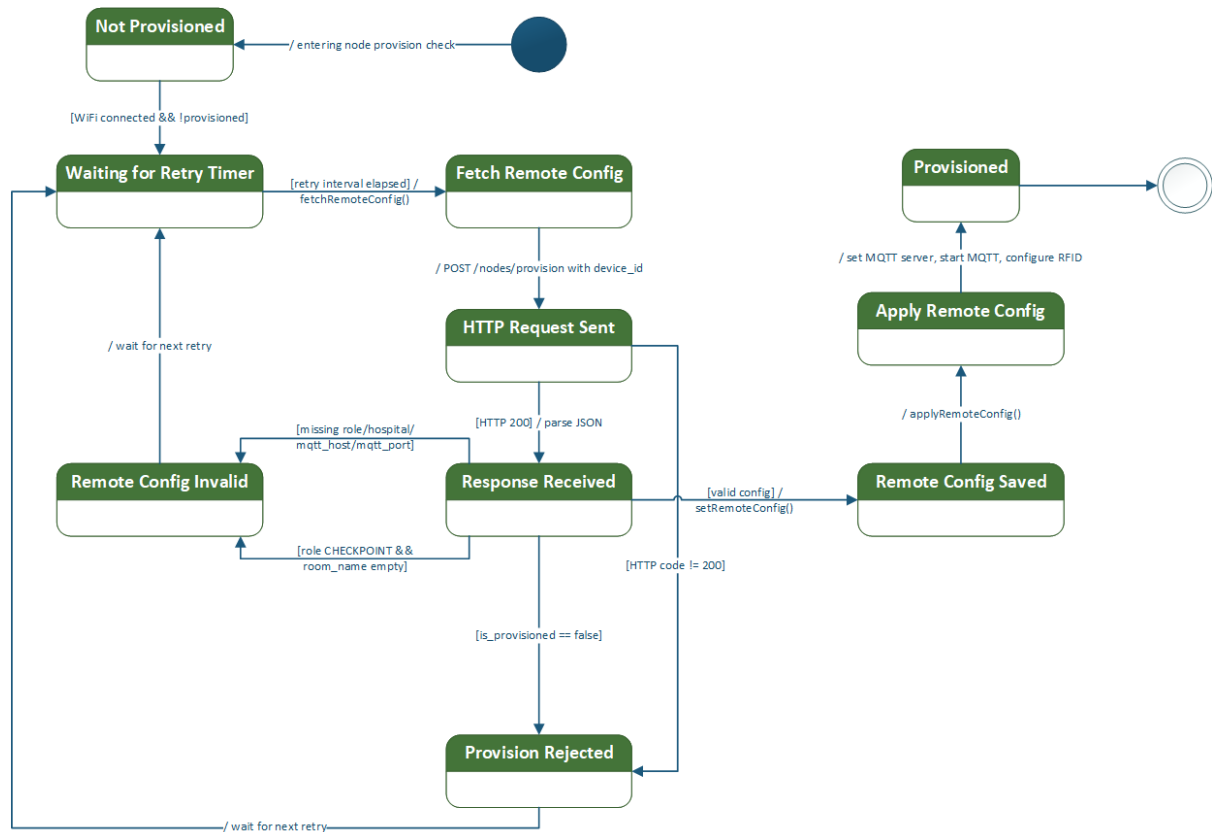


Figure 7. 2. 1. 3 Provisioning state machine.

7.2.1.4. MQTT Connection State Machine

The MQTT Connection state machine begins when Wi-Fi is available and the broker configuration is valid. The client attempts a broker connection, subscribes to the node command topics after success, and enters the connected state used for publishing scans, detections, heartbeats, and acknowledgments. Loss of Wi-Fi or broker connectivity returns the client to a disconnected or waiting state with controlled retry timing. Every successful reconnection repeats the subscription setup, so remote commands continue to work after an interruption.

The diagram below shows the MQTT connection, reconnect, and subscription states.

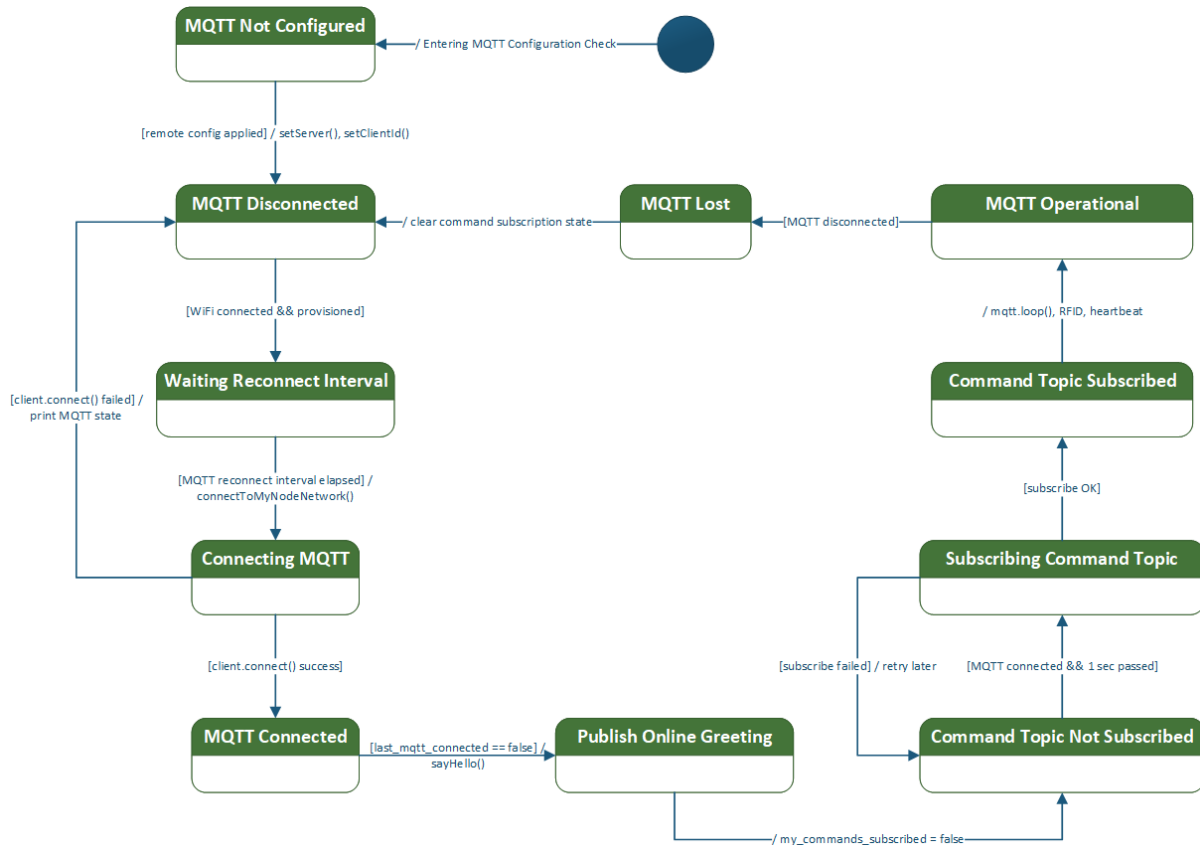


Figure 7. 2. 1. 4 MQTT connection state machine.

7.2.1.5. MQTT Blink State Machine

The MQTT Blink Command state machine begins when the subscribed command handler receives a message. Firmware validates the topic, target node, command name, and payload before changing the LED controller from its normal status pattern to the temporary identification pattern. When the blink duration finishes, the previous status indication is restored, and an acknowledgment is published. Invalid or unsupported commands do not activate the LEDs and are either ignored or acknowledged as failed per the message contract.

The diagram below shows how a blink command is received, executed, and acknowledged.

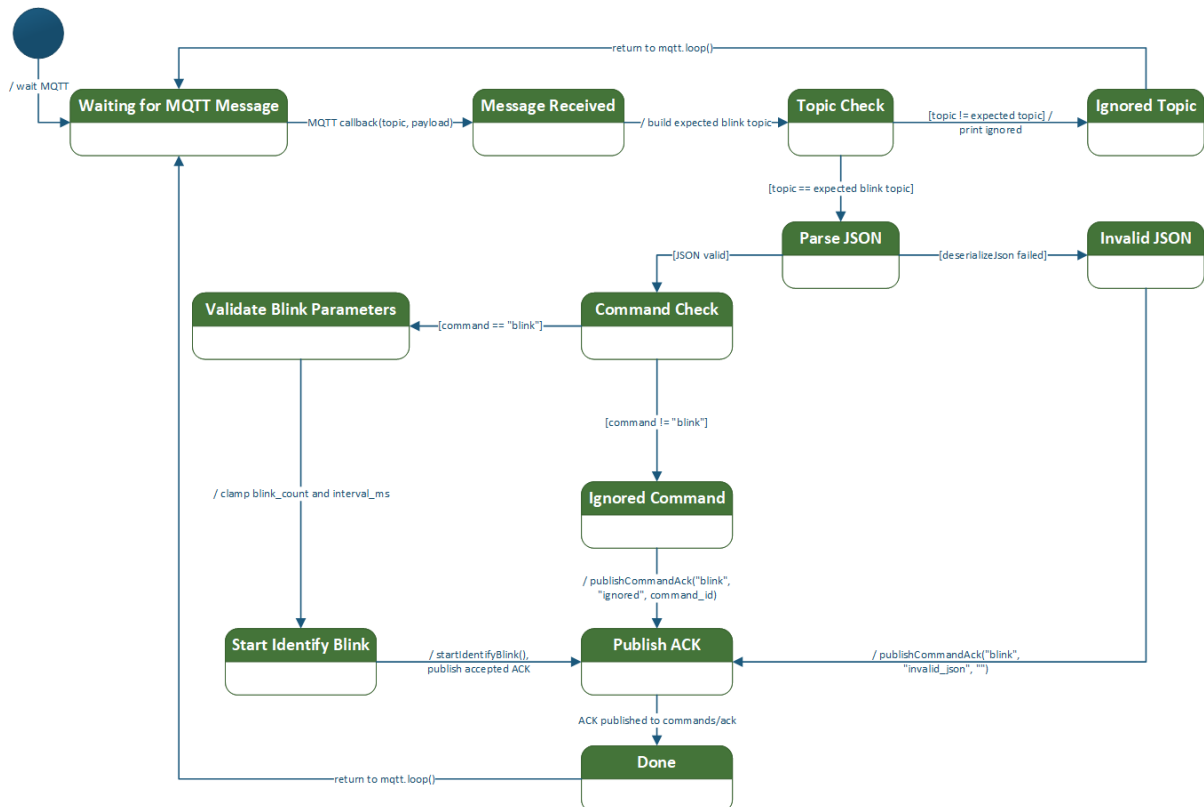


Figure 7.2. 1. 5 MQTT blink command state machine.

7.2.1.6. RFID Scan State Machine

The RFID Scan state machine describes repeated non-blocking polling of the UHF reader. When a tag is available, the firmware reads and normalizes its identifier, checks whether the same observation was published too recently, and constructs the appropriate event for the node's role. An accepted observation is published through MQTT and enters a short suppression or cooldown period before the same tag can be sent again. Empty reads, malformed data, reader errors, or publish failures return control to the polling cycle without stopping other firmware tasks.

The diagram below shows tag-reading and scan-publication behavior.

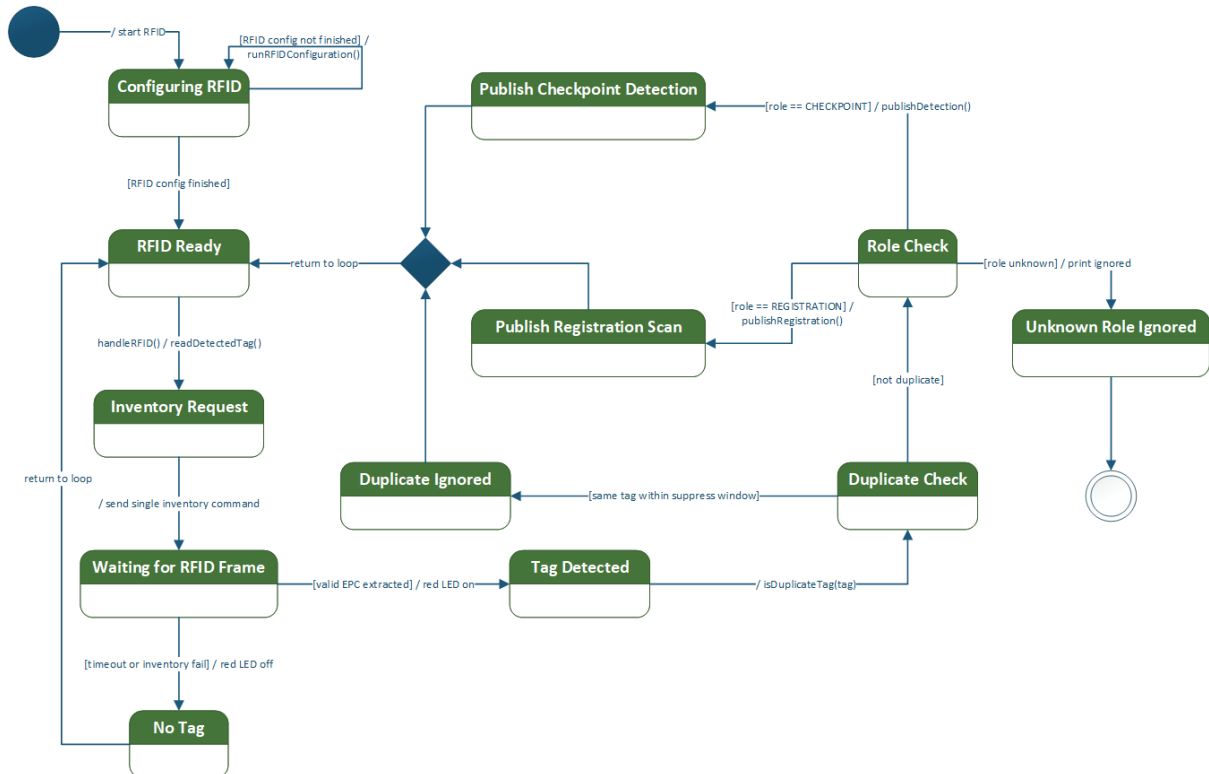


Figure 7.2. 1. 6 RFID scan state machine.

7.2.1.7. Heartbeat Status Publish State Machine

A periodic timer drives the Heartbeat Status Publish state machine. When the interval expires and MQTT is available, the firmware builds a status payload from node identity, role, connection, and runtime information and attempts to publish it. A successful publish updates the last-heartbeat time and returns to the waiting state. If connectivity is unavailable or publishing fails, the heartbeat remains pending for a later cycle while the MQTT connection state machine performs recovery.

The diagram below shows heartbeat publish timing and status reporting.

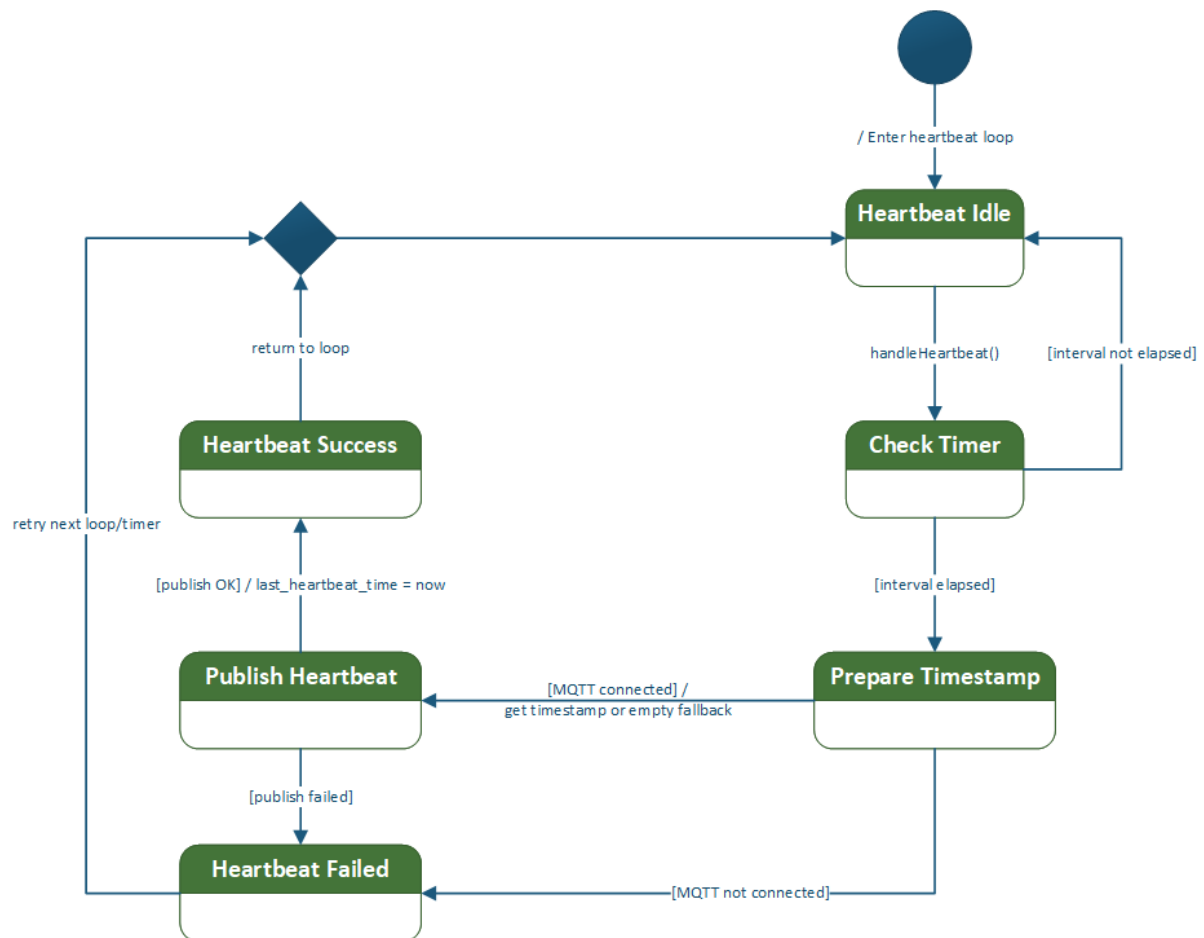


Figure 7.2. 1. 7 Heartbeat publish state machine.

7.2.1.8. Remote Config Refresh State Machine

The Remote Config Refresh state machine allows an operating node to obtain backend-controlled changes without manual reflashing. A scheduled interval or supported refresh command starts the request, after which the firmware validates the returned configuration before applying and persisting it. Changes that affect MQTT topics, roles, or locations cause the corresponding runtime modules to be updated or reconnected. Invalid or unavailable data is rejected, and the previously known valid configuration remains active until a later retry.

The diagram below shows a periodic or requested remote configuration refresh.

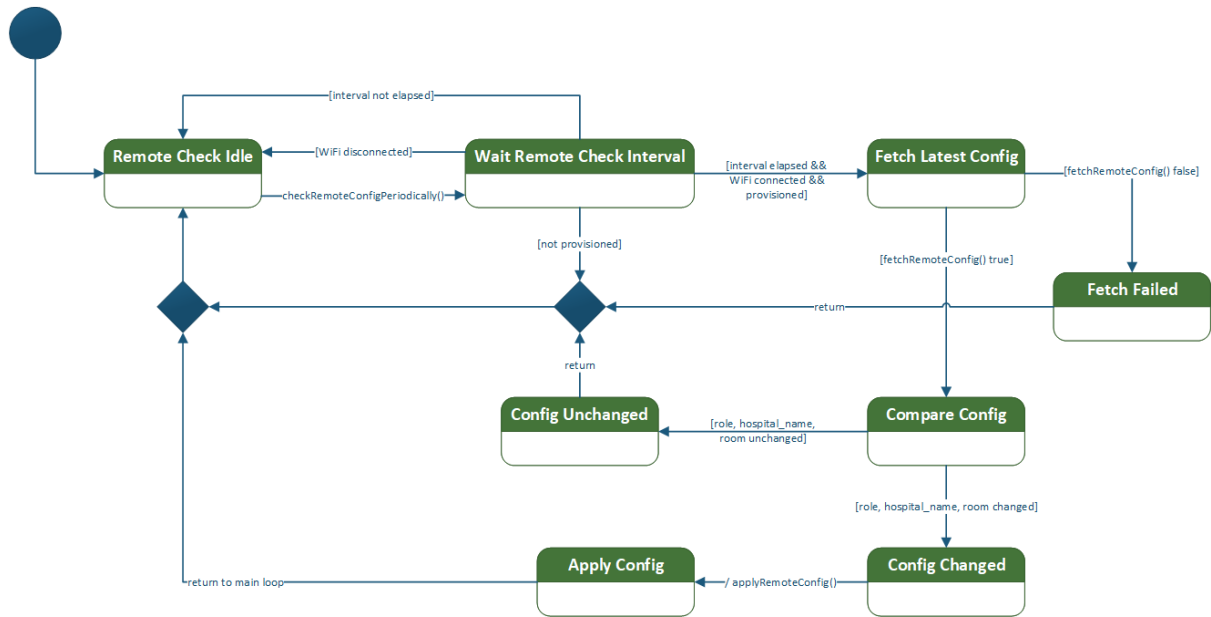


Figure 7.2. 1. 8 Remote configuration refresh state machine.

7.2.1.9. Reset Button State Machine

The Reset Button state machine separates an intentional configuration reset from accidental electrical noise or a short press. Firmware detects the button transition, applies debouncing and hold-duration checks, and enters reset confirmation only after the required condition is maintained. A confirmed reset clears the stored network and node configurations, provides visible LED feedback, and restarts into the captive portal setup. Releasing the button too early cancels the action and returns to normal operation.

The diagram below shows reset/configuration button behavior.

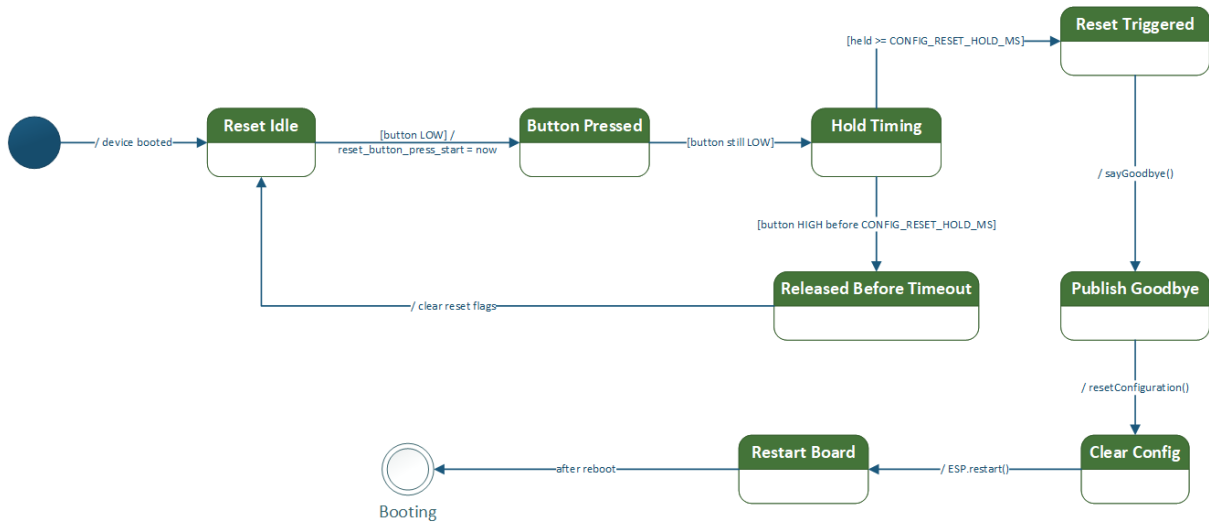


Figure 7.2. 1. 9 Reset button state machine.

7.2.1.10. LED State Machine

The LED State machine defines the normal visual patterns representing setup, connecting, online, offline, and error conditions, as well as the temporary pattern used for physical identification. Status changes select the appropriate base pattern without blocking MQTT, RFID, or heart-beat processing. A blink command temporarily overrides the base pattern for a fixed duration, after which the controller restores the indication for the current operational state.

The diagram below shows LED states for normal status and identification behavior.

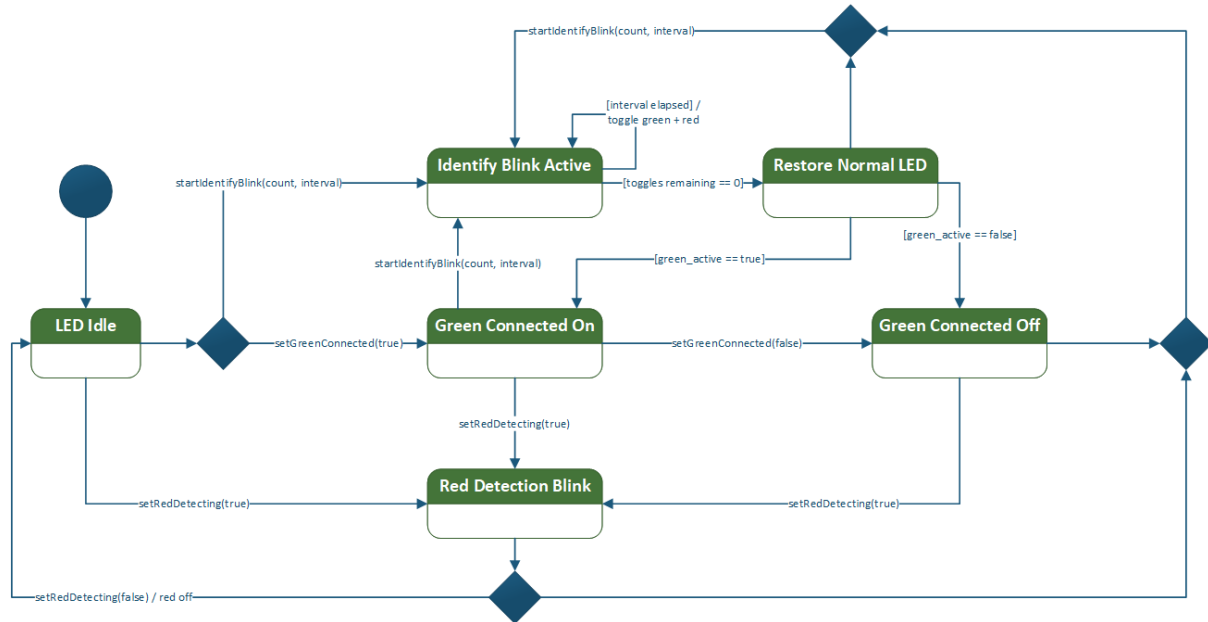


Figure 7. 2. 1. 10 LED state machine.

7.2.2. System and Application State Machines

The system and application state machines describe the lifecycle and operational behavior of records and processes managed outside the embedded firmware. Together, they explain how node records move through discovery, assignment, online, offline, and disabled states; how assets are registered, deregistered, located, and moved; how movement requests are approved, rejected, canceled, or completed; how authenticated sessions and WebSocket connections are established and terminated; and how alerts are opened and resolved. Each subsection identifies the event that initiates a transition, any guard or validation condition that must be satisfied, and the resulting state when the transition succeeds or fails.

7.2.2.1. Node Lifecycle State Machine

The Node Lifecycle State Machine represents the backend-managed record, not the internal firmware connection state. A previously unknown device enters DISCOVERED when the backend accepts its discovery or first valid contact. An authorized assignment specifies its role and location; a valid heartbeat makes it ONLINE, while a heartbeat timeout makes it OFFLINE. Administrators or technicians may enable, disable, or unassign an eligible node; deletion is permitted only in accordance with the documented safety rules. UNKNOWN remains a frontend fallback for unexpected values rather than a normal stored state.

The diagram below shows DISCOVERED, ASSIGNED, ONLINE, OFFLINE, DISABLED, and DELETED, including heartbeat, timeout, assignment, enable/disable, unassign, and deletion transitions.

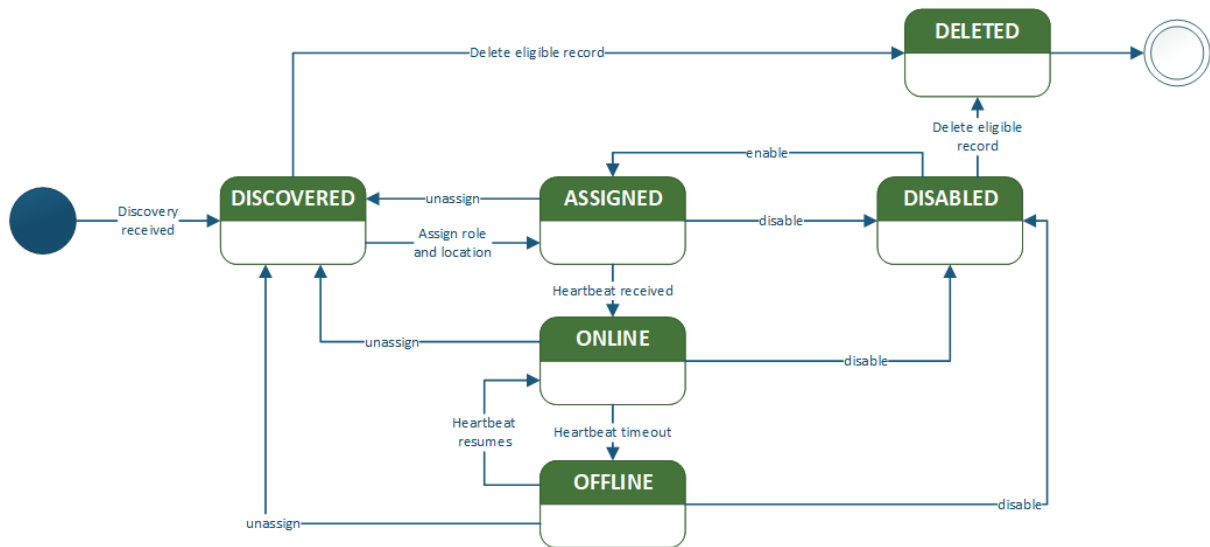


Figure 7. 2. 2. 1 Node lifecycle state machine.

7.2.2.2. Asset Lifecycle State Machine

The Asset Lifecycle State Machine indicates whether an RFID-tagged asset is in active tracking. Before registration, it is UNREGISTERED; successful validation and creation move it to ACTIVE; authorized deregistration moves it to DEREGISTERED while preserving historical detections and movement records. A later registration may reuse the physical tag only when the backend's uniqueness and history policy allows it. Location and movement conditions are intentionally excluded because the separate Asset Flow State Machine owns them.

The diagram below shows the UNREGISTERED, ACTIVE, and DEREGISTERED states, with transitions for registration, deregistration, and re-registration.

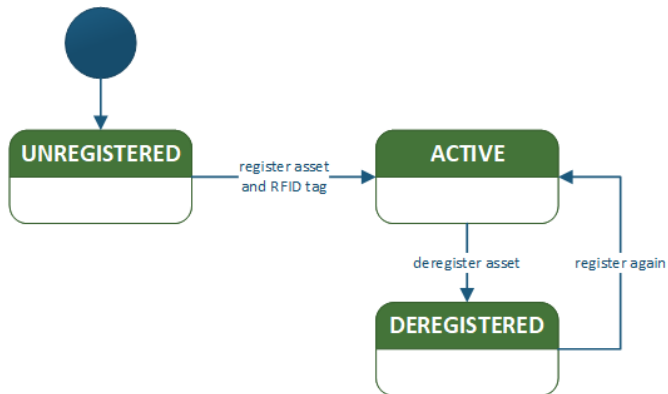


Figure 7. 2. 2. 2 Asset lifecycle state machine.

7.2.2.3. Asset Flow State Machine

The Asset Flow State Machine describes the operational status shown in both applications. A newly registered asset begins ON THE MOVE until a checkpoint detects it in its assigned room, which produces IN PLACE. A submitted request produces PENDING APPROVAL; rejection or cancellation returns the asset to its prior valid location state, while approval authorizes ON THE MOVE. Detection at the approved destination returns the asset to IN PLACE, completing the request. Detection in an unexpected room results in UNAUTHORIZED until the asset returns to the expected room or enters another valid, approved movement.

The diagram below shows ON THE MOVE, IN PLACE, PENDING APPROVAL, and UNAUTHORIZED, including registration, request, approval/rejection, expected-room detection, wrong-location detection, and return transitions.

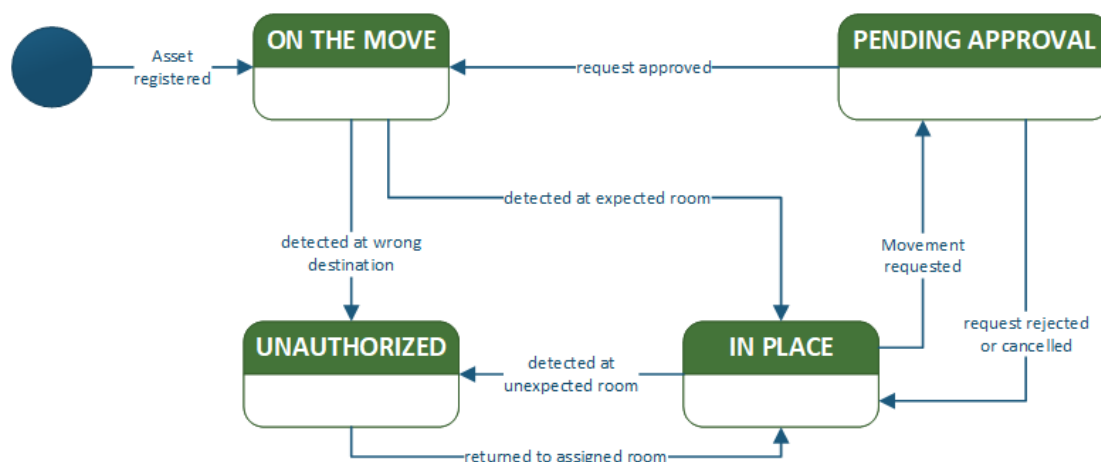


Figure 7. 2. 2. 3 Asset flow state machine.

7.2.2.4. Movement Request State Machine

The Movement Request State Machine applies to one persistent request record. Creation produces PENDING, from which only permitted users may approve or reject, and the requesting workflow may cancel under the implemented rules. APPROVED means movement is authorized but still incomplete; only a detection at the approved destination changes it to COMPLETED. REJECTED, CANCELLED, and COMPLETED are terminal outcomes for that record, which prevent an old decision from being reopened or completed through an unrelated later detection.

The diagram below shows PENDING, APPROVED, REJECTED, CANCELLED, and COMPLETED, including the required destination detection after approval.

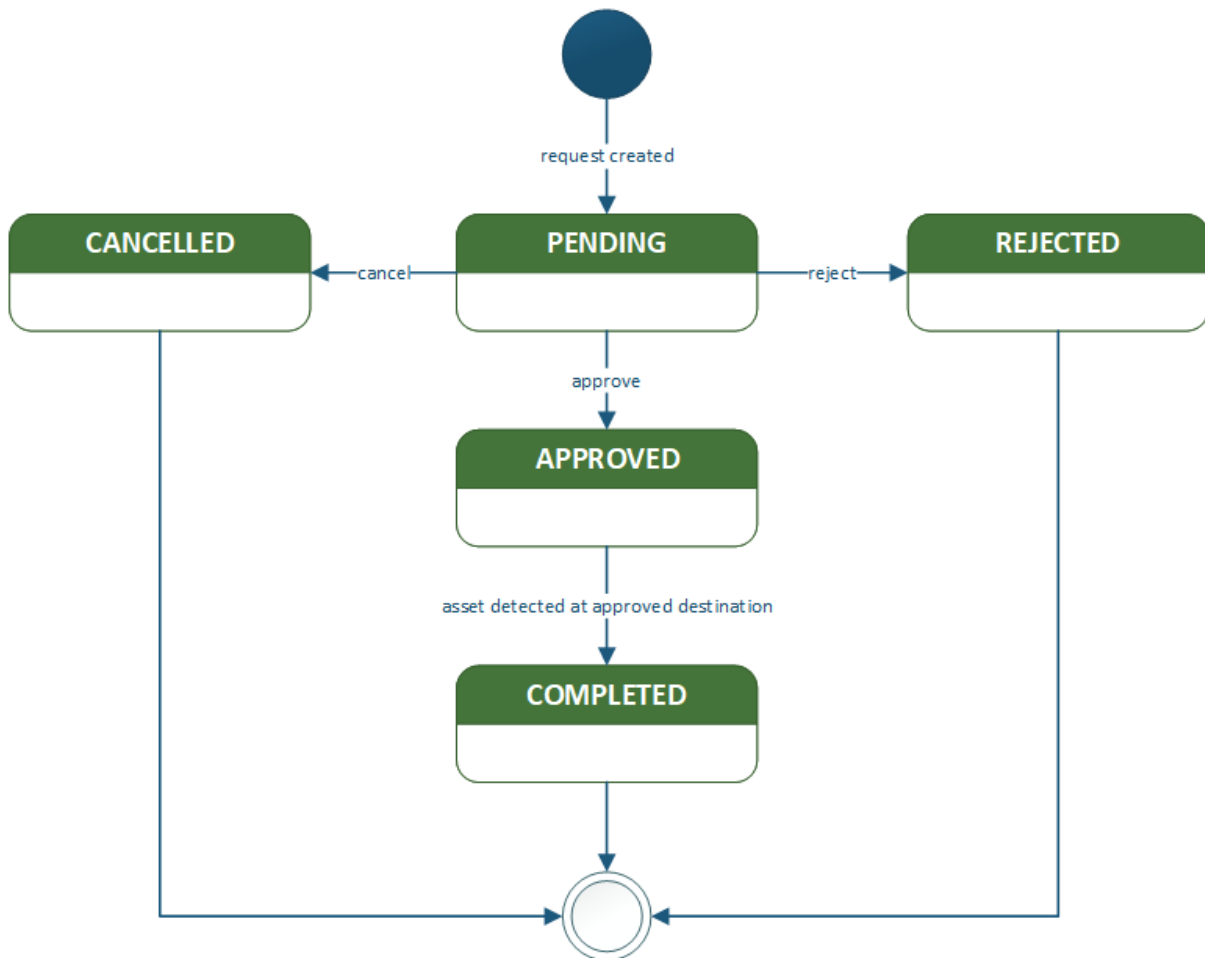


Figure 7. 2. 2. 4 Movement request state machine.

7.2.2.5. Authentication and Session State Machine

The Authentication and Session State Machine models the user-facing session shared by the two browser applications. A signed-out user enters AUTHENTICATING after submitting credentials and becomes AUTHENTICATED only after the backend accepts the account and role. Invalid credentials return the session to the signed-out state. At the same time, token expiry or an access-denied response moves the session into a failure state, clearing local authentication data and redirecting to the login page. Explicit logout follows the same cleanup path. First-admin setup is a separate pre-authentication condition available only while no administrator exists.

The diagram below shows SIGNED OUT, AUTHENTICATING, AUTHENTICATED, TOKEN EXPIRED, and ACCESS DENIED, with login, rejection, logout, expiry, and session-cleanup transitions.

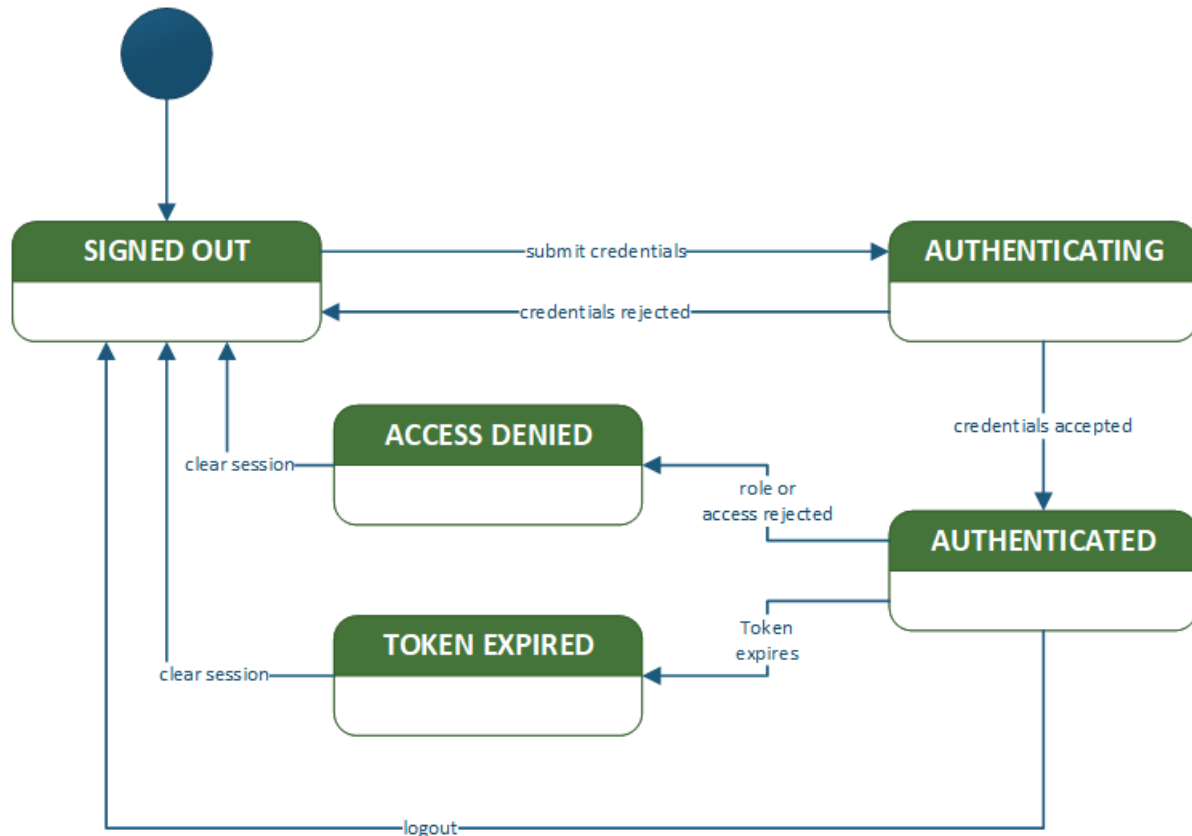


Figure 7. 2. 2. 5 Authentication and session state machine.

7.2.2.6. WebSocket Connection State Machine

The WebSocket Connection State Machine begins only when a valid authenticated session exists. The client transitions from DISCONNECTED to CONNECTING, then to CONNECTED after the socket opens. Temporary transport loss enters RECONNECTING with controlled retry timing; a successful retry restores CONNECTED and may trigger an API refresh to recover missed data. Logout, token invalidation, or a deliberate application shutdown moves the connection to STOPPED, so retry loops do not continue without authentication.

The diagram below shows DISCONNECTED, CONNECTING, CONNECTED, RECONNECTING, and STOPPED, including valid-session checks, failure retry, recovery, logout, and token invalidation.

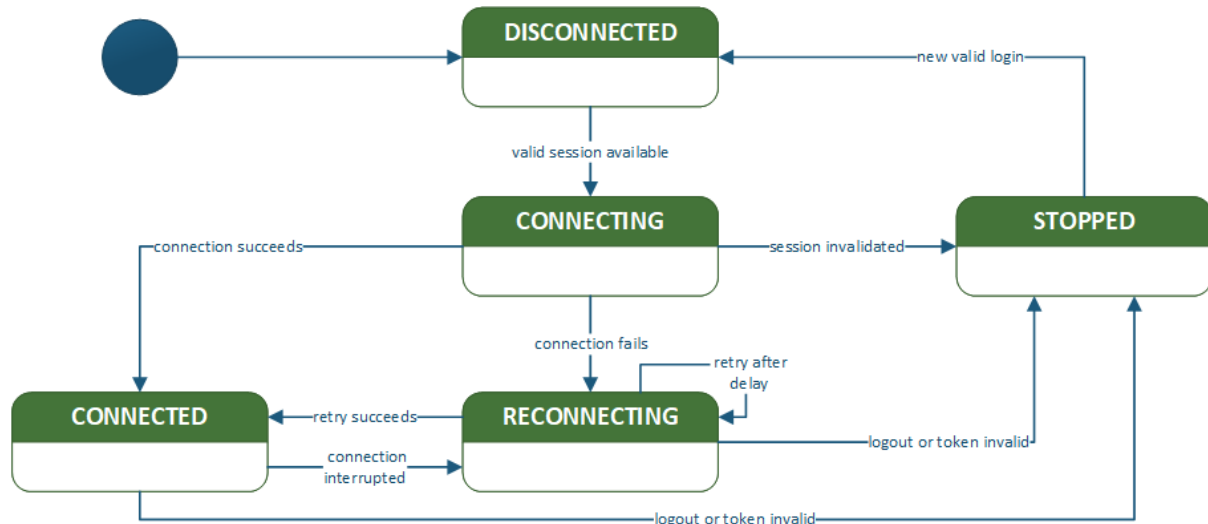


Figure 7. 2. 2. 6 WebSocket connection state machine.

7.2.2.7. Alert Lifecycle State Machine

The Alert Lifecycle State Machine describes the persisted status of an operational warning. Detection of an unknown tag, node heartbeat timeout, unauthorized asset location, or another implemented condition creates or reopens an OPEN alert with the relevant asset or node context. When the underlying condition clears, such as a node returning online or an asset returning to its expected location, the backend marks the alert RESOLVED according to its resolution policy. ACKNOWLEDGED is intentionally omitted because it should not appear in the design unless the backend provides and stores that action.

The diagram below shows OPEN and RESOLVED states, with alert creation and condition-cleared/resolution transitions, and includes examples of alert-triggering conditions.

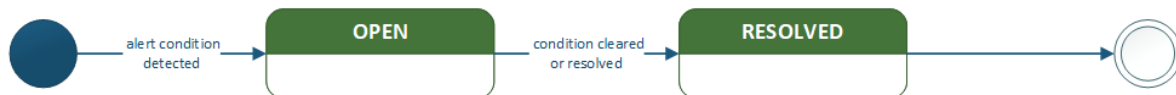


Figure 7. 2. 2. 7 Alert lifecycle state machine.

8. Data Flow and Communication Design

This section describes how data moves between users, web applications, backend services, the database, the MQTT broker, and sensor nodes. It also explains why each protocol is used.

8.1. Overall Communication Flow

The overall communication flow connects hospital users, browser applications, the backend, the database, the MQTT broker, and sensor nodes. User actions travel through HTTPS APIs, sensor-node events travel through MQTT, and backend live updates travel through WebSocket.

8.2. Web Application and API Communication

The web applications communicate with the backend using HTTPS JSON requests. Requests include authentication, data loading, node actions, asset actions, movement actions, user management actions, and system setting actions. Every protected request should include a valid bearer token.

8.3. Sensor Node and MQTT Communication

Sensor nodes communicate with the MQTT broker using configured topics. Registration Desk Nodes publish registration scans, while Checkpoint Nodes publish RFID detections and heart-beat messages. The backend subscribes to these topics, validates messages, updates state, and may publish supported commands back to nodes.

8.4. Backend Data and Live Update Communication

After the backend processes a valid request or MQTT event, it updates the PostgreSQL database through the data access layer. When the update affects user-visible state, the backend sends a WebSocket event to connected clients. The code-level frontend real-time diagrams are listed in Sections 6.3.3 and 6.4.3, while the backend MQTT processing diagram is listed in Section 6.2.4.

8.5. Communication Protocol Design

This subsection describes the protocols used by the system. It does not require a diagram. A protocol table is clearer because each protocol serves a different purpose and follows a different communication direction.

Table 8. 5. 1 List of system communication protocols and their purposes.

No.	Protocol	Used between	Purpose
1.	HTTPS/JSON	- Registration App - Monitor App - FastAPI backend	Used for normal request-response operations such as login, loading tables, submitting forms, node actions, asset actions, movement actions, and settings.
2.	WebSocket/WSS	FastAPI to browser clients	Used for live updates to assets, nodes, alerts, activities, scans, and movement requests without manual refresh.
3.	MQTT	- ESP32 nodes - MQTT broker - Backend MQTT client	Used for RFID scans, checkpoint detections, heartbeats, node commands, and command acknowledgments.
4.	UART/Serial	- ESP32 firmware - UHF RFID reader	Used for local tag-reading data exchange between the microcontroller and the RFID reader module.
5.	Wi-Fi	- ESP32 nodes - Hospital/network infrastructure	Provides network access for MQTT communication and node configuration.
6.	PostgreSQL connection	- Backend - PostgreSQL Database service	Used by backend services to store users, nodes, assets, detections, movements, alerts, activity records, and settings.

8.6. MQTT Topic Design

MQTT topics should include enough structure to identify the hospital or environment, the node, the message category, and the command or event type. Topic names should be documented in a reference table so firmware and backend implementations remain compatible.

Table 8. 6. 1 List of system MQTT topic designs.

No.	Message category	Example content to document	Recommended format
1.	Registration scan	- Node ID - Tag ID - Timestamp - Message type	hospital/{hospital_ID}/nodes/{node_ID}/registration/scan
2.	Checkpoint detection	- Node ID - Tag ID - Detected time - Reader status	hospital/{hospital_ID}/nodes/{node_ID}/rfid/detection
3.	Heartbeat	- Node ID - Firmware version - Online status - Timestamp	hospital/{hospital_ID}/nodes/{node_ID}/heartbeat
4.	Backend command	- Command type such as “blink.” - Command ID - Target node ID	hospital/{hospital_ID}/nodes/{node_ID}/commands
5.	Acknowledgement	- Command ID - Node ID - Success/failure feedback - Message	hospital/{hospital_ID}/nodes/{node_ID}/commands/ack

8.7. WebSocket Event Design

WebSocket events should use clear event names and predictable payload structures. The frontend should explicitly handle known event types and safely ignore unknown ones.

Table 8. 7. 1 List of system WebSocket designs.

No.	Event type	Typical producer	Pages that may consume
1.	asset_updated	- Asset registration service - Movement service - RFID detection processing	- Registration App asset page - Monitor App dashboard - Assets page
2.	node_updated	- Node service - Heartbeat processing - Offline monitor	- Registration App asset page - Monitor App dashboard - Assets page
3.	alert_created	- Alert service after an unknown tag - Offline node - Unauthorized movement	- Monitor App dashboard - Alerts page

4.	activity_created	○ Activity logging service	○ Monitor App activity page
5.	movement_request_updated	○ Movement request service	○ Registration App movement requests ○ Monitor App assets page

8.8. API Request and Response Design

API requests and responses should use consistent JSON structures. Success responses should return the updated or requested resource where useful. Error responses should return a safe message and a machine-readable error code.

Table 8. 8. 1 List of system API requests and responses.

No.	API area	Design content to document
1.	Authentication	Login, logout/session clearing, first-admin setup, password change
2.	Nodes	List, assign, edit, unassign, delete, blink, heartbeat-derived status
3.	Assets	Register, deregister, list, scan handling, location state
4.	Movement requests	Request, cancel, approve, reject, complete from detection
5.	Alerts and activity	List alerts, list activity, filter/search/sort

9. Interface Design

This section defines the main interfaces used by the system: hardware, firmware, backend, MQTT, WebSocket, frontend, and external service.

9.1. Hardware Interfaces

Hardware interfaces include ESP32-to-UHF-RFID-reader communication, ESP32 GPIO connections to status LEDs, power input, and physical reset or configuration controls. The wiring sketch from the attachment should be used here because it explains physical connections better than text alone.

The diagram below shows how the ESP32, UHF RFID reader, power source, LEDs, and reset button of the node module are wired and interconnected.

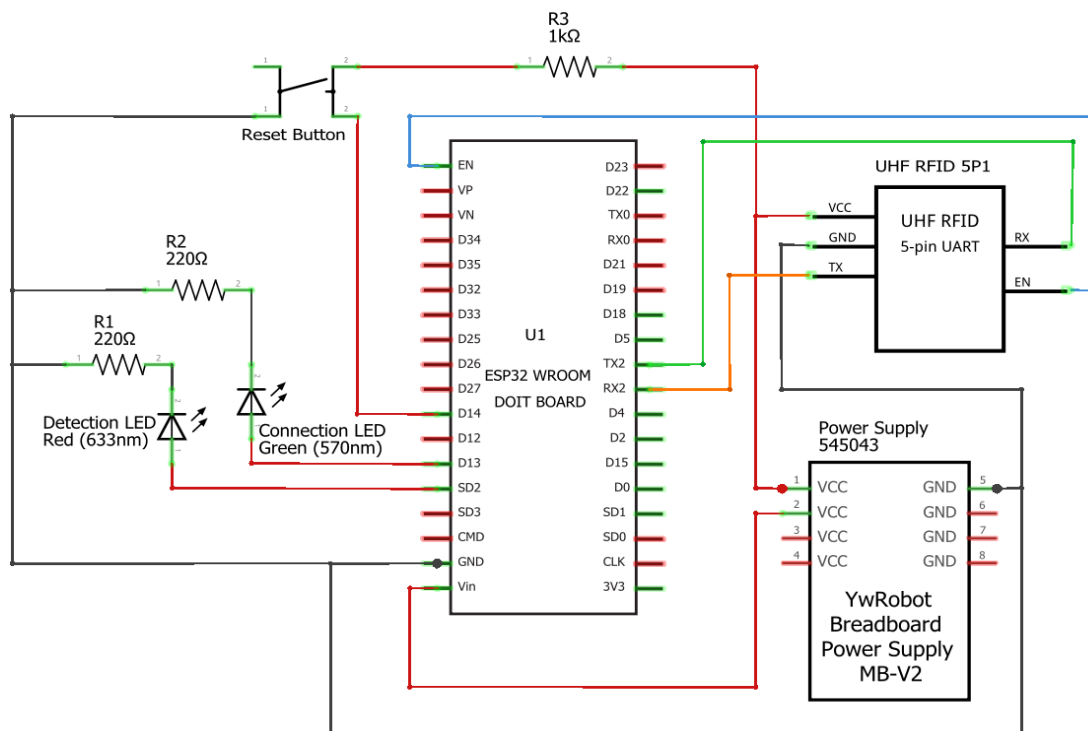


Figure 9. 1. 1 Hardware wiring overview.

9.2. Firmware Interfaces

Firmware interfaces define how internal firmware modules interact. The detailed firmware interface diagrams are placed in Section 6.3 to avoid duplication. This section should summarize what each interface provides and how it supports testing.

9.3. Backend API Interfaces

Backend API interfaces define routes, request bodies, response bodies, authentication requirements, and role permissions. This section should use text and endpoint tables instead of images.

9.4. MQTT Interfaces

MQTT interfaces define topic names, payload fields, event types, command types, acknowledgment rules, and reconnection expectations. Firmware and backend must agree on these definitions.

9.5. WebSocket Interfaces

WebSocket interfaces define how browser clients connect, authenticate, receive events, and reconnect. The backend should send clear event types and payloads so frontend handlers can update the correct page state.

9.6. Frontend User Interface Design

The user interface design should describe the main pages, navigation behavior, role-based visibility, forms, tables, status badges, modals, search, sorting, pagination, and responsive layout requirements. No screenshot is required in this version unless final UI screenshots are created later. Detailed component structures for the Registration App and Monitor App are provided in Sections 6.3 and 6.4.

9.7. External Service Interfaces

External service interfaces include the MQTT broker, cloud hosting platform, PostgreSQL service, and deployment-level configuration services. These interfaces should be documented because service availability affects the whole system.

10. Error Handling and Recovery Design

This section describes how the system should respond to errors, invalid input, service interruption, connection loss, and recovery situations. It does not require a diagram from the current attachment.

10.1. Frontend Error Handling

Frontend error handling covers failed API requests, validation errors, unauthorized access, expired tokens, loading states, WebSocket connection failures, and backend service unavailability. Users should receive clear messages, and the UI should avoid appearing frozen.

Table 10. 1 List of the system's frontend error handling.

No.	Scenario	Expected System Handling	Recovery or Outcome
1.	API request fails	Frontend error handling covers failed API requests, validation errors, unauthorized access, expired tokens, loading states, WebSocket connection failures, and backend service unavailability. Users should receive clear messages, and the UI should avoid appearing frozen.	The user retries after the backend or network becomes available.
2.	Form data is invalid.	Highlight the affected fields and prevent submission until the values are corrected.	The user retries after the backend or network becomes available.

3.	Session expires, or access is denied.	Clear invalid session data and redirect the user to the login page or an access-denied result.	The user signs in again with an authorized account.
----	---------------------------------------	--	---

10.2. Backend Error Handling

Backend error handling covers invalid input, authentication and authorization failures, business-rule violations, database failures, MQTT-processing failures, and unexpected server exceptions. Known errors should produce controlled responses, while unexpected exceptions should be safely logged without exposing internal details.

Table 10. 2 List of the system's backend error handling.

No.	Scenario	Expected system handling	Recovery or outcome
1.	The business rule is violated.	Reject the operation with a consistent error code and a user-safe explanation.	No database change is committed; the frontend displays the reason.
2.	An unauthorized role calls a protected route.	Return an authentication or authorization error before executing the service operation.	The restricted action is not performed, and the attempt may be logged.
3.	An unexpected server exception occurs.	Roll back active work, log technical details, and return a generic server error.	Maintainers investigate the log without exposing internal details to users.

10.3. MQTT Error Handling

MQTT error handling includes broker disconnection, invalid topics, malformed payloads, unknown node identities, unsupported command types, and missed acknowledgments.

Table 10. 3 List of the system's MQTT error handling.

No.	Scenario	Expected system handling	Recovery or outcome
1.	Broker connection is lost.	Mark MQTT as disconnected, then retry the connection at controlled intervals.	Subscriptions are restored after reconnecting.
2.	Topic or payload is invalid.	Reject the message, record the validation problem, and avoid changing system data.	Valid later messages continue to be processed normally.
3.	Node command acknowledgment is not received.	Treat the command as unconfirmed after the configured timeout and report the result.	The user may retry after checking node connectivity.

10.4. WebSocket Reconnection

WebSocket reconnection design defines how browser clients reconnect after a temporary disconnection and reload data to recover missed updates.

Table 10. 4 List of the system's WebSocket error handling.

No.	Scenario	Expected system handling	Recovery or outcome
1.	Temporary WebSocket interruption	Show a disconnected state and attempt reconnection with a limited retry delay.	Live updates resume automatically when the connection returns.
2.	The connection token is invalid or expired.	Stop reconnecting, clear the invalid session, and return the user to the login screen.	A new authenticated session is required.
3.	Updates may have been missed	Reload the current data through the API after reconnection succeeds.	The visible state is synchronized with the backend source of truth.

10.5. Firmware Recovery

Firmware recovery handles Wi-Fi loss, MQTT disconnection, invalid configuration, RFID reader communication issues, and command processing failures.

Table 10. 5 List of the system's firmware recovery.

No.	Scenario	Expected system handling	Recovery or outcome
1.	Wi-Fi connection is lost.	Keep the node running and periodically retry the configured network connection.	Normal communication resumes without a manual restart when Wi-Fi returns.
2.	MQTT connection is lost	Reconnect to the broker and resubscribe to the required command topics.	Publishing, commands, and acknowledgments resume after recovery.
3.	The configuration, or the RFID reader, is unavailable	Enter setup or error-handling mode and indicate the problem via the node status output.	A technician corrects configuration, wiring, power, or reader communication.

10.6. Database Error Handling

Database error handling protects data integrity when writes fail or transactions cannot be completed.

Table 10. 6 List of the system's database error handling.

No.	Scenario	Expected system handling	Recovery or outcome
1.	Multi-step update fails	Abort the operation and return a temporary service error without pretending it succeeded.	The action can be retried after database connectivity is restored.
2.	Multi-step update fails	Roll back the complete transaction so related records are not partially updated.	The previous consistent state remains stored.
3.	Unique or integrity constraint is violated.	Reject the write and translate the database error into a safe validation response.	The user corrects duplicate or conflicting data before retrying.

10.7. User Feedback and Error Messages

User-facing error messages should be clear, short, and safe. They should not expose stack traces, database details, security tokens, or secrets.

Table 10. 7 List of the system's user feedback and error messages.

No.	Scenario	Expected system handling	Recovery or outcome
1.	User-correctable input error	State what must be corrected and place the message near the affected field or action.	The user can fix the problem immediately.
2.	Temporary service or connection failure	Explain that the operation could not be completed, and provide a retry action when it is safe to do so.	The user retries without duplicate submission.
3.	Internal or security-sensitive failure	Display a generic reference-safe message while logging technical details privately.	Secrets, stack traces, tokens, and database details remain hidden.

11. Security Design

This section describes the security-related design decisions for authentication, authorization, sessions, passwords, API protection, sensitive configuration, and admin safety. It can remain text- or table-based.

11.1. Authentication Design

Authentication is based on user credentials and token-based sessions. The backend verifies account status, password hash, and role before issuing a token.

11.2. Role-Based Authorization

Role-based authorization controls access to pages and backend actions. The frontend improves usability by hiding unavailable actions, while the backend remains authoritative for security enforcement.

11.3. Token and Session Handling

Token and session handling should allow users to remain signed in while the token is valid and clear the session upon logout, token expiry, or access rejection.

11.4. Password Security

Passwords are hashed before storage and are never stored or logged in plain text. Password creation and password-change operations validate the submitted values and reject unauthorized or invalid changes.

11.5. API Protection

API protection includes authentication checks, role checks, validation, error sanitization, CORS configuration, and controlled access to API documentation in production.

11.6. Sensitive Configuration Handling

Sensitive values such as database URLs, JWT secrets, broker credentials, and production secrets should be stored in environment variables or protected configuration.

11.7. Admin Safety Rules

Admin safety rules prevent actions that could lock all administrators out of the system, such as disabling, deleting, or demoting the last active admin account.

12. Deployment Design

This section describes where the system components are deployed and how development and production environments are configured.

12.1. Deployment Overview

The system is deployed across hospital client devices, physical ESP32 sensor nodes, cloud-hosted frontend applications, a cloud-hosted backend service, a PostgreSQL database, and an MQTT broker.

Below is the system deployment overview. The complete version can be seen in the appendices at the end of this document.

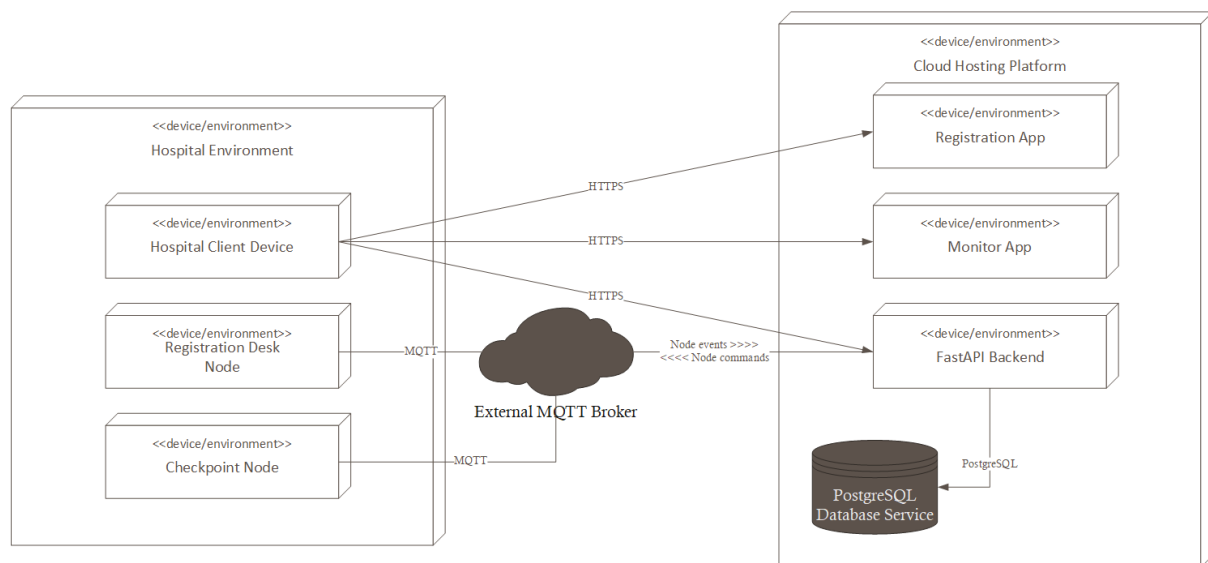


Figure 12. 1. 1 Deployment overview.

12.2. Local Development Deployment

In local development, the backend may run on the developer machine, the frontend applications may run through Vite development servers, and the database may be local or hosted. Local development should use separate environment variables or topic namespaces when possible.

12.3. Production Deployment

In production, the frontend applications are deployed as static web applications, the backend is deployed as a web service, and PostgreSQL is provided as a managed database service. Production communication should use HTTPS and secure WebSocket connections.

12.4. Frontend Deployment

Frontend deployment creates production builds of the Registration App and Monitor App. The build output is hosted as static application files and loaded by the hospital user's browser.

The image below shows browser clients and frontend production builds.

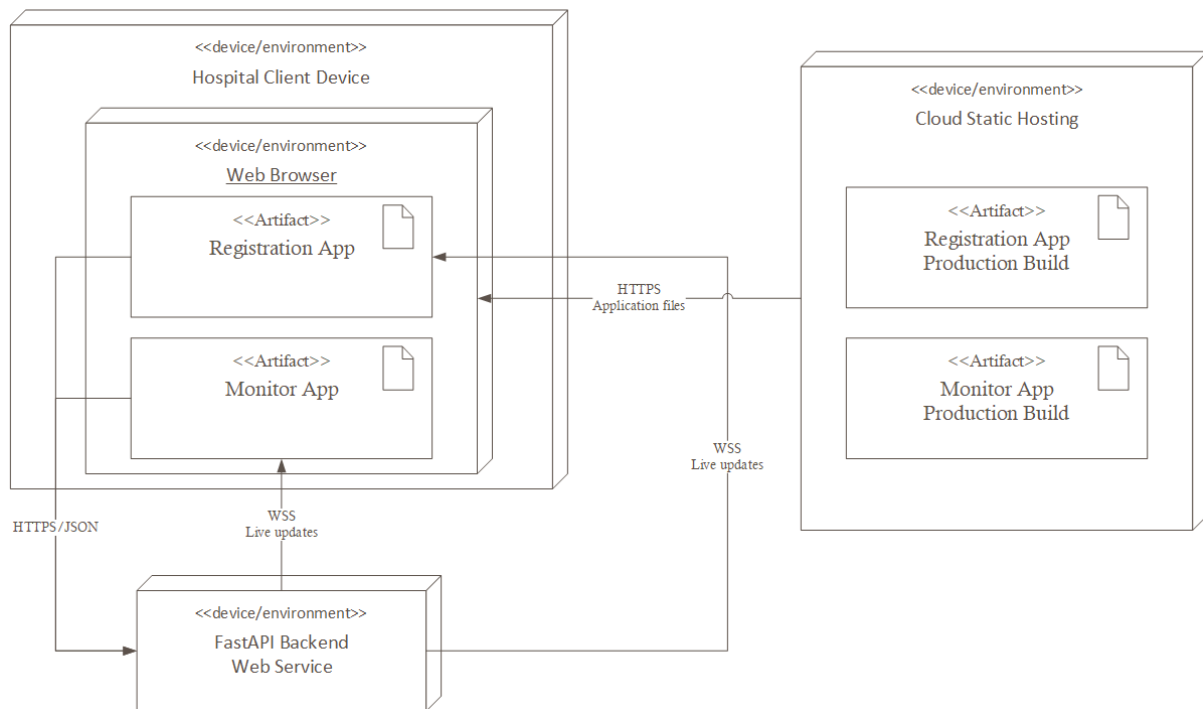


Figure 12. 4. 1 Frontend deployment diagram.

12.5. Backend Deployment

Backend deployment runs the FastAPI application with the required environment variables for database access, JWT configuration, CORS, MQTT configuration, and API documentation settings.

The image below shows the backend service, PostgreSQL, MQTT client, and browser clients.

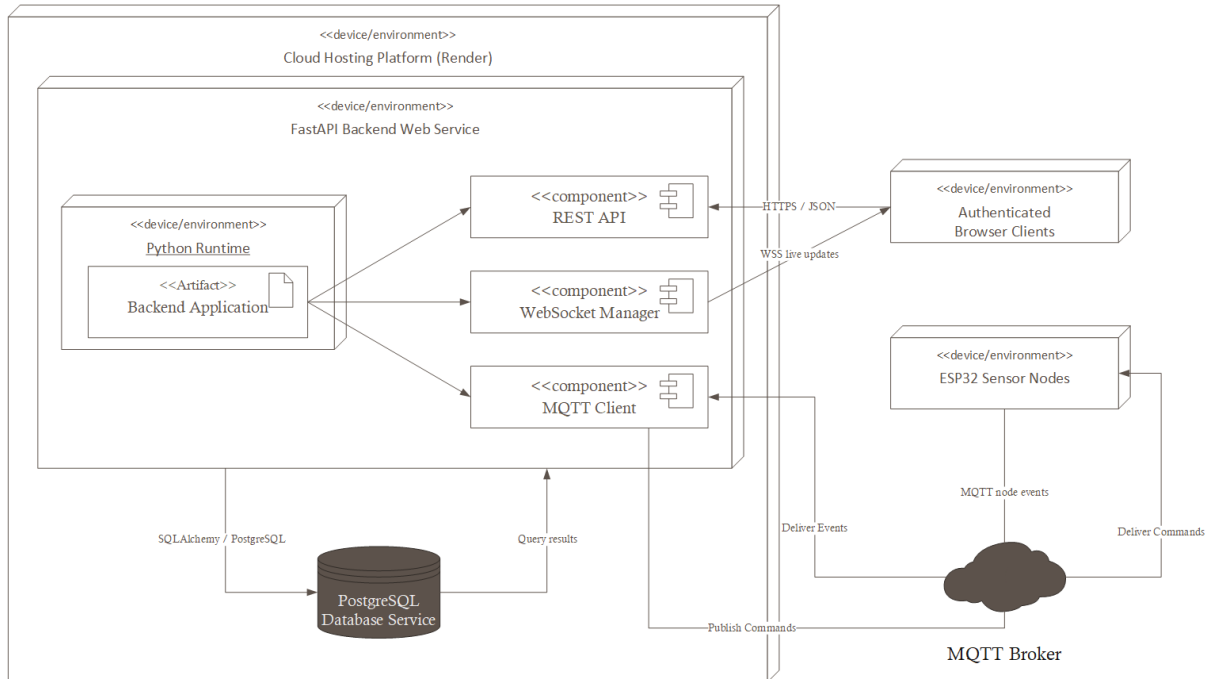


Figure 12. 5. 1 Backend deployment diagram.

12.6. Database Deployment

Database deployment provides persistent storage for system records. The database should support schema migrations, backups, and recovery procedures appropriate for the deployment stage.

12.7. MQTT Broker Deployment

The MQTT broker relays messages between sensor nodes and the backend MQTT client. The deployment should document the broker URL, port, topic namespace, authentication, TLS settings, and local/production separation.

12.8. Environment Variables and Configuration

Environment variables should configure database connection, JWT secret, allowed origins, MQTT broker settings, hospital or topic namespace, API documentation visibility, and frontend API endpoints.

12.9. Firmware Deployment

Firmware deployment installs the ESP32 firmware onto Registration Desk Nodes and Checkpoint Nodes. Deployment includes configuration of Wi-Fi, node identity, role, MQTT broker, RFID reader, and physical installation notes.

The image below shows an ESP32, an RFID reader, LEDs, Wi-Fi, and an MQTT broker.

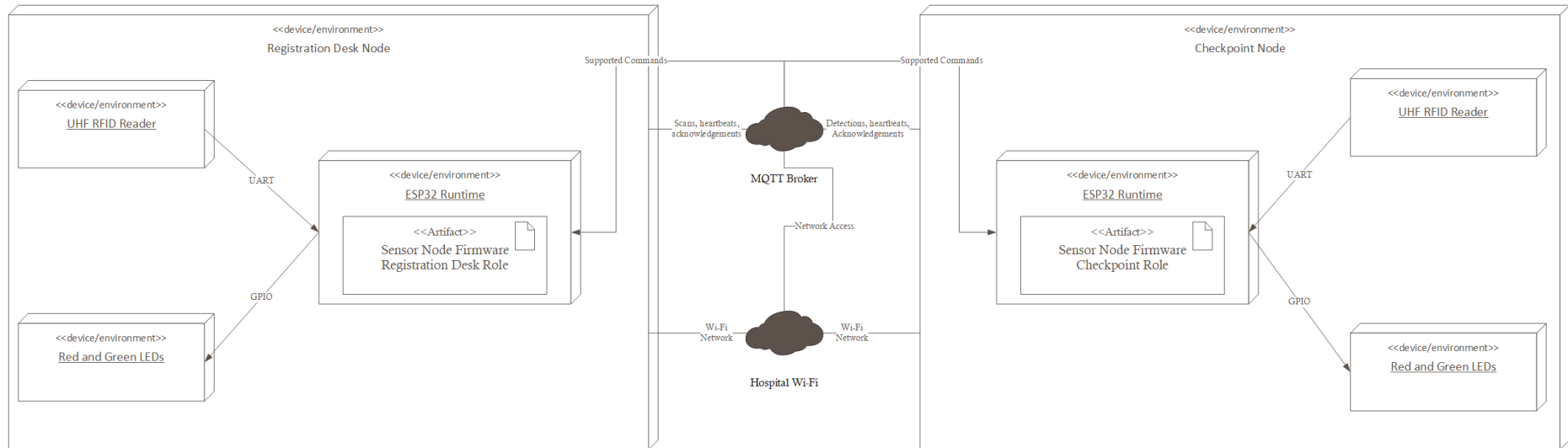


Figure 12. 9. 1 Firmware deployment diagram.

13. Testing and Verification

This section describes the testing approach needed to verify firmware, backend, frontend, integration, end-to-end, and acceptance behavior. It does not require an image from the current attachment.

13.1. Firmware Testing

Firmware testing should verify Wi-Fi setup, MQTT connection, topic subscription, RFID reading, heartbeat publishing, command acknowledgment, reconnection behavior, and LED control.

13.2. Backend Testing

Backend testing should cover authentication, authorization, user management, node management, asset registration and deregistration, movement requests, approvals and rejections, MQTT processing, alerts, activity, and WebSocket broadcasting.

13.3. Frontend Testing

Frontend testing should cover page rendering, role-based navigation, form validation, loading states, API error handling, WebSocket event handling, sorting, searching, pagination, and responsive layout behavior.

13.4. Integration Testing

Integration testing verifies that multiple components work together, such as frontend-to-backend API requests, backend-to-database updates, MQTT event processing, and backend-to-frontend WebSocket updates.

13.5. End-to-End Testing

End-to-end testing verifies complete user and sensor workflows, from first-admin setup and login to asset registration, movement approval, checkpoint detection, and UI update.

13.6. Acceptance Testing

Acceptance testing checks whether the implemented system satisfies the agreed requirements and use cases using representative normal and alternative workflows.

14. Design Decisions and Trade-Offs

This section records important design decisions and their consequences. It helps reviewers understand why the system was designed this way and what limitations remain.

Table 14. 1 List of design trade-offs.

No.	Decision	Alternative considered	Reason selected	Benefit	Trade-off or risk	Mitigation
1.	Checkpoint-based location	Continuous indoor positioning or RTLS	Provides useful latest-known-location information with lower hardware and deployment complexity.	Lower cost and simpler installation.	No continuous position between checkpoints.	Place checkpoints at meaningful transitions and display the result as the latest known location.
2.	MQTT for node events	Direct HTTP polling from each node	Lightweight asynchronous messaging suits ESP32 nodes and recurring events.	Loose coupling and efficient event delivery.	Broker dependency and topic-contract complexity.	Validate topics/payloads, reconnect, resubscribe, and separate environments.
3.	WebSocket live updates	Manual refresh or frequent HTTP polling	Operational pages need prompt updates after backend changes.	Reduced manual refresh and lower polling overhead.	Connection lifecycle and missed-event recovery are more complex.	Use controlled reconnect and refresh authoritative API data after recovery.
4.	Backend-enforced roles	Frontend-only navigation restrictions	Browser clients cannot be trusted to enforce authorization.	Protected business rules remain effective against direct API calls.	More route dependencies and policy tests.	Centralize role checks and test allow/deny behavior for every protected action.
5.	Cloud deployment	Local-only deployment	Supports remote development and demonstration access.	Simpler shared access to frontend, backend, and database.	External availability, cost, and configuration constraints.	Maintain local development support, backups, environment separation, and secure configuration.

14.1. Checkpoint-Based Tracking Decision

The system uses checkpoint-based tracking instead of continuous indoor positioning. This reduces complexity, cost, and hardware requirements while still providing useful last-known-location information.

14.2. MQTT Communication Decision

MQTT is used for sensor-node communication because it is lightweight and suitable for asynchronous events from embedded devices. The trade-off is broker dependency and the need for careful topic design.

14.3. WebSocket Live Update Decision

WebSocket is used to push live updates from the backend to connected frontend clients. The trade-off is added reconnection logic and event-type handling in the frontend.

14.4. Role-Based Access Decision

Role-based access separates responsibilities between administrators, technicians, registration staff, monitoring staff, and test users. The trade-off is additional complexity in frontend navigation and backend route protection.

14.5. Cloud Hosting Decisions

Cloud hosting simplifies access to the backend, frontend, and database during development and demonstration. The trade-off is dependency on external hosting availability, pricing limits, and secure configuration management.

14.6. Known Limitations

Known limitations include checkpoint-based location precision, RFID reading-range behavior, network dependency, MQTT broker dependency, and possible cloud free-tier constraints during testing.

15. Appendices

This section contains supplementary design information referenced throughout the main document. The appendices consolidate diagram descriptions, status and enumeration values, MQTT topic definitions, WebSocket event definitions, and backend API summaries into accessible reference tables. These references support developers, testers, maintainers, and reviewers by providing implementation-level details without repeating them in the main design sections. Where the implementation changes, the corresponding appendix entries should be updated together with the affected design sections and interface definitions.

15.1. Diagram List

The original embedded diagrams have been replaced by descriptive tables in the main document. This appendix provides a consolidated index of every diagram reference, its section, title, and intended content.

Table 15. 1 List of design diagrams.

No.	Diagram ID	Section	Diagram title	Description
1.	Figure 3. 1. 1	3.1	System's high-level architecture diagram.	Shows the browser applications communicating with the FastAPI backend through HTTPS, the backend accessing PostgreSQL, sensor nodes exchanging MQTT messages through the broker, and WebSocket updates returning to authenticated browser clients.
2.	Figure 3. 3. 1	3.3	System's context diagram.	Identifies the product boundary, hospital users, RFID-tagged assets, sensor nodes, hospital Wi-Fi, the MQTT broker, and the information exchanged with the system.
3.	Figure 3. 5. 1	3.5	System's communication overview diagram.	Summarizes the three primary communication paths: HTTPS/JSON for browser requests, MQTT for sensor-node events and commands, and WebSocket/WSS for live backend-to-browser updates.
4.	Figure 4. 1. 1. 1	4.1.1	System's registration desk firmware diagram.	Describes the Registration Desk Node, UHF RFID reader, Registration App, backend, database, and tag-scan path used during asset registration and deregistration.
5.	Figure 4. 1. 2. 1	4.1.2	System's checkpoint node firmware diagram.	Describes a Checkpoint Node publishing RFID detections and heartbeat messages through the MQTT broker for backend validation and location processing.
6.	Figure 4. 2. 1	4.2	System's backend and communication layer diagram.	Summarizes the backend boundary with PostgreSQL, MQTT processing, WebSocket broadcasting, and browser clients.
7.	Figure 4. 2. 7. 1	4.2.7	System's MQTT processing diagram.	Shows the backend MQTT client receiving node events, validating them, invoking services, and publishing supported commands and acknowledgments.
8.	Figure 4. 4. 1	4.4	Monitor app communication.	Shows the Monitor App loading data through HTTPS and receiving user-visible changes through WebSocket.
9.	Figure 4. 4. 7. 1	4.4.7	Movement request modal communication.	Shows the movement request modal invoking the backend movement API and receiving the updated request state.
10.	Figure 5. 2. 1	5.2	Identity and configuration logical ERD.	Logical relationship between User and System Setting, including the user who last updated a setting.
11.	Figure 5. 2. 2	5.2	Hospital, room, and node logical ERD.	Logical relationship among Hospital, Room, and Node, including hospital and room assignment for a sensor node.
12.	Figure 5. 2. 3	5.2	Asset location and detection logical ERD.	Logical relationship among Room, Node, Asset, and Detection Event for assigned location, current location, last detecting node, and detection history.
13.	Figure 5. 2. 4	5.2	Movement request logical ERD.	Logical relationship among User, Room, Asset, and Movement Request for requester, decision maker, source room, destination room, status, and timestamps.
14.	Figure 5. 2. 5	5.2	Asset and alert logical ERD.	Logical relationship among Asset, Node, and Alert for asset-related, node-related, and operational warning records.
15.	Figure 5. 2. 6	5.2	Activity event logical ERD.	Logical relationship among User, Node, Asset, and Activity Event for user-triggered and system-triggered operational history.
16.	Figure 5. 8. 1	5.8	Detection event and alert relationship.	Shows that a validated detection event may generate an alert when the observed condition requires user attention.

17.	Figure 6. 1. 1. 1	6.1.1	Node manager class.	The class that coordinates node setup, runtime behavior, communication, and configuration.
18.	Figure 6. 1. 1. 2	6.1.1	Node status LED class.	The status indications and the physical blink-identification responsibilities.
19.	Figure 6. 1. 1. 3	6.1.1	Node board control class.	Board-level control responsibilities and the reset or hardware-control boundary.
20.	Figure 6. 1. 1. 4	6.1.1	Node clock class.	The time abstraction used for intervals, timeouts, and heartbeat scheduling.
21.	Figure 6. 1. 1. 5	6.1.1	Node MQTT class dependencies.	The MQTT connection, publishing, subscribing, command, and acknowledgment responsibilities.
22.	Figure 6. 1. 1. 6	6.1.1	Node network config class dependencies.	The configuration responsibilities for the persistent Wi-Fi, broker, and node.
23.	Figure 6. 1. 1. 7	6.1.1	Node Wi-Fi class.	Wi-Fi connection, captive portal, and reconnection behavior.
24.	Figure 6. 1. 1. 8	6.1.1	Node RFID and config state relation with hardware serial dependency.	The RFID reader boundary, tag-reading flow, and scan-result handling.
25.	Figure 6. 1. 1. 9	6.1.1	Firmware main module coordination.	How the main firmware coordinates the network, MQTT, RFID, LED, clock, and board-control modules.
26.	Figure 6. 1. 1. 10	6.1.1	Firmware module dependencies.	Behavioral dependencies between the main firmware and its modules.
27.	Figure 6. 1. 2. 1	6.1.2	Bootstrap configuration structure.	The local configuration values required before the node can connect and identify itself.
28.	Figure 6. 1. 2. 2	6.1.2	Remote configuration structure.	Configuration values that were received or refreshed from the backend.
29.	Figure 6. 1. 2. 3	6.1.2	Firmware roles and state enumerations.	Important firmware roles, connection states, and configuration states.
30.	Figure 6. 1. 3. 1	6.1.3	MQTT interfaces.	The connector, publisher, and subscriber interfaces used by the MQTT implementation.
31.	Figure 6. 1. 3. 2	6.1.3	Wi-Fi interface.	The abstraction used for Wi-Fi status, connection, and setup behavior.
32.	Figure 6. 1. 3. 3	6.1.3	Network configuration interface.	The interface between runtime logic and stored network or node configuration.
33.	Figure 6. 1. 3. 4	6.1.3	RFID interface.	The abstraction used by the firmware to read RFID tags.
34.	Figure 6. 1. 3. 5	6.1.3	Status LED interface.	The status LED and blink control abstraction.
35.	Figure 6. 1. 3. 6	6.1.3	Clock interface.	The time and timer abstraction used by scheduling and timeout logic.
36.	Figure 6. 1. 3. 7	6.1.3	Board control interface.	The hardware-control abstraction used for board-level actions.
37.	Figure 6. 1. 3. 8	6.1.3	Firmware interface implementations.	The diagram below combines the main firmware interfaces and their concrete implementations.
38.	Figure 6. 2. 1. 1	6.2.1	Backend layer overview.	The relationships among API routes, core services, event processing, real-time communication, and database models.
39.	Figure 6. 2. 1. 2	6.2.1	Backend domain models.	The important persistent models and their principal fields and associations.
40.	Figure 6. 2. 2. 1	6.2.2	Authentication and user services.	Authentication dependencies, first-admin setup, user validation, password handling, and last-active-admin safety policy.
41.	Figure 6. 2. 2. 2	6.2.2	Node management services.	Assignment, validation, repository access, reference checks, heartbeat/offline monitoring, and demo-node policy.
42.	Figure 6. 2. 3. 1	6.2.3	Asset registration services.	The registration facade, validation, repository, Asset model, and Node dependency.
43.	Figure 6. 2. 3. 2	6.2.3	Asset movement services.	The creation, approval, rejection, cancellation, validation, execution, and persistence of movement.
44.	Figure 6. 2. 3. 3	6.2.3	Asset detection and flow resolution.	How an MQTT detection updates location, resolves flow state, completes approved movement, and writes events.
45.	Figure 6. 2. 4. 1	6.2.4	Backend MQTT processing.	MQTT settings, workers, routing, topic helpers, event handlers, publishing, asset location updates, and WebSocket broadcasting.

46.	Figure 6. 2. 4. 2	6.2.4	Authentication and user API schemas.	Groups the request and response schemas used for login, first-admin setup, token/session results, and user account information.
47.	Figure 6. 2. 4. 3	6.2.4	Node API schemas.	Groups node discovery, assignment, update, status, blink-command, and node response schemas.
48.	Figure 6. 2. 4. 4	6.2.4	Asset API schemas.	Groups asset registration, deregistration, list, location, scan, and asset response schemas.
49.	Figure 6. 2. 4. 5	6.2.4	Movement request API schemas.	Groups movement request creation, decision, cancellation, completion, and response schemas.
50.	Figure 6. 2. 4. 6	6.2.4	Configuration API schemas.	Groups system-setting and configuration request/response schemas used at the HTTP boundary.
51.	Figure 6. 3. 1. 1	6.3.1	Registration app architecture.	The application shell, pages, feature components, hooks, services, context, and external backend connection.
52.	Figure 6. 3. 1. 2	6.3.1	Registration app authentication and routing.	Route protection, authentication context, session helpers, login flow, role-aware navigation, and backend authentication service.
53.	Figure 6. 3. 2. 1	6.3.2	Registration dashboard components.	Dashboard page composition, role-aware cards, summary data, and navigation dependencies.
54.	Figure 6. 3. 2. 2	6.3.2	Node registration components.	The node page, toolbar, assignment form, node table, hooks, node API, status utilities, and live updates.
55.	Figure 6. 3. 2. 3	6.3.2	Asset registration and movement components.	The asset registration and deregistration forms, the asset table, the movement request panel, hooks, services, and real-time events.
56.	Figure 6. 3. 2. 4	6.3.2	User management components.	The user management page, the account form/table components, the user API, validation, and role protection.
57.	Figure 6. 3. 2. 5	6.3.2	Account security and settings components.	The account-security and system-setting pages, along with authentication, validation, API services, and shared controls.
58.	Figure 6. 3. 3. 1	6.3.3	Registration app API and real-time flow.	This diagram explains how registration-side features communicate with the backend. Feature-specific services call the shared API client, which attaches the current bearer token and applies common error handling. The WebSocket layer receives supported scan, asset, node, movement, alert, and activity events and passes them to the relevant hooks or page state. Initial and recovery data still come from HTTP APIs, so the WebSocket supplements rather than replaces the persistent backend source of truth.
59.	Figure 6. 4. 1. 1	6.4.1	Monitor app architecture.	The application shell, monitoring pages, feature components, hooks, services, utilities, and backend connection.
60.	Figure 6. 4. 1. 2	6.4.1	Monitor app authentication and routing.	Route protection, login/session handling, shared shell components, role checks, and navigation.
61.	Figure 6. 4. 2. 1	6.4.2	Monitor dashboard components.	Dashboard summaries, cards, hooks, filtered counts, and links to detailed monitoring pages.
62.	Figure 6. 4. 2. 2	6.4.2	Asset monitor components.	The asset page, table, badges, search, sorting, pagination, movement actions, hooks, and API services.
63.	Figure 6. 4. 2. 3	6.4.2	Movement request modal components.	The movement modal, selection and validation state, request/cancel actions, and movement API dependencies.
64.	Figure 6. 4. 2. 4	6.4.2	Node monitor components.	The node page, table, status badge, search, sorting, pagination, and node data hook.
65.	Figure 6. 4. 2. 5	6.4.2	Alerts page components.	Describes the Alerts page components, alert rows, filters, sorting, pagination, data hooks, and alert display utilities.

66.	Figure 6. 4. 2. 6	6.4.2	Activity page components.	Describes the Activity page components, activity rows, filters, sorting, pagination, data hooks, and date/text formatting utilities.
67.	Figure 6. 4. 3. 1	6.4.3	Monitor data orchestration.	How monitor data hooks coordinate initial API loading, local state, WebSocket events, alerts, asset flow, sorting, and pagination.
68.	Figure 6. 4. 3. 2	6.4.3	Monitor API and WebSocket infrastructure.	API client configuration, authentication headers, monitor API modules, socket lifecycle, and backend boundaries.
69.	Figure 7. 1. 1	7.1.1	User login sequence.	The login request, backend verification, token response, and role-based result.
70.	Figure 7. 1. 2	7.1.2	Registration dashboard loading sequence.	Initial data loading for the registration-side dashboard.
71.	Figure 7. 1. 3	7.1.3	Assign node sequence.	How to assign a discovered node to a role/location.
72.	Figure 7. 1. 4	7.1.4	Identify node blink sequence.	Backend command publishing and node acknowledgment.
73.	Figure 7. 1. 5	7.1.5	Registration desk scan sequence.	The registration node scans via MQTT and the UI updates.
74.	Figure 7. 1. 6	7.1.6	Register asset sequence.	The user submits asset registration after a scanned tag is available.
75.	Figure 7. 1. 7	7.1.7	Request asset movement sequence.	How movement requests are created in the Monitor App.
76.	Figure 7. 1. 8	7.1.8	Approve movement request sequence.	Registration-side approval and live update.
77.	Figure 7. 1. 9	7.1.9	RFID detection location update sequence.	Checkpoint detection and asset location update.
78.	Figure 7. 1. 10	7.1.10	Approved movement completion sequence.	Approved movement completion at the destination node.
79.	Figure 7. 1. 11	7.1.11	Node heartbeat sequence.	Heartbeat publishing and node status update.
80.	Figure 7. 1. 12	7.1.12	Offline node detection sequence.	Backend offline timeout detection.
81.	Figure 7. 2. 1. 1	7.2.1.1	Main firmware lifecycle state machine.	The node startup and main firmware lifecycle.
82.	Figure 7. 2. 1. 2	7.2.1.2	Captive portal state machine.	The configuration portal behavior when setup is needed.
83.	Figure 7. 2. 1. 3	7.2.1.3	Provisioning state machine.	How node provisioning/configuration is handled.
84.	Figure 7. 2. 1. 4	7.2.1.4	MQTT connection state machine.	The MQTT connection, reconnect, and subscription states.
85.	Figure 7. 2. 1. 5	7.2.1.5	MQTT blink command state machine.	How a blink command is received, executed, and acknowledged.
86.	Figure 7. 2. 1. 6	7.2.1.6	RFID scan state machine.	Tag-reading and scan-publication behavior.
87.	Figure 7. 2. 1. 7	7.2.1.7	Heartbeat publish state machine.	Heartbeat publish timing and status reporting.
88.	Figure 7. 2. 1. 8	7.2.1.8	Remote configuration refresh state machine.	A periodic or requested remote configuration refresh.
89.	Figure 7. 2. 1. 9	7.2.1.9	Reset button state machine.	Reset/configuration button behavior.
90.	Figure 7. 2. 1. 10	7.2.1.10	LED state machine.	LED states for normal status and identification behavior.
91.	Figure 7. 2. 2. 1	7.2.2.1	Node lifecycle state machine.	DISCOVERED, ASSIGNED, ONLINE, OFFLINE, DISABLED, and DELETED, including heartbeat, timeout, assignment, enable/disable, unassign, and deletion transitions.
92.	Figure 7. 2. 2. 2	7.2.2.2	Asset lifecycle state machine.	The UNREGISTERED, ACTIVE, and DEREGISTERED states, with transitions for registration, deregistration, and re-registration.
93.	Figure 7. 2. 2. 3	7.2.2.3	Asset flow state machine.	ON THE MOVE, IN PLACE, PENDING APPROVAL, and UNAUTHORIZED, including registration, request, approval/rejection, expected-room detection, wrong-location detection, and return transitions.
94.	Figure 7. 2. 2. 4	7.2.2.4	Movement request state machine.	PENDING, APPROVED, REJECTED, CANCELLED, and COMPLETED, including the required destination detection after approval.
95.	Figure 7. 2. 2. 5	7.2.2.5	Authentication and session state machine.	SIGNED OUT, AUTHENTICATING, AUTHENTICATED, TOKEN EXPIRED, and ACCESS DENIED, with login, rejection, logout, expiry, and session-cleanup transitions.
96.	Figure 7. 2. 2. 6	7.2.2.6	WebSocket connection state machine.	DISCONNECTED, CONNECTING, CONNECTED, RECONNECTING, and STOPPED, including valid-session checks, failure retry, recovery, logout, and token invalidation.

97.	Figure 7. 2. 2. 7	7.2.2.7	Alert lifecycle state machine.	OPEN and RESOLVED states, with alert creation and condition-cleared/resolution transitions, and includes examples of alert-triggering conditions.
98.	Figure 9. 1. 1	9.1	Hardware wiring overview.	Textual wiring overview covering the ESP32, UHF RFID reader serial interface, power source, status LEDs, and reset/configuration control.
99.	Figure 12. 1. 1	12.1	Deployment overview.	Deployment topology for browser clients, frontend applications, FastAPI backend, PostgreSQL database, MQTT broker, hospital network, and physical sensor nodes.
100.	Figure 12. 4. 1	12.4	Frontend deployment.	Deployment of the Registration App and Monitor App as production web builds loaded by authenticated browser clients.
101.	Figure 12. 5. 1	12.5	Backend deployment.	Deployment of the FastAPI backend with PostgreSQL access, MQTT client processing, WebSocket broadcasting, and browser-facing HTTPS APIs.
102.	Figure 12. 9. 1	12.9	Firmware deployment.	Deployment of ESP32 firmware with the UHF RFID reader, status LEDs, Wi-Fi connectivity, MQTT broker configuration, and node-specific settings.

15.2. Status Value Reference

This reference separates values by the object or connection that owns the state. It prevents movement-request statuses from being confused with asset-flow states and distinguishes stored values from frontend fallbacks.

Table 15. 2 List of system state status values.

No.	Category	Value	Meaning	Note
1.	Node lifecycle/ status	DISCOVERED	The backend has accepted a previously unknown node.	Normal stored or derived state.
2.		ASSIGNED	The node has an approved role and location configuration.	Normal stored state.
3.		ONLINE	A recent valid heartbeat confirms availability.	Operational state.
4.		OFFLINE	The heartbeat timeout has been exceeded.	Operational state.
5.		DISABLED	The node has been administratively disabled.	Normal stored state.
6.		DELETED	The eligible node record has been removed.	Terminal lifecycle outcome.
7.	Node display fallback	UNKNOWN	The frontend received an unexpected status value.	Fallback only; not a normal persisted value.
8.	Asset lifecycle	UNREGISTERED	No active tracked asset record exists.	Conceptual pre-registration state.
9.		ACTIVE	The asset is registered and actively tracked.	Persistent active state.
10.		DEREGISTERED	The asset is no longer actively tracked; history remains.	Persistent inactive/history state.
11.	Asset flow	ON_THE_MOVE	The asset awaits detection at its expected or approved destination.	Displayed flow state.
12.		IN_PLACE	The latest valid detection matches the expected room.	Displayed flow state.
13.		PENDING_APPROVAL	A movement request awaits a decision.	Displayed flow state.
14.		UNAUTHORIZED	The latest location does not match the expected or valid approved destination.	Displayed flow state; may create an alert.
15.	Movement request	PENDING	The request awaits approval, rejection, or eligible cancellation.	Persistent request status.
16.		APPROVED	Movement is authorized but not complete.	Persistent request status.
17.		REJECTED	The request was denied.	Terminal request status.
18.		CANCELLED	The eligible request was canceled.	Terminal request status.
19.		COMPLETED	The asset was detected at the approved destination.	Terminal request status.
20.	Alert	OPEN	The warning condition requires attention.	Persistent alert status.
21.		RESOLVED	The underlying condition has cleared or been resolved.	Persistent alert status.
22.	Session	SIGNED_OUT	No authenticated browser session exists.	Frontend session state.
23.		AUTHENTICATING	Credentials are being verified.	Frontend session state.
24.		AUTHENTICATED	The backend accepted the account and role.	Frontend session state.
25.		TOKEN_EXPIRED	The access token is no longer valid.	Failure/cleanup state.
26.		ACCESS_DENIED	The account lacks permission for the requested application or action.	Failure/cleanup state.

27.	WebSocket	DISCONNECTED	No active live-update connection exists.	Frontend connection state.
28.		CONNECTING	The client is attempting to open a connection.	Frontend connection state.
29.		CONNECTED	The live-update connection is active.	Frontend connection state.
30.		RECONNECTING	A temporary interruption is being retried.	Frontend connection state.
31.		STOPPED	Reconnection is intentionally disabled after logout, invalidation, or shutdown.	Frontend connection state.

15.3. MQTT Topic Reference

The topic patterns below are the canonical patterns documented by this design. Firmware and backend constants must use the same namespace, node identifier, event category, payload fields, and acknowledgment contract.

Table 15. 3 System's MQTT communication reference list.

No.	Message category	Direction	Topic pattern	Principal payload fields	Processing rule
1.	Registration scan	Node → broker → backend	hospital/{hospital_id}/nodes/{node_id}/registration/scan	• Node ID • Tag ID • Timestamp • Message type	Published by a Registration Desk Node after a valid tag observation.
2.	Checkpoint detection	Node → broker → backend	hospital/{hospital_id}/nodes/{node_id}/rfid/detection	• Node ID • Tag ID • Detected time • Reader status	Published by a Checkpoint Node and validated before location processing.
3.	Heartbeat	Node → broker → backend	hospital/{hospital_id}/nodes/{node_id}/heartbeat	• Node ID • Firmware version • Online/runtime status • Timestamp	Updates last-ping information and supports offline detection.
4.	Backend command	Backend → broker → node	hospital/{hospital_id}/nodes/{node_id}/commands	• Command type • Command ID • Target node ID • Command parameters	Used for supported node commands such as physical blink identification.
5.	Command acknowledgment	Node → broker → backend	hospital/{hospital_id}/nodes/{node_id}/commands/ack	• Command ID • Node ID • Success/failure result • Message	Confirms command processing or reports controlled failure.

15.4. WebSocket Event Reference

The event names below are the names explicitly documented in this design. Exact payload fields and any separate registration-scan or alert-resolution event names must remain synchronized with backend broadcaster and frontend socket constants.

Table 15. 4 List of WebSocket event references.

No.	Event type	Typical producer	Consumer	Purpose
1.	asset_updated	- Asset registration service - Movement service - Checkpoint detection processing	Registration App Asset Registration page; Monitor App Dashboard; Monitor App Assets page	Refreshes or merges changed asset identity, location, flow, activity-related, or movement-related information.
2.	node_updated	- Node service - Heartbeat processing - Offline monitor	Registration App Node Registration page; Monitor App Dashboard; Monitor App Nodes page	Updates assignment, role, location, online/offline, enabled/disabled, and last-ping information.
3.	alert_updated	- Alert service after an unknown tag - Offline node - Unauthorized movement - Disconnected node - Detect an unidentified node	Monitor App Dashboard; Monitor App Alerts page	Adds a new operational warning to visible state. Alert-resolution updates must use the implemented event contract or an API refresh.
4.	activity_updated	Activity logging service	Monitor App Activity page	Adds a user-visible operational-history record while excluding excessive heartbeat noise.
5.	movement_request_updated	Movement request service	Registration App Movement Requests area; Monitor App Assets page	Updates request creation, cancellation, approval, rejection, and completion results.

15.5. API Endpoint Summary

This appendix summarizes API capabilities, authorization boundaries, and purposes. Exact HTTP methods and route paths are intentionally not invented here; the final route-path list must be generated from the current FastAPI OpenAPI schema and synchronized with the source code.

Table 15. 5 List of API Endpoints.

No.	API area	Operation	Authorized actor or role	Purpose
1.	Authentication and session	First-admin setup	Unauthenticated setup condition only	Creates the first Admin account only when no administrator has ever been created.
2.		Login	Permitted active roles	Verifies credentials and account status and returns token/session information.
3.		Logout/session clearing	Authenticated user	Clears browser session state; server-side token revocation depends on the implemented token model.
4.		Change own password	Authenticated user	Validates current/new credentials and updates the stored password hash.
5.	Users	List and inspect users	Admin	Returns account records required for user management.
6.		Create user	Admin	Creates an account with an allowed role and active state.
7.		Update role or active state.	Admin	Applies administrator safety rules before changing another account.
8.	Nodes	List nodes	Admin, Technician, and authorized monitoring roles according to route policy	Returns discovered and configured node data.
9.		Assign or edit node.	Admin or Technician	Validates stable node ID, role, hospital, room, and current assignment state.
10.		Unassign, enable, disable, or delete eligible node	Admin or Technician according to policy	Applies lifecycle, reference, and safety rules.
11.		Blink node	Admin or Technician according to policy	Publishes a node command and reports acknowledgment or timeout.
12.	Assets	List and inspect assets	Authorized Registration and Monitor App roles	Returns identity, assigned/current room, flow status, and latest detection information.
13.		Register asset	Admin or Registration Staff according to policy	Creates an active asset after a valid registration scan, tag uniqueness check, room selection, and eligible registration-node check.
14.		Deregister asset	Admin or Registration Staff according to policy	Ends active tracking while preserving history and tag-reuse rules.
15.	Movement requests	Create movement request	Admin or Monitor Staff according to policy	Creates a PENDING request when no conflicting active movement workflow exists.
16.		Cancel eligible request	Requesting/authorized monitoring-side role according to policy	Moves an eligible request to CANCELLED without changing an approved/completed workflow.
17.		Approve or reject request.	Admin or Registration Staff according to policy	Records the decision and updates the request and asset flow state.
18.		Complete from checkpoint detection.	Backend internal processing	Completes an APPROVED request only when detection occurs at the approved destination.
19.	Alerts	List alerts	Authorized monitoring roles	Returns open/resolved alerts with filtering, searching, sorting, and pagination as implemented.
20.	Activity	List activity	Authorized monitoring roles	Returns user-visible operational history while excluding or filtering heartbeat noise.
21.	Settings	Read or update system settings	Admin according to policy	Reads or updates supported configuration values and records the updating user where implemented.